```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py                启动并自动打开浏览器
    python run.py --no-browser   只启动服务，不打开浏览器
    python run.py --dev          开发模式（开启代码热重载与调试器）
    python run.py --port 5001    指定端口
默认以 127.0.0.1:5000 启动，就绪后自动打开浏览器。
"""
import os
import sys
import threading
import time
import webbrowser
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")   # 控制台中文正常显示
from app import create_app
from app.models import db
PORT = 5000
for i, a in enumerate(sys.argv):
    if a == "--port" and i + 1 < len(sys.argv):
        PORT = int(sys.argv[i + 1])
URL = "http://127.0.0.1:%d" % PORT
NO_BROWSER = "--no-browser" in sys.argv
DEV = "--dev" in sys.argv
app = create_app()
def open_browser_later(delay=1.2):
    """等服务工作起来再打开浏览器，避免打开时页面还没就绪。
    放在后台线程里，不用 sleep 阻塞服务启动；线程设为守护线程，
    用户 Ctrl+C 停止服务时不会残留。
    """
    def run():
        time.sleep(delay)
        try:
            webbrowser.open(URL)
        except Exception:
            pass            # 打开失败不影响服务本身
    t = threading.Thread(target=run, daemon=True)
    t.start()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()     # 首次运行时自动建表
        from app.utils.schema import ensure_schema
        added = ensure_schema(db)     # 旧库补齐新增字段
        if added:
            print("已补充字段：%s" % "、".join(added))
    print("=" * 58)
    print("   历史遗留废弃矿山自然恢复适宜性评价系统  Ｖ１．０")
    print("=" * 58)
```

```
    print("   访问地址：%s" % URL)
    print("   演示账号：admin      密码：admin123")
    print("   停止运行：按 Ctrl+C，或直接关闭本窗口")
    print("=" * 58)
    if not NO_BROWSER and not DEV:
        print("   正在打开浏览器……")
    print()
    #  开发模式才启用热重载（重载会二次启动进程，故此时不自动开浏览器）
    if not NO_BROWSER and not DEV:
        open_browser_later()
    app.run(host="127.0.0.1", port=PORT, debug=DEV,
    use_reloader=DEV)
#  ==== app/__init__.py ====
#  -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from flask import Flask
from flask_login import LoginManager
from app.config import Config
from app.models import db, User
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    #  注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
    app.register_blueprint(constraint.bp)
    app.register_blueprint(ahp.bp)
    app.register_blueprint(indicator.bp)
    app.register_blueprint(evaluate.bp)
    app.register_blueprint(report.bp)
    #  全局登录保护：除登录页与静态资源外，均需登录
    from flask import request, redirect, url_for
    from flask_login import current_user, login_user
    PUBLIC_ENDPOINTS = {"auth.login", "static"}
    @app.before_request
    def require_login():
        #  截图模式（仅用于生成说明书配图）：自动以管理员身份登录
```

```
    if app.config.get("SCREENSHOT_MODE"):
        if (request.endpoint not in ("auth.login",
"static")
                and not current_user.is_authenticated):
            admin = User.query.filter_by(role="admin").f
irst()
            if admin:
                login_user(admin)
        return None
    if request.endpoint in PUBLIC_ENDPOINTS or
request.endpoint is None:
        return None
    if not current_user.is_authenticated:
        return redirect(url_for("auth.login"))
    return None
# 侧边栏导航：按论文评价流程的顺序编排，分为「评价流程」与「系统管理」两组。
# 不使用模块编号，避免出现与实际流程不一致的代号。
@app.context_processor
def inject_nav():
    path = request.path or "/"
    flow = [
        ("项目与矿山档案", "/project/", path.startswith("/proje
ct")),
        ("政策与标准知识库", "/standard/",
path.startswith("/standard")),
        ("限制性指标判定", "/constraint/",
path.startswith("/constraint")),
        ("微地质单元", "/unit/", path.startswith("/unit")),
        ("指标体系与权重", "/ahp/", path.startswith("/ahp")),
        ("指标数据采集", "/indicator/",
path.startswith("/indicator")),
        ("适宜性定量评价", "/evaluate/",
path.startswith("/evaluate")),
        ("报告与结果输出", "/report/", path.startswith("/report
")),
    ]
    admin = [
        ("用户与操作日志", "/users", path.startswith(("/users",
"/logs"))),
    ]
    groups = [
        {"title": "评价流程", "links": flow},
        {"title": "系统管理", "links": admin},
    ]
    # 层级引导用：项目是归档单位，图斑才是评价对象。
    # 下游各页面按图斑组织数据，因此需要提示「哪些项目下还没有图斑」。
    counts = {"project_count": 0, "patch_count": 0,
              "projects_without_patch": []}
    if current_user.is_authenticated:
        from app.models import Project, MinePatch
```

```
            projects = Project.query.order_by(Project.id).al
    l()
            counts["project_count"] = len(projects)
            counts["patch_count"] = MinePatch.query.count()
            counts["projects_without_patch"] = [
                p for p in projects if p.patches.count() ==
    0]
        counts["nav_groups"] = groups
        counts["home_active"] = path == "/"
        return counts
    # 操作审计：统一在响应阶段记录成功的写操作（操作日志）
    from app.services import audit
    @app.after_request
    def audit_writes(response):
        if request.method in ("POST", "PUT", "PATCH",
    "DELETE"):
            audit.record(request.endpoint,
    response.status_code)
        return response
    # 错误页：给出可读提示，而不是默认的英文错误页
    from flask import render_template
    @app.errorhandler(403)
    def forbidden(e):
        return render_template("error.html", code=403,
                               message="没有权限执行该操作"), 403
    @app.errorhandler(404)
    def not_found(e):
        return render_template("error.html", code=404,
                               message="请求的页面不存在"), 404
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY",
    "mine-restoration-dev-key")
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
    # 影像与数据文件上传（遥感影像挂接、批量导入）
```

```
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024          # 单次上传上限
    32 MB
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
    ALLOWED_DATA_EXT = {"csv", "xlsx", "xls"}
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from datetime import datetime
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（用户与权限管理）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 角色：管理员 admin / 评价员 evaluator / 查看者 viewer
    role = db.Column(db.String(20), nullable=False,
    default="viewer")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
```

```
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, index=True)
class Project(db.Model):
    """评价项目（项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    patches = db.relationship("MinePatch",
    backref="project", lazy="dynamic",
                              cascade="all, delete-orphan")
class MinePatch(db.Model):
    """矿山图斑。"""
    __tablename__ = "mine_patches"
    id = db.Column(db.Integer, primary_key=True)
    project_id = db.Column(db.Integer,
    db.ForeignKey("projects.id"), nullable=False)
    name = db.Column(db.String(128), nullable=False)
    admin_region = db.Column(db.String(128), nullable=True)
    # 所在行政区
    center_x = db.Column(db.Float, nullable=True)
    # 中心经度 / 横坐标
    center_y = db.Column(db.Float, nullable=True)
    # 中心纬度 / 纵坐标
    area_ha = db.Column(db.Float, nullable=True)
    # 图斑面积（公顷）
    mineral_type = db.Column(db.String(64), nullable=True)
    # 开采矿种
    mining_method = db.Column(db.String(64), nullable=True)
    # 开采方式
    stop_years = db.Column(db.Float, nullable=True)
    # 停产年限
    responsible_entity_missing = db.Column(db.Boolean,
    default=True)  # 责任主体是否缺失
    # 遥感底图 / 无人机正射影像挂接
    image_path = db.Column(db.String(256), nullable=True)
    # 相对 static 的路径
    image_note = db.Column(db.String(256), nullable=True)
    # 影像说明（如来源、时相）
    image_uploaded_at = db.Column(db.DateTime,
    nullable=True)
    created_at = db.Column(db.DateTime,
```

```
    default=datetime.utcnow)                                          301
    units = db.relationship("MicroUnit", backref="patch",             302
    lazy="dynamic",                                                   303
                                    cascade="all, delete-orphan")     304
class ReferenceEcosystem(db.Model):                                   305
    """参照生态系统：标准化基准值，与图斑绑定。"""                    306
    __tablename__ = "reference_ecosystems"                            307
    id = db.Column(db.Integer, primary_key=True)                      308
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat          309
    ches.id"), nullable=False)                                        310
    coverage_benchmark = db.Column(db.Float, default=75.0)            311
    # 植被覆盖度基准 %                                                312
    shannon_benchmark = db.Column(db.Float, default=0.7)              313
    # Shannon-Wiener 基准                                             314
    dominant_species = db.Column(db.String(256),                      315
    nullable=True)  # 建群种                                         316
    soil_benchmark = db.Column(db.String(256),                        317
    nullable=True)     # 土壤基准                                     318
class Standard(db.Model):                                             319
    """地方标准 / 政策文件（政策与标准知识库）。"""                   320
    __tablename__ = "standards"                                       321
    id = db.Column(db.Integer, primary_key=True)                      322
    province = db.Column(db.String(32), nullable=False)               323
    # 省份                                                            324
    name = db.Column(db.String(256), nullable=False)                  325
    # 名称                                                            326
    doc_number = db.Column(db.String(128), nullable=True)             327
    # 标准编号 / 文号                                                 328
    publish_date = db.Column(db.String(32), nullable=True)            329
    implement_date = db.Column(db.String(32), nullable=True)          330
    # 类别：地方标准 / 政策文件                                       331
    category = db.Column(db.String(32), nullable=False,               332
    default="地方标准")                                               333
    summary = db.Column(db.Text, nullable=True)                       334
    # 特点概述                                                        335
    rules = db.relationship("StandardRule",                           336
    backref="standard",                                               337
                                    cascade="all, delete-orphan")     338
class StandardRule(db.Model):                                         339
    """结构化规则：将标准条文拆解为可执行规则。"""                    340
    __tablename__ = "standard_rules"                                  341
    id = db.Column(db.Integer, primary_key=True)                      342
    standard_id = db.Column(db.Integer,                               343
    db.ForeignKey("standards.id"), nullable=False)                    344
    item = db.Column(db.String(128), nullable=False)                  345
    # 判定项                                                          346
    operator = db.Column(db.String(8), nullable=False)                347
    # 比较符：> >= < <= ==                                            348
    threshold = db.Column(db.String(64), nullable=True)               349
    # 阈值                                                            350
```

```
    condition_type = db.Column(db.String(32),                    351
    default="必要条件")    # 必要条件 / 充分条件                    352
    region = db.Column(db.String(64), nullable=True)             353
    # 适用分区（宁夏分区差异化）                                    354
    note = db.Column(db.Text, nullable=True)                     355
class ConstraintCheck(db.Model):                                 356
    """限制性判定记录。"""                                         357
    __tablename__ = "constraint_checks"                          358
    id = db.Column(db.Integer, primary_key=True)                 359
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat     360
    ches.id"), nullable=False)                                   361
    policy_pass = db.Column(db.Boolean, nullable=True)           362
    # 政策约束                                                     363
    location_pass = db.Column(db.Boolean, nullable=True)         364
    # 区位约束                                                     365
    pollution_pass = db.Column(db.Boolean, nullable=True)        366
    # 土壤污染                                                     367
    geo_pass = db.Column(db.Boolean, nullable=True)              368
    # 地质灾害                                                     369
    farmland_pass = db.Column(db.Boolean, nullable=True)         370
    # 耕地属性                                                     371
    passed = db.Column(db.Boolean, nullable=True)                372
    # 总体是否通过                                                 373
    reason = db.Column(db.Text, nullable=True)                   374
    # 不通过原因                                                   375
    checked_at = db.Column(db.DateTime,                          376
    default=datetime.utcnow)                                     377
class MicroUnit(db.Model):                                       378
    """微地质单元：岩质边坡 / 碎石边坡 / 平台。"""                  379
    __tablename__ = "micro_units"                                380
    id = db.Column(db.Integer, primary_key=True)                 381
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat     382
    ches.id"), nullable=False)                                   383
    unit_type = db.Column(db.String(32), nullable=False)         384
    # 单元类型                                                     385
    area = db.Column(db.Float, nullable=True)                    386
    # 面积                                                         387
    ratio = db.Column(db.Float, nullable=True)                   388
    # 面积占比（自动计算）                                         389
    description = db.Column(db.Text, nullable=True)              390
    # 现场描述                                                     391
class Indicator(db.Model):                                       392
    """指标定义：指标树，含角色与启用单元类型。"""                  393
    __tablename__ = "indicators"                                 394
    id = db.Column(db.Integer, primary_key=True)                 395
    code = db.Column(db.String(32), unique=True,                 396
    nullable=False)    # 指标编码                                397
    name = db.Column(db.String(64), nullable=False)              398
    # 指标名称                                                     399
    level = db.Column(db.Integer, nullable=False)                400
```

```
    #  1/2/3 级                                                                    401
    parent_code = db.Column(db.String(32), nullable=True)                         402
    #  父指标编码                                                                  403
    #  角色：土壤污染指标同时作为限制性指标与评价指标                              404
    is_constraint = db.Column(db.Boolean, default=False)                          405
    is_evaluative = db.Column(db.Boolean, default=True)                           406
    unit = db.Column(db.String(32), nullable=True)                                407
    #  单位                                                                        408
    ideal_value = db.Column(db.String(64), nullable=True)                         409
    #  理想值（可配置）                                                            410
    ideal_type = db.Column(db.String(32), nullable=True)                          411
    #  benefit/cost/mid/discrete                                                  412
    #  启用该指标的微地质单元（逗号分隔）。岩质边坡不启用土壤指标、平台不启用地形指标  413
    enabled_units = db.Column(db.String(128), nullable=True)                      414
    description = db.Column(db.Text, nullable=True)                               415
    #  取值依据说明                                                                416
    def units_list(self):                                                         417
        return [s for s in (self.enabled_units or                                 418
    "").split(",") if s]                                                          419
class WeightScheme(db.Model):                                                     420
    """权重方案：按微地质单元类型分别保存。"""                                    421
    __tablename__ = "weight_schemes"                                              422
    id = db.Column(db.Integer, primary_key=True)                                  423
    unit_type = db.Column(db.String(32), nullable=False)                          424
    #  所属单元类型                                                                425
    name = db.Column(db.String(64), nullable=True)                                426
    #  方案名称                                                                    427
    level = db.Column(db.Integer, nullable=False,                                 428
    default=1)          #  1=一级指标  2=二级指标                                 429
    is_default = db.Column(db.Boolean, default=False)                             430
    #  是否默认（论文表 3）                                                        431
    created_at = db.Column(db.DateTime,                                           432
    default=datetime.utcnow)                                                      433
class WeightItem(db.Model):                                                       434
    """权重明细：方案内的各项权重。"""                                            435
    __tablename__ = "weight_items"                                                436
    id = db.Column(db.Integer, primary_key=True)                                  437
    scheme_id = db.Column(db.Integer,                                             438
    db.ForeignKey("weight_schemes.id"), nullable=False)                           439
    indicator_code = db.Column(db.String(32),                                     440
    nullable=False)         #  指标编码                                           441
    weight = db.Column(db.Float, nullable=False)                                  442
    #  权重值                                                                      443
class ExpertMatrix(db.Model):                                                     444
    """专家判断矩阵：多专家打分的原始矩阵。"""                                    445
    __tablename__ = "expert_matrices"                                             446
    id = db.Column(db.Integer, primary_key=True)                                  447
    scheme_id = db.Column(db.Integer,                                             448
    db.ForeignKey("weight_schemes.id"), nullable=True)                            449
    #  直接标注适用对象，便于无需先建权重方案即可录入专家矩阵                      450
```

```
    unit_type = db.Column(db.String(32), nullable=True)
    # 微地质单元类型
    l1_name = db.Column(db.String(64), nullable=True)
    # 所属一级指标（二级矩阵用）
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 矩阵对应指标层级
    matrix_json = db.Column(db.Text, nullable=False)
    # 判断矩阵（JSON 二维数组）
    cr = db.Column(db.Float, nullable=True)
    # 该专家矩阵的一致性比例
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class IndicatorData(db.Model):
    """指标实测数据：按微地质单元组织。"""
    __tablename__ = "indicator_data"
    id = db.Column(db.Integer, primary_key=True)
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni
    ts.id"), nullable=False, unique=True)
    soil_ratio = db.Column(db.Float, nullable=True)
    # 土石比
    thickness = db.Column(db.Float, nullable=True)
    # 土层厚度 cm
    ph = db.Column(db.Float, nullable=True)
    organic_matter = db.Column(db.Float, nullable=True)
    # 有机质 g/kg
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null
    heavy_metals = db.Column(db.Text, nullable=True)
    coverage = db.Column(db.Float, nullable=True)
    # 植被覆盖度 %
    shannon = db.Column(db.Float, nullable=True)
    # Shannon-Wiener 多样性指数
    slope = db.Column(db.Float, nullable=True)
    # 坡度 °
    aspect = db.Column(db.String(16), nullable=True)
    # 坡向：阳坡/阴阳坡/阴坡
    def heavy_metals_dict(self):
        return json.loads(self.heavy_metals) if
    self.heavy_metals else {}
class EvaluationResult(db.Model):
    """评价结果。"""
    __tablename__ = "evaluation_results"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    final_score = db.Column(db.Float, nullable=True)
    # 最终评价因子 F
    grade = db.Column(db.String(32), nullable=True)
    # 适宜性等级
```

```
    measures = db.Column(db.Text, nullable=True)
    # 推荐措施
    unit_scores = db.Column(db.Text, nullable=True)
    # 各单元得分 JSON
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class ScoreDetail(db.Model):
    """得分明细：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import DataRequired, Length
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES
class AHPSelectForm(FlaskForm):
    """选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
```

```
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """登录表单。支持「记住我」以在关闭浏览器后保持登录状态。"""
    username = StringField("用户名", validators=[DataRequired(), Length(max=64)])
    password = PasswordField("密码", validators=[DataRequired()])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
# ==== app/utils/constants.py ====
# -*- coding: utf-8 -*-
"""评价指标常量表。
全部数值取自论文，作为系统计算的唯一基准：
- 论文 4.2.2 节：指标标准化理想极值与取值依据
- 论文 4.2.3 节 / 表 4：适宜性等级划分
- 论文表 3：按微地质单元分别确定的一级、二级指标权重
- 论文 4.2.1 节：平均随机一致性指标 RI
"""
# ------------------------------------------------------------------
# 一、微地质单元类型
# ------------------------------------------------------------------
# 论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")
# 每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
UNIT_ENABLED_INDICATORS = {
    "岩质边坡": ("植被指标", "地形指标"),                    # 不设土壤指标
    "碎石边坡": ("土壤指标", "植被指标", "地形指标"),
    "平台": ("土壤指标", "植被指标"),                        # 不设地形指标
}
# ------------------------------------------------------------------
# 二、指标标准化理想极值（论文 4.2.2 节）
# ------------------------------------------------------------------
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 （cm）
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）
IDEAL_ORGANIC = 30.0            # 有机质理想值
    （g/kg），全国第二次土壤普查养分等级划分标准二等最低值
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 （%）
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统
# 重金属极值 （mg/kg），《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值
# 论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",
```

```
    "Zn")
HEAVY_METAL_LIMIT = {
    "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,
    "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,
}
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0
# 坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号
SLOPE_BREAK = 25.0
# 坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")
# pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
PH_TOL_LOW = 4.0
PH_TOL_HIGH = 10.0
# ---------------------------------------------------------------------
    ------------------
# 三、适宜性等级划分（论文表 4）
# ---------------------------------------------------------------------
    ------------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按论文表 4 判定适宜性等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ---------------------------------------------------------------------
    ------------------
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2，3，4，5，其余为常用扩展值）
# ---------------------------------------------------------------------
    ------------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
```

```
    1.24,                                                                    651
                7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}                         652
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）                  653
CR_THRESHOLD = 0.1                                                           654
# ------------------------------------------------------------------------   655
    ----------                                                               656
# 五、默认权重方案（论文表 3）                                                657
# ------------------------------------------------------------------------   658
    ----------                                                               659
# 论文 4.2.2 节："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，  660
# 分别计算评价指标权重"（论文摘要、图 2 流程与表 5 均作"碎石边坡"，           661
# 系统统一采用"碎石边坡"）。权重由 10 位专家打分后经 AHP 方法确定。           662
# 下表为论文表 3 所载的归一化权重，作为系统默认方案。                         663
DEFAULT_WEIGHTS = {                                                          664
    "岩质边坡": {                                                            665
        "l1": {"植被指标": 0.45, "地形指标": 0.55},                          666
        "l2": {                                                              667
            "植被指标": {"植被覆盖度": 0.67, "生物多样性": 0.33},            668
            "地形指标": {"坡度": 0.64, "坡向": 0.36},                        669
        },                                                                   670
    },                                                                       671
    "碎石边坡": {                                                            672
        "l1": {"土壤指标": 0.44, "植被指标": 0.31, "地形指标": 0.25},        673
        "l2": {                                                              674
            "土壤指标": {"土石比": 0.25, "土层厚度": 0.25, "pH": 0.17,       675
                         "有机质": 0.17, "污染指标": 0.17},                  676
            "植被指标": {"植被覆盖度": 0.64, "生物多样性": 0.36},            677
            "地形指标": {"坡度": 0.67, "坡向": 0.33},                        678
        },                                                                   679
    },                                                                       680
    "平台": {                                                                681
        "l1": {"土壤指标": 0.54, "植被指标": 0.46},                          682
        "l2": {                                                              683
            "土壤指标": {"土石比": 0.25, "土层厚度": 0.25, "pH": 0.17,       684
                         "有机质": 0.17, "污染指标": 0.17},                  685
            "植被指标": {"植被覆盖度": 0.61, "生物多样性": 0.39},            686
        },                                                                   687
    },                                                                       688
}                                                                            689
# ------------------------------------------------------------------------   690
    ----------                                                               691
# 六、限制性指标（论文表 2 + 4.1 节流程）                                     692
# ------------------------------------------------------------------------   693
    ----------                                                               694
# 论文摘要与 4.1 节：四项限制性指标；流程段另明确"是否为耕地"为约束条件       695
CONSTRAINT_ITEMS = ("政策", "区位约束", "土壤污染", "地质灾害", "耕地属性")   696
# ==== app/utils/schema.py ====                                              697
# -*- coding: utf-8 -*-                                                      698
"""轻量结构迁移。                                                            699
SQLite 下 `db.create_all()` 只建缺失的表，不会给已有表补列。                 700
```

```
本模块在启动时检查目标列是否存在，缺失则 ALTER TABLE 补上，
使旧数据库无需重建即可用上新字段。
"""
from sqlalchemy import text
# （表名， 列名， 列定义）
REQUIRED_COLUMNS = [
    # 遥感影像挂接
    ("mine_patches", "image_path", "VARCHAR(256)"),
    ("mine_patches", "image_note", "VARCHAR(256)"),
    ("mine_patches", "image_uploaded_at", "DATETIME"),
    # 指标维护
    ("indicators", "enabled_units", "VARCHAR(128)"),
    ("indicators", "description", "TEXT"),
    # 多专家矩阵
    ("expert_matrices", "unit_type", "VARCHAR(32)"),
    ("expert_matrices", "l1_name", "VARCHAR(64)"),
    ("expert_matrices", "cr", "FLOAT"),
]
def ensure_schema(db):
    """为已有表补齐缺失列。返回补列清单。"""
    added = []
    try:
        with db.engine.connect() as conn:
            for table, column, ddl in REQUIRED_COLUMNS:
                rows = conn.execute(text("PRAGMA
    table_info(%s)" % table)).fetchall()
                if not rows:
                    continue                                #
    表还不存在，create_all 会建
                existing = {r[1] for r in rows}
                if column in existing:
                    continue
                conn.execute(text(
                    "ALTER TABLE %s ADD COLUMN %s %s" %
    (table, column, ddl)))
                conn.commit()
                added.append("%s.%s" % (table, column))
    except Exception as e:                                  #
    迁移失败不应阻断启动
        print("[schema] 结构检查跳过：%s" % e)
    return added
# ==== app/utils/uploads.py ====
# -*- coding: utf-8 -*-
"""上传文件处理（遥感影像、批量导入）。"""
import os
import re
import uuid
from datetime import datetime
from werkzeug.utils import secure_filename
def _safe_ext(filename):
```

```
    return os.path.splitext(filename)[1].lower().lstrip(".")
def ext_allowed(filename, allowed):
    return _safe_ext(filename) in allowed
def save_upload(file_storage, subdir, allowed_ext,
    upload_root):
    """保存上传文件，返回相对 static 的路径（如 uploads/patches/xxx.png）。
    文件名重命名为「时间戳_随机串.扩展名」，避免中文名与重名问题。
    """
    if file_storage is None or not file_storage.filename:
        return None, "未选择文件"
    ext = _safe_ext(file_storage.filename)
    if ext not in allowed_ext:
        return None, "不支持的文件类型：.%s" % ext
    target_dir = os.path.join(upload_root, subdir)
    os.makedirs(target_dir, exist_ok=True)
    # 仅保留安全字符，并加时间戳与随机串防重名
    base = secure_filename(os.path.splitext(file_storage.fil
    ename)[0]) or "file"
    base = re.sub(r"[^A-Za-z0-9_\-]", "", base)[:40] or
    "file"
    name = "%s_%s.%s" % (datetime.now().strftime("%Y%m%d%H%M
    %S"),
                         uuid.uuid4().hex[:6], ext)
    abs_path = os.path.join(target_dir, name)
    file_storage.save(abs_path)
    rel = os.path.join("uploads", subdir,
    name).replace("\\", "/")
    return rel, None
def remove_upload(rel_path, upload_root):
    """删除已保存的上传文件（相对 static 的路径）。"""
    if not rel_path:
        return
    abs_path = os.path.join(upload_root,
    rel_path.replace("uploads/", "", 1))
    try:
        if os.path.isfile(abs_path):
            os.remove(abs_path)
    except OSError:
        pass
# ==== app/utils/charts.py ====
# -*- coding: utf-8 -*-
"""轻量图表。
用纯 SVG 输出，不依赖任何绘图库或前端脚本：
- 服务端渲染，页面与打印（PDF）中都能稳定显示
- 每个图元都带数值标签，黑白打印时仍可读（不单靠颜色区分）
- 单色方案，符合界面「一个视图一个强调色」的约定
"""
from html import escape
ACCENT = "#1f3a5f"
ACCENT_SOFT = "#7f93ad"
```

```
INK = "#23262b"
MUTED = "#6b7280"
LINE = "#dfe3e8"
#  灰度梯度：黑白打印时相邻扇区 / 条柱仍可区分
GRAYS = ["#1f3a5f", "#456188", "#6d86a8", "#96a9c4",
    "#bcc9da", "#dbe3ec"]
def _fmt(v, digits=4):
    try:
        return ("%%.%df" % digits) % float(v)
    except (TypeError, ValueError):
        return str(v)
def hbars(items, width=620, bar_h=22, gap=12, label_w=150,
    value_w=76):
    """水平条形图。items: [(标签, 数值, 显示文本)]"""
    if not items:
        return '<p class="muted">暂无数据</p>'
    n = len(items)
    height = n * (bar_h + gap) + 10
    max_v = max((abs(float(v)) for _, v, _ in items),
    default=1) or 1
    body_w = width - label_w - value_w
    parts = ['<svg viewBox="0 0 %d %d" width="100%%"
    height="%d" '
             'role="img" aria-label="条形图" '
             'style="max-width:%dpx;font-family:inherit;">'
    % (width, height, height, width)]
    for i, (label, value, disp) in enumerate(items):
        y = i * (bar_h + gap) + 5
        frac = (abs(float(value)) / max_v) if max_v else 0
        w = max(2.0, body_w * frac)
        fill = GRAYS[i % len(GRAYS)]
        parts.append(
            '<text x="%d" y="%d" text-anchor="end"
    font-size="12" fill="%s">%s</text>'
            % (label_w - 10, y + bar_h * 0.72, INK,
    escape(str(label))))
        parts.append(
            '<rect x="%d" y="%d" width="%.1f" height="%d"
    rx="3" fill="%s"/>'
            % (label_w, y, w, bar_h, fill))
        parts.append(
            '<text x="%d" y="%d" font-size="12" fill="%s" '
            'style="font-variant-numeric:tabular-nums;">%s</
    text>'
            % (label_w + w + 8, y + bar_h * 0.72, MUTED,
    escape(str(disp))))
    parts.append('</svg>')
    return "".join(parts)
def donut(items, size=210, thickness=34):
    """环形图（权重分布）。items: [(标签, 数值, 显示文本)]
```

```
图例中给出数值与占比，黑白打印时靠图例与灰度深浅区分。
"""
vals = [(l, float(v), d) for l, v, d in items if
float(v) > 0]
if not vals:
    return '<p class="muted">暂无数据</p>'
total = sum(v for _, v, _ in vals) or 1
r = (size - thickness) / 2
cx = cy = size / 2
import math
parts = ['<svg viewBox="0 0 %d %d" width="%d"
height="%d" role="img" '
         'aria-label="权重分布环形图">' % (size, size, size,
size)]
angle = -math.pi / 2
for i, (label, v, disp) in enumerate(vals):
    sweep = 2 * math.pi * (v / total)
    a0, a1 = angle, angle + sweep
    x0, y0 = cx + r * math.cos(a0), cy + r *
math.sin(a0)
    x1, y1 = cx + r * math.cos(a1), cy + r *
math.sin(a1)
    large = 1 if sweep > math.pi else 0
    parts.append(
        '<path d="M %.2f %.2f A %.2f %.2f 0 %d 1 %.2f
%.2f" '
        'fill="none" stroke="%s" stroke-width="%d"/>'
        % (x0, y0, r, r, large, x1, y1, GRAYS[i %
len(GRAYS)], thickness))
    angle = a1
parts.append('<text x="%d" y="%d" text-anchor="middle"
font-size="12" '
             'fill="%s">合计</text>' % (cx, cy - 2,
MUTED))
parts.append('<text x="%d" y="%d" text-anchor="middle"
font-size="15" '
             'fill="%s" style="font-variant-numeric:tabu
lar-nums;">%s</text>'
             % (cx, cy + 18, INK, _fmt(total, 4)))
parts.append('</svg>')
legend = ['<table style="width:auto;margin-top:10px;"><t
head><tr>'
          '<th scope="col">项目</th><th
scope="col">数值</th>'
          '<th scope="col">占比</th></tr></thead><tbody>']
for i, (label, v, disp) in enumerate(vals):
    legend.append(
        '<tr><td style="text-align:left;">'
        '<span style="display:inline-block;width:10px;he
ight:10px;'
```

```
                'border-radius:2px;background:%s;margin-right:6p
    x;"></span>%s</td>'
                '<td class="mono">%s</td><td
    class="mono">%.1f%%</td></tr>'
              % (GRAYS[i % len(GRAYS)], escape(str(label)),
    escape(str(disp)),
                 100.0 * v / total))
    legend.append('</tbody></table>')
    return ('<div style="display:flex;flex-wrap:wrap;gap:20p
    x;align-items:center;">'
              + "".join(parts) + "".join(legend) + '</div>')
# ==== app/utils/browser.py ====
# -*- coding: utf-8 -*-
"""HTML 转 PDF（报告导出）。
复用本机已安装的 Chrome/Edge 无头模式打印，避免引入 reportlab、
weasyprint 等额外依赖。生成过程使用系统临时目录中的 ASCII 文件名，
避免中文路径在 file:// URL 下出问题。
"""
import os
import shutil
import subprocess
import tempfile
import uuid
CANDIDATES = [
    r"C:\Program Files\Google\Chrome\Application\chrome.exe"
    ,
    r"C:\Program Files (x86)\Google\Chrome\Application\chrom
    e.exe",
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msed
    ge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe
    ",
]
# 允许通过环境变量覆盖，便于在其他机器上指定浏览器
ENV_KEY = "DSH_BROWSER_PATH"
def browser_path():
    """定位可用的浏览器可执行文件，找不到返回 None。"""
    override = os.environ.get(ENV_KEY)
    if override and os.path.isfile(override):
        return override
    for p in CANDIDATES:
        if os.path.isfile(p):
            return p
    for name in ("chrome", "msedge", "chromium"):
        found = shutil.which(name)
        if found:
            return found
    return None
def html_to_pdf(html, timeout=90):
    """把 HTML 字符串渲染为 PDF，返回字节；失败返回 (None, 原因)。"""
```

```
    exe = browser_path()
    if exe is None:
        return None, ("未找到可用的浏览器（Chrome/Edge），无法生成 PDF。"
                      "可设置环境变量 %s 指向浏览器可执行文件。" % ENV_KEY)
    tmpdir = tempfile.mkdtemp(prefix="dshpdf_")
    tag = uuid.uuid4().hex[:8]
    html_path = os.path.join(tmpdir, "r_%s.html" % tag)
    pdf_path = os.path.join(tmpdir, "r_%s.pdf" % tag)
    try:
        with open(html_path, "w", encoding="utf-8") as f:
            f.write(html)
        cmd = [
            exe, "--headless=new", "--disable-gpu",
    "--no-sandbox",
            "--no-first-run", "--no-default-browser-check",
            "--no-pdf-header-footer",
            "--virtual-time-budget=4000",
            "--print-to-pdf=%s" % pdf_path,
            "file:///" + html_path.replace("\\", "/"),
        ]
        p = subprocess.run(cmd, capture_output=True,
    text=True,
                           timeout=timeout,
    encoding="utf-8", errors="replace")
        if not os.path.isfile(pdf_path) or
    os.path.getsize(pdf_path) < 800:
            return None, "浏览器未能生成 PDF（%s）" % (p.stderr or
    "").strip()[:200]
        with open(pdf_path, "rb") as f:
            return f.read(), None
    except subprocess.TimeoutExpired:
        return None, "生成 PDF 超时，请稍后重试"
    except Exception as e:
        return None, "生成 PDF 失败：%s" % e
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)
# ==== app/services/ahp_service.py ====
# -*- coding: utf-8 -*-
"""AHP 权重引擎。
严格按论文 4.2.1 节实现：
- 式(2)(3)(4)：特征根法求权重（行元素乘积 → n 次方根 → 归一化）
- 式(5)(6)(7)：一致性检验（λmax、CI、CR）
- 论文要求 CR < 0.1；RI 按阶数查表
"""
import numpy as np
from app.utils.constants import RI_TABLE, CR_THRESHOLD
def matrix_from_upper(upper):
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
```

```
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
```

```
    返回                                                                1051
    ----                                                                1052
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)          1053
    及对应取值。                                                        1054
    """                                                                 1055
    A = np.asarray(A, dtype=float)                                      1056
    r = weights_and_consistency(A)                                      1057
    w = r["weights"]                                                    1058
    n = A.shape[0]                                                      1059
    best = None                                                         1060
    for i in range(n):                                                  1061
        for j in range(i + 1, n):                                       1062
            expected = w[i] / w[j]                                      1063
            if expected == 0:                                           1064
                continue                                                1065
            deviation = abs(A[i, j] - expected) / expected              1066
            if best is None or deviation >                              1067
    best["deviation"]:                                                  1068
                best = {"i": i, "j": j, "value": A[i, j],               1069
                        "expected": expected, "deviation":              1070
    deviation}                                                          1071
    return best                                                         1072
def aggregate_matrices(matrices):                                       1073
    """多专家判断矩阵聚合：几何平均（保持互反性）。                     1074
    参数                                                                1075
    ----                                                                1076
    matrices : list[np.ndarray]，多个同阶判断矩阵。                     1077
    """                                                                 1078
    stack = np.stack([np.asarray(m, dtype=float) for m in               1079
    matrices])                                                          1080
    return np.prod(stack, axis=0) ** (1.0 / len(stack))                 1081
def judge_from_weights(weights):                                        1082
    """由权重向量构造完全一致判断矩阵并检验。                           1083
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量     1084
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。                          1085
    """                                                                 1086
    w = np.asarray(weights, dtype=float)                                1087
    n = w.shape[0]                                                      1088
    A = np.outer(w, 1.0 / w)                                            1089
    r = weights_and_consistency(A)                                      1090
    r["matrix"] = A                                                     1091
    return r                                                            1092
# ==== app/services/standardization.py ====                           1093
# -*- coding: utf-8 -*-                                               1094
"""指标标准化（的一部分）。                                           1095
按论文 4.2.2 节式(8) 实现：                                           1096
    x' = (x - x_min) / (x_max - x_min)                                  1097
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。           1098
所有理想极值取自 app.utils.constants，可配置。                        1099
"""                                                                     1100
```

```
from app.utils.constants import (
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,
    PH_TOL_LOW, PH_TOL_HIGH,
)
def std_benefit(value, ideal):
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, value / ideal))
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染（=1）处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    """
    total = 0.0
    for metal, extreme in HEAVY_METAL_LIMIT.items():
        value = heavy_metals.get(metal)
```

```
        total += std_cost(value, extreme) if value is not None else 1.0
    return total / len(HEAVY_METAL_LIMIT)
def standardize_unit(unit_type, data, weights=None):
    """对单个微地质单元的全部指标做标准化。
    参数
    ----
    unit_type : str，岩质边坡 / 碎石边坡 / 平台
    data : dict，实测数据，字段见接口契约：
        soil_ratio, thickness, ph, organic_matter,
    heavy_metals(dict),
        coverage, shannon, slope, aspect
    weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIGHTS 结构），
            未提供时用默认值。
    返回
    ----
    dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }
    标准化值可能为 None，表示该指标不适用或方法待定。
    """
    from app.utils.constants import DEFAULT_WEIGHTS
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    l1_enabled = weights["l1"].keys()
    l2w = weights["l2"]
    result = {}
    # ---- 土壤指标 ----
    if "土壤指标" in l1_enabled:
        result["土壤指标"] = {
            "土石比": (data.get("soil_ratio"),
                       std_benefit(data.get("soil_ratio"),
    IDEAL_SOIL_RATIO),
                       l2w["土壤指标"]["土石比"]),
            "土层厚度": (data.get("thickness"),
                         std_benefit(data.get("thickness"),
    IDEAL_THICKNESS),
                         l2w["土壤指标"]["土层厚度"]),
            "pH": (data.get("ph"),
                   std_mid_ph(data.get("ph"),
    *IDEAL_PH_RANGE),
                   l2w["土壤指标"]["pH"]),
            "有机质": (data.get("organic_matter"),
                       std_benefit(data.get("organic_matter")
    , IDEAL_ORGANIC),
                       l2w["土壤指标"]["有机质"]),
            "污染指标": (data.get("heavy_metals") or {},
                         pollution_score(data.get("heavy_meta
    ls") or {}),
                         l2w["土壤指标"]["污染指标"]),
        }
```

```
    # ---- 植被指标 ----
    if "植被指标" in l1_enabled:
        result["植被指标"] = {
            "植被覆盖度": (data.get("coverage"),
                           std_benefit(data.get("coverage"),
    IDEAL_COVERAGE),
                           l2w["植被指标"]["植被覆盖度"]),
            "生物多样性": (data.get("shannon"),
                           std_benefit(data.get("shannon"),
    IDEAL_SHANNON),
                           l2w["植被指标"]["生物多样性"]),
        }
    # ---- 地形指标 ----
    if "地形指标" in l1_enabled:
        result["地形指标"] = {
            "坡度": (data.get("slope"),
                     std_slope(data.get("slope")),
                     l2w["地形指标"]["坡度"]),
            "坡向": (data.get("aspect"),
                     aspect_score(data.get("aspect")),
                     l2w["地形指标"]["坡向"]),
        }
    return result
# ==== app/services/weight_service.py ====
# -*- coding: utf-8 -*-
"""权重方案的存取与生效权重解析。
系统有两套权重来源：
1. 论文表 3 给出的默认权重（app.utils.constants.DEFAULT_WEIGHTS），
   是**权威取值来源**，案例复算与报告结论以此为准；
2. 通过层次分析法在本系统中实际算出的权重，保存在
   weight_schemes / weight_items 两张表中，可标记为「生效」。
本模块负责把这两者合成为计算引擎直接可用的嵌套权重结构：
以论文表 3 为底，用生效方案中已保存的条目覆盖。
没有任何生效方案时，即完全使用论文表 3 的权重。
这样既保证默认行为与论文一致，又使 AHP 计算结果真正参与评价，
而不是算完即弃。
"""
import copy
from app.models import db, WeightScheme, WeightItem
from app.utils.constants import DEFAULT_WEIGHTS
from app.services.indicator_service import TREE
# 方案名称：每个微地质单元类型固定一个可生效的方案
ACTIVE_NAME = "自定义权重方案（AHP 计算结果）"
def _code_maps():
    """由指标树导出 编码 → 名称 的映射，避免名称在两处各写一遍。"""
    l1, l2 = {}, {}
    for row in TREE:
        code, name, level, parent = row[0], row[1], row[2],
    row[3]
        if level == 1:
```

```
                l1[code] = name                                                  1251
            elif level == 2:                                                     1252
                l2[code] = (l1.get(parent, parent), name)                        1253
    return l1, l2                                                                1254
def effective_weights(unit_type):                                                1255
    """返回该单元类型当前生效的权重。                                                1256
    返回 (weights, source_label, scheme)                                         1257
        weights       嵌套权重结构，可直接交给标准化与评价引擎                          1258
        source_label  权重来源说明，用于界面与报告标注                                1259
        scheme        生效的权重方案对象，无自定义方案时为 None                       1260
    """                                                                          1261
    base = DEFAULT_WEIGHTS.get(unit_type)                                        1262
    if base is None:                                                             1263
        return None, "未定义该微地质单元类型的权重", None                              1264
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ                     1265
    e,                                                                           1266
                                                                                 1267
    name=ACTIVE_NAME).first()                                                    1268
    if scheme is None:                                                           1269
        return copy.deepcopy(base), "论文表 3 默认权重", None                        1270
    merged = copy.deepcopy(base)                                                 1271
    l1_map, l2_map = _code_maps()                                                1272
    applied = 0                                                                  1273
    for item in WeightItem.query.filter_by(scheme_id=scheme.                     1274
    id).all():                                                                   1275
        code, w = item.indicator_code, item.weight                               1276
        if w is None or w <= 0:                                                  1277
            continue                                                             1278
        if code in l1_map and l1_map[code] in merged["l1"]:                      1279
            merged["l1"][l1_map[code]] = w                                       1280
            applied += 1                                                         1281
        elif code in l2_map:                                                     1282
            l1_name, l2_name = l2_map[code]                                      1283
            if l1_name in merged["l2"] and l2_name in                            1284
    merged["l2"][l1_name]:                                                       1285
                merged["l2"][l1_name][l2_name] = w                               1286
                applied += 1                                                     1287
    if applied == 0:                                                             1288
        return copy.deepcopy(base), "论文表 3 默认权重", None                        1289
    return merged, "自定义方案（%d 项覆盖论文表 3）" % applied, scheme                 1290
def save_weights(unit_type, code_weight_pairs, note=None):                       1291
    """把一组「指标编码 → 权重」写入生效方案（已存在则更新对应条目）。                    1292
    仅覆盖本次传入的条目，其余条目保留，便于分层次（一级 / 二级）                        1293
    分次录入而无需一次填完。                                                       1294
    """                                                                          1295
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ                     1296
    e,                                                                           1297
                                                                                 1298
    name=ACTIVE_NAME).first()                                                    1299
    if scheme is None:                                                           1300
```

```
            scheme = WeightScheme(unit_type=unit_type,
        name=ACTIVE_NAME,
                                  level=1, is_default=True)
            db.session.add(scheme)
            db.session.flush()
        if note:
            scheme.level = scheme.level or 1
        if not scheme.is_default:
            scheme.is_default = True
        written = 0
        for code, w in code_weight_pairs:
            if w is None or w <= 0:
                continue
            item = WeightItem.query.filter_by(scheme_id=scheme.i
        d,

        indicator_code=code).first()
            if item is None:
                item = WeightItem(scheme_id=scheme.id,
        indicator_code=code)
                db.session.add(item)
            item.weight = float(w)
            written += 1
        db.session.commit()
        return scheme, written
def reset_weights(unit_type):
        """删除该单元类型的自定义方案，恢复为论文表 3 的默认权重。"""
        schemes = WeightScheme.query.filter_by(unit_type=unit_ty
        pe,

        name=ACTIVE_NAME).all()
        n = 0
        for s in schemes:
            WeightItem.query.filter_by(scheme_id=s.id).delete(
                synchronize_session=False)
            db.session.delete(s)
            n += 1
        db.session.commit()
        return n
def scheme_items(unit_type):
        """返回该单元类型已保存的自定义权重条目 {指标编码: 权重}。"""
        scheme = WeightScheme.query.filter_by(unit_type=unit_typ
        e,

        name=ACTIVE_NAME).first()
        if scheme is None:
            return {}, None
        items = {i.indicator_code: i.weight
                 for i in WeightItem.query.filter_by(scheme_id=s
        cheme.id).all()}
```

```
    return items, scheme
def summary():
    """各单元类型的权重来源概览，供权重首页展示。"""
    rows = []
    for unit_type in DEFAULT_WEIGHTS:
        weights, label, scheme = effective_weights(unit_type
    )
        rows.append({
            "unit_type": unit_type,
            "label": label,
            "custom": scheme is not None,
        })
    return rows
# ==== app/services/evaluation_service.py ====
# -*- coding: utf-8 -*-
"""适宜性评价引擎。
按论文式（9） 实现：
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）
- 矿山级：F = Σ（Si × F'i），Si 为各微地质单元面积占比
- 等级判定按论文表 4。
"""
from app.utils.constants import DEFAULT_WEIGHTS, grade_of
from app.services import standardization as std
def score_unit(unit_type, data, weights=None):
    """计算单个微地质单元的评价因子 F'。
    返回
    ----
    dict：{ l1_scores: {一级指标: 得分}, unit_score: F', details:
    [...] }
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，
    则按其余项权重归一化后计算，保证单元得分始终有定义。
    """
    if weights is None:
        weights = DEFAULT_WEIGHTS[unit_type]
    std_result = std.standardize_unit(unit_type, data,
    weights)
    l1_weights = weights["l1"]
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
    raw,
                    "std_value": None, "weight": w2,
    "contribution": None,
```

```
                    })
                    continue
                contribution = std_value * w2
                acc += contribution
                wsum += w2
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw": raw,
                    "std_value": std_value, "weight": w2,
                    "contribution": contribution,
                })
            l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
        # 一级指标权重按实际参与计算的指标归一化。
        # 论文表 3 的权重本身已归一（和为 1），此处为恒等变换；
        # 若某层指标缺失或自定义权重不完整，可避免单元得分被整体压低。
        lw_sum = sum(l1_weights[k] for k in l1_scores if k in
        l1_weights)
        if lw_sum > 0:
            unit_score = sum(l1_weights[k] * l1_scores[k]
                             for k in l1_scores if k in
        l1_weights) / lw_sum
        else:
            unit_score = 0.0
        return {"l1_scores": l1_scores, "unit_score":
        unit_score, "details": details}
def score_mine(units, weights_map=None):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    weights_map : dict，{单元类型: 权重结构}。未提供的单元类型
        使用论文表 3 的默认权重。权重可来自 AHP 计算并经
        weight_service 保存生效，从而真正参与评价。
    返回
    ----
    dict:{ final_score, grade, grade_info, unit_results }
    """
    weights_map = weights_map or {}
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"],
                       weights_map.get(u["unit_type"]))
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
```

```
                "weighted": weighted,
                "l1_scores": r["l1_scores"],
                "details": r["details"],
            })
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
from datetime import datetime
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
```

```
    for i, htext in enumerate(headers):
        _set_run(hdr[i].paragraphs[0].add_run(), htext,
    10.5, bold=True)
    for row in rows:
        cells = t.add_row().cells
        for i, val in enumerate(row):
            _set_run(cells[i].paragraphs[0].add_run(),
    str(val), 10.5)
    return t
def build_report(patch, mine_result):
    """生成评价报告 Word 文档，返回 BytesIO。
    参数
    ----
    patch : MinePatch，图斑对象
    mine_result : evaluation_service.score_mine 的返回 dict
    """
    doc = Document()
    # 页边距
    for section in doc.sections:
        section.top_margin = Cm(2.5)
        section.bottom_margin = Cm(2.5)
        section.left_margin = Cm(2.5)
        section.right_margin = Cm(2.5)
    _heading(doc, "历史遗留废弃矿山自然恢复适宜性评价报告", 0)
    _para(doc, "软件：历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ1．0",
          size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER)
    _para(doc, "生成时间：%s" % datetime.now().strftime("%Y-%m-%d
    %H：%M：%S"),
          size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER,
    space_after=12)
    # 图斑信息
    _heading(doc, "一、图斑基本信息", 1)
    info = [
        ("图斑名称", patch.name),
        ("所在行政区", patch.admin_region or "—"),
        ("图斑面积（公顷）", str(patch.area_ha) if patch.area_ha is
    not None else "—"),
        ("开采矿种", patch.mineral_type or "—"),
        ("开采方式", patch.mining_method or "—"),
    ]
    _table(doc, ["项目", "内容"], info)
    # 评价结论
    _heading(doc, "二、评价结论", 1)
    gi = mine_result["grade_info"]
    concl = [
        ("最终评价因子 F", "%.4f" % mine_result["final_score"]),
        ("适宜性等级", mine_result["grade"]),
        ("判定依据", gi["meaning"]),
        ("推荐措施", gi["measure"]),
    ]
```

```
    _table(doc, ["项目", "结果"], concl)
    # 各单元得分
    _heading(doc, "三、各微地质单元得分", 1)
    rows = []
    for u in mine_result["unit_results"]:
        rows.append((u["unit_type"], "%.2f%%" % (u["ratio"] * 100),
                     "%.4f" % u["unit_score"], "%.4f" % u["weighted"]))
    _table(doc, ["微地质单元", "面积占比", "单元评价因子 F'", "加权贡献"], rows)
    # 完整得分明细
    _heading(doc, "四、得分明细", 1)
    for u in mine_result["unit_results"]:
        _para(doc, "%s（占比 %.2f%%）" % (u["unit_type"], u["ratio"] * 100),
              size=11, bold=True, space_after=4)
        detail_rows = []
        for d in u["details"]:
            raw = "—" if d["raw"] is None else d["raw"]
            std = "—" if d["std_value"] is None else "%.4f" % d["std_value"]
            w = "%.4f" % d["weight"]
            c = "—" if d["contribution"] is None else "%.4f" % d["contribution"]
            detail_rows.append((d["l1"], d["l2"], raw, std, w, c))
        _table(doc, ["一级指标", "二级指标", "实测值", "标准化值", "权重", "贡献"],
               detail_rows)
        # 一级指标得分
        l1_row = []
        for k, v in u["l1_scores"].items():
            l1_row.append("%s=%.4f" % (k, v))
        _para(doc, "一级指标得分：" + "，".join(l1_row), size=10.5, space_after=10)
    buf = io.BytesIO()
    doc.save(buf)
    buf.seek(0)
    return buf
# ==== app/services/indicator_service.py ====
# -*- coding: utf-8 -*-
"""指标定义维护。
把论文表 2/表 3 的指标体系写入 indicators 表，使指标名称、单位、
理想值与取值依据可在界面上查看维护，而不再只存在于代码常量中。
代码常量（app.utils.constants）仍是计算时的**权威取值来源**；
本表用于展示、核对与记录取值依据。二者由 seed_indicators() 保持同步。
"""
from app.models import db, Indicator
from app.utils.constants import (
```

```
    UNIT_TYPES, UNIT_ENABLED_INDICATORS, IDEAL_SOIL_RATIO,
    IDEAL_THICKNESS,
    IDEAL_PH_RANGE, IDEAL_ORGANIC, IDEAL_COVERAGE,
    IDEAL_SHANNON,
    HEAVY_METAL_LIMIT, SLOPE_BREAK, ASPECT_SCORES,
)
ALL_UNITS = ",".join(UNIT_TYPES)
# (code, name, level, parent, unit, ideal_value,
    ideal_type, enabled_units, desc, is_constraint)
TREE = [
    # ---- 一级指标 ----
    ("soil", "土壤指标", 1, None, None, None, None, "碎石边坡,平台",
    "论文表 2 评价指标之一级指标", False),
    ("veg", "植被指标", 1, None, None, None, None, ALL_UNITS,
    "论文表 2 评价指标之一级指标", False),
    ("terrain", "地形指标", 1, None, None, None, None,
    "岩质边坡,碎石边坡", "论文表 2 评价指标之一级指标", False),
    # ---- 二级指标：土壤 ----
    ("soil_ratio", "土石比", 2, "soil", "—",
    str(IDEAL_SOIL_RATIO), "benefit", "碎石边坡,平台",
    "论文 4.2.2 节：理想值 0.8，根据矿山修复实际情况确定", False),
    ("thickness", "土层厚度", 2, "soil", "cm",
    str(IDEAL_THICKNESS), "benefit", "碎石边坡,平台",
    "论文 4.2.2 节：理想值 20 cm", False),
    ("ph", "pH", 2, "soil", "—", "%g~%g" % IDEAL_PH_RANGE,
    "mid", "碎石边坡,平台",
    "论文 4.2.2 节：pH 用适中指标的标准化方法，理想区间 %g~%g" %
    IDEAL_PH_RANGE, False),
    ("organic_matter", "有机质", 2, "soil", "g/kg",
    str(IDEAL_ORGANIC), "benefit", "碎石边坡,平台",
    "《全国第二次土壤普查养分等级划分标准》二等（较丰富）最低值 30 g/kg", False),
    ("pollution", "污染指标", 2, "soil", "mg/kg", "见三级指标",
    "cost", "碎石边坡,平台",
    "同时作为限制性指标与评价指标；8 种重金属等权合成（各 0.125）", True),
    # ---- 二级指标：植被 ----
    ("coverage", "植被覆盖度", 2, "veg", "%",
    str(IDEAL_COVERAGE), "benefit", ALL_UNITS,
    "论文 4.2.2 节：参照生态系统取值 75%", False),
    ("shannon", "生物多样性", 2, "veg", "—", str(IDEAL_SHANNON),
    "benefit", ALL_UNITS,
    "Shannon-Wiener 多样性指数，参照生态系统取值 0.7", False),
    # ---- 二级指标：地形 ----
    ("slope", "坡度", 2, "terrain", "°", "%g° 分界" %
    SLOPE_BREAK, "cost", "岩质边坡,碎石边坡",
    "论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号：25°
    为陡坡重要分界", False),
    ("aspect", "坡向", 2, "terrain", "—",
    "阳坡0.2/阴阳坡0.6/阴坡1.0", "discrete", "岩质边坡,碎石边坡",
    "论文 4.2.2 节：阴坡及半阴坡水分条件更优，故赋值高于阳坡", False),
]
```

```
# 污染指标下的三级指标（8 种重金属）
for _m, _limit in HEAVY_METAL_LIMIT.items():
    TREE.append((
        "hm_" + _m, _m, 3, "pollution", "mg/kg",
    str(_limit), "cost",
        "碎石边坡,平台",
        "《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值 %g mg/kg" %
    _limit,
        False,
    ))
def seed_indicators():
    """写入/更新指标定义（幂等）。返回写入条数。"""
    code_map = {
        "benefit": "越大越优", "cost": "越小越优",
        "mid": "适中型", "discrete": "离散赋值",
    }
    n = 0
    for (code, name, level, parent, unit, ideal, itype,
    units, desc, is_con) in TREE:
        ind = Indicator.query.filter_by(code=code).first()
        if ind is None:
            ind = Indicator(code=code)
            db.session.add(ind)
        ind.name = name
        ind.level = level
        ind.parent_code = parent
        ind.unit = unit
        ind.ideal_value = ideal
        ind.ideal_type = itype
        ind.enabled_units = units
        ind.description = desc
        ind.is_constraint = is_con
        ind.is_evaluative = (level in (2, 3))
        n += 1
    db.session.commit()
    return n
def ideal_type_label(code):
    return {"benefit": "越大越优", "cost": "越小越优",
            "mid": "适中型", "discrete": "离散赋值"}.get(code,
    code or "—")
# ==== app/services/import_service.py ====
# -*- coding: utf-8 -*-
"""指标数据批量导入。
支持 CSV 与 Excel（.xlsx/.xls）。一行对应一个微地质单元，
按「单元类型」匹配图斑内已有的微地质单元后写入指标数据。
表头约定（缺失列按不适用处理）：
    单元类型， 土石比， 土层厚度， pH， 有机质，
    Hg， As， Ni， Cu， Cd， Pb， Cr， Zn，
    植被覆盖度， Shannon， 坡度， 坡向
"""
```

```
import csv
import io
import json
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
    UNIT_TYPES
# 表头（顺序即导出模板顺序）
HEADERS = (["单元类型", "土石比", "土层厚度", "pH", "有机质"]
           + list(HEAVY_METALS)
           + ["植被覆盖度", "Shannon", "坡度", "坡向"])
FIELD_ALIASES = {
    "单元类型": "unit_type",
    "土石比": "soil_ratio",
    "土层厚度": "thickness",
    "pH": "ph", "PH": "ph", "ph": "ph",
    "有机质": "organic_matter",
    "植被覆盖度": "coverage",
    "Shannon": "shannon", "shannon": "shannon", "生物多样性":
    "shannon",
    "坡度": "slope",
    "坡向": "aspect",
}
for _m in HEAVY_METALS:
    FIELD_ALIASES[_m] = "hm_" + _m
def build_template_csv():
    """生成导入模板（含表头与示例行）。"""
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerow(HEADERS)
    # 示例：碎石边坡一行，说明每列含义
    row = {h: "" for h in HEADERS}
    row.update({"单元类型": "碎石边坡", "土石比": "0.34", "土层厚度": "10",
                "pH": "9.24", "有机质": "4.89", "Hg": "0.05",
    "As": "5.17",
                "Ni": "12.60", "Cu": "11.70", "Cd": "0.04",
    "Pb": "6.50",
                "植被覆盖度": "87.92", "Shannon": "0.60",
                "坡度": "28.75", "坡向": "阳坡"})
    w.writerow([row[h] for h in HEADERS])
    return buf.getvalue()
def _read_rows(filename, stream):
    """读取上传文件为「表头 -> 值」的字典列表。"""
    ext = filename.rsplit(".", 1)[-1].lower()
    if ext == "csv":
        text = stream.read().decode("utf-8-sig",
    errors="replace")
        reader = csv.DictReader(io.StringIO(text))
        return [dict(r) for r in reader]
    if ext in ("xlsx", "xls"):
        from openpyxl import load_workbook
        wb = load_workbook(io.BytesIO(stream.read()),
```

```
    data_only=True)
        ws = wb.active
        rows = list(ws.iter_rows(values_only=True))
        if not rows:
            return []
        header = [str(c).strip() if c is not None else ""
for c in rows[0]]
        out = []
        for r in rows[1:]:
            if all(c is None or str(c).strip() == "" for c
in r):
                continue
            out.append({header[i]: (r[i] if i < len(r) else
None)
                        for i in range(len(header)) if
header[i]})
        return out
    raise ValueError("不支持的文件类型：.%s" % ext)
def _num(value):
    if value is None:
        return None, None
    s = str(value).strip()
    if s == "":
        return None, None
    try:
        return float(s), None
    except ValueError:
        return None, "「%s」不是有效数值" % s
def parse_upload(filename, stream, unit_type_map):
    """解析上传文件。
    参数
    ----
    unit_type_map : dict, {单元类型: MicroUnit 对象}
    返回
    ----
    (parsed, errors)
      parsed : list[dict], 每项 {unit_type, fields{...},
    heavy_metals{}}
      errors : list[str], 逐行问题说明
    """
    rows = _read_rows(filename, stream)
    parsed, errors = [], []
    if not rows:
        return [], ["文件为空或没有数据行"]
    # 表头校验
    unknown = [h for h in (rows[0].keys() if rows else [])
    if h not in FIELD_ALIASES]
    if unknown:
        errors.append("存在无法识别的列，将被忽略：%s" %
    "、".join(map(str, unknown)))
```

```
    for idx, raw in enumerate(rows, start=2):       # 第 1 行是表头
        rec = {}
        for k, v in raw.items():
            key = FIELD_ALIASES.get(str(k).strip())
            if key:
                rec[key] = v
        unit_type = str(rec.get("unit_type") or "").strip()
        if not unit_type:
            errors.append("第 %d 行：缺少「单元类型」" % idx)
            continue
        if unit_type not in unit_type_map:
            errors.append("第 %d 行：图斑内不存在单元类型「%s」（现有：%s）"
                          % (idx, unit_type,
"、".join(unit_type_map) or "无"))
            continue
        if unit_type not in UNIT_TYPES:
            errors.append("第 %d 行：单元类型「%s」不是系统支持的三种之一"
                          % (idx, unit_type))
            continue
        fields, heavy = {}, {}
        ok = True
        for key, val in rec.items():
            if key in ("unit_type",):
                continue
            if key.startswith("hm_"):
                n, err = _num(val)
                if err:
                    errors.append("第 %d 行：%s %s" % (idx,
key[3:], err))
                    ok = False
                elif n is not None:
                    heavy[key[3:]] = n
                continue
            if key == "aspect":
                s = str(val).strip() if val is not None
else ""
                if s == "":
                    fields["aspect"] = None
                elif s in ASPECT_TYPES:
                    fields["aspect"] = s
                else:
                    errors.append("第 %d 行：坡向「%s」无效（应为 %s）"
                                  % (idx, s,
"、".join(ASPECT_TYPES)))
                    ok = False
                continue
            n, err = _num(val)
            if err:
                errors.append("第 %d 行：%s %s" % (idx, key,
err))
```

```
                    ok = False
                else:
                    fields[key] = n
            if ok:
                parsed.append({"unit_type": unit_type,
        "fields": fields,
                               "heavy_metals": heavy})
        return parsed, errors
# ==== app/services/workflow.py ====
# -*- coding: utf-8 -*-
"""评价流程前置条件（定量评价与报告输出的流程门禁）。
论文 4.1 节确定的评价流程存在严格的先后关系：
    建立图斑 → 限制性条件判定（通过） → 划分微地质单元并计算面积占比
    → 逐单元录入指标数据 → 计算评价因子并判定等级 → 输出报告
论文原文：「先进行政策、位置约束地质灾害判断……对于满足限制性条件的
废弃矿山，按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡三种
地质微单元类型，并计算各种地形面积占比。」限制性条件判定（一票否决）
在前，微地质单元划分在后，顺序不可颠倒。
本模块把该顺序固化为可检查的状态。后续模块在入口处调用它，
前置步骤未完成时直接拦截并给出「缺什么、去哪补」的提示，
避免在条件不具备时产出看似合理的评价结论。
"""
from app.models import (db, MicroUnit, IndicatorData,
    ConstraintCheck,
                        EvaluationResult)
# 指标数据非空字段（用于判断某单元是否已录入数据）
DATA_FIELDS = ("soil_ratio", "thickness", "ph",
    "organic_matter",
               "coverage", "shannon", "slope", "aspect")
def _has_data(ind):
    if ind is None:
        return False
    if any(getattr(ind, f, None) is not None for f in
    DATA_FIELDS):
        return True
    return bool(ind.heavy_metals_dict())
def patch_progress(patch_id):
    """检查某图斑的评价流程完成情况。
    返回 dict：
        units       {done, count, ratio_sum, detail}
        constraint  {done, passed, detail}
        data        {done, ready, missing, detail}
        blocking    未满足的前置步骤列表（按流程顺序）
        ready       是否可进入定量评价
    """
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    ratio_sum = sum(u.ratio or 0.0 for u in units)
    # ---- 步骤一：微地质单元 ----
    units_step = {
```

```
        "done": bool(units),
        "count": len(units),
        "ratio_sum": ratio_sum,
        "detail": ("已划分 %d 个微地质单元，面积占比合计 %.2f%%"
                   % (len(units), ratio_sum * 100))
                  if units else "尚未划分微地质单元",
    }
    # ---- 步骤二：限制性条件判定 ----
    cc = ConstraintCheck.query.filter_by(patch_id=patch_id) \
        .order_by(ConstraintCheck.id.desc()).first()
    if cc is None:
        constraint_step = {"done": False, "passed": False,
                           "detail": "尚未执行限制性条件判定"}
    elif cc.passed:
        constraint_step = {"done": True, "passed": True,
                           "detail": "限制性条件判定已通过"}
    else:
        constraint_step = {"done": True, "passed": False,
                           "detail": "限制性条件判定未通过（%s）" %
    (cc.reason or "")}
    # ---- 步骤三：指标数据 ----
    missing = []
    for u in units:
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi
    rst()
        if not _has_data(ind):
            missing.append(u.unit_type)
    if not units:
        data_step = {"done": False, "ready": False,
    "missing": [],
                     "detail": "尚无可录入数据的微地质单元"}
    elif missing:
        data_step = {"done": False, "ready": False,
    "missing": missing,
                     "detail": "以下微地质单元尚未录入指标数据：%s"
                               % "、".join(missing)}
    else:
        data_step = {"done": True, "ready": True,
    "missing": [],
                     "detail": "全部 %d 个微地质单元的指标数据已录入" %
    len(units)}
    ratio_ok = ratio_sum > 0.999 if units else False
    if units and not ratio_ok:
        units_step["detail"] += "（占比合计不等于 １００%，请检查面积录入）"
    # ---- 汇总未满足的前置条件（按论文 ４．１ 节的流程顺序：限制性判定优先） ----
    blocking = []
    if not constraint_step["done"]:
        blocking.append({
            "step": "constraint", "label": "执行限制性条件判定",
```

```
                "reason": "尚未执行限制性条件判定。论文流程要求先判定政策、区位约束、"
                          "土壤污染、地质灾害与耕地属性，通过后方可进入定量评价。",
                "fix_url": "/constraint/%d" % patch_id,
            "fix_label": "去执行限制性判定",
            })
        elif not constraint_step["passed"]:
            blocking.append({
                "step": "constraint", "label": "限制性条件判定未通过",
                "reason": "限制性条件判定未通过。按论文流程，此类矿山应直接判定为"
                          "「人工辅助或生态重建」，不进入定量评价。",
                "fix_url": "/constraint/%d" % patch_id,
            "fix_label": "去复核判定结果",
            })
        if not units_step["done"]:
            blocking.append({
                "step": "units", "label": "划分微地质单元",
                "reason": "该图斑尚未划分微地质单元，无法确定各单元的面积权重。",
                "fix_url": "/unit/%d" % patch_id, "fix_label":
            "去划分微地质单元",
            })
        elif not ratio_ok:
            blocking.append({
                "step": "units", "label": "补齐单元面积",
                "reason": "各微地质单元面积占比合计不等于 １００%，加权结果会失真。",
                "fix_url": "/unit/%d" % patch_id, "fix_label":
            "去核对单元面积",
            })
        if units_step["done"] and not data_step["ready"]:
            blocking.append({
                "step": "data", "label": "录入指标数据",
                "reason": data_step["detail"] +
            "。缺少数据时计算会把空值按不适用跳过，"
                          "得出的结论不能反映实际情况。",
                "fix_url": "/indicator/", "fix_label":
            "去录入指标数据",
            })
        return {
            "units": units_step,
            "constraint": constraint_step,
            "data": data_step,
            "blocking": blocking,
            "ready": not blocking,
        }
    def latest_result(patch_id):
        """取该图斑最近一次的评价记录。"""
        return EvaluationResult.query.filter_by(patch_id=patch_i
        d) \
            .order_by(EvaluationResult.id.desc()).first()
    def report_progress(patch_id):
        """报告输出的前置条件：评价流程齐备，且已有评价记录。
```

```
    报告本质上是评价结论的呈现，因此必须先真实执行过定量评价，
    导出时读取已保存的评价结果，而不是在导出环节重新计算。
    否则评价完成后再修改指标数据，报告会与系统记录的评价结论不一致，
    同一份评价出现两个数字，结论无法追溯。
    返回 dict：{ready, blocking, result, flow}
        result 为最近一次评价记录（EvaluationResult），无记录时为空。
    """
    flow = patch_progress(patch_id)
    blocking = list(flow["blocking"])
    record = latest_result(patch_id)
    if not blocking and record is None:
        blocking.append({
            "step": "evaluate", "label": "执行适宜性评价",
            "reason": "该图斑尚无评价记录。报告是评价结论的呈现，"
                      "需先执行定量评价后才能生成。",
            "fix_url": "/evaluate/%d" % patch_id,
            "fix_label": "去执行适宜性评价",
        })
    return {
        "ready": not blocking,
        "blocking": blocking,
        "result": record,
        "flow": flow,
    }
# ==== app/services/audit.py ====
# -*- coding: utf-8 -*-
"""操作审计。
在应用工厂中以 after_request 钩子统一埋点：凡是成功的写操作（POST/PUT/DELETE）
且已登录，即写入 operation_logs 表，无需在每个视图里手工调用。
动作名称由「端点 → 中文动作」映射表给出，未收录的端点回退为方法 + 路径。
"""
from flask import request
from flask_login import current_user
from app.models import db, OperationLog
# 端点 → 可读动作名
ACTION_NAMES = {
    "auth.login": "登录系统",
    "auth.logout": "退出登录",
    "auth.add_user": "新增用户",
    "auth.edit_user": "编辑用户",
    "auth.delete_user": "删除用户",
    "auth.reset_password": "重置用户密码",
    "project.add": "新建项目",
    "project.edit": "编辑项目",
    "project.delete": "删除项目",
    "project.patch_add": "新建矿山图斑",
    "project.patch_edit": "编辑矿山图斑",
    "project.patch_delete": "删除矿山图斑",
    "project.upload_image": "上传遥感影像",
    "unit.add": "添加微地质单元",
```

```
    "unit.edit": "编辑微地质单元",
    "unit.delete": "删除微地质单元",
    "standard.add": "添加标准/政策文件",
    "standard.edit": "编辑标准/政策文件",
    "standard.delete": "删除标准/政策文件",
    "standard.rules": "维护标准规则",
    "standard.rule_delete": "删除标准规则",
    "constraint.check": "执行限制性条件判定",
    "ahp.save_expert": "录入专家判断矩阵",
    "ahp.delete_expert": "删除专家判断矩阵",
    "ahp.aggregate": "汇总多专家矩阵",
    "indicator.edit": "保存指标数据",
    "indicator.import_data": "批量导入指标数据",
    "indicator.update_meta": "维护指标定义",
    "evaluate.run": "执行适宜性评价",
}
# 无需记录的操作（查看类或纯导航）
SKIP_ENDPOINTS = {"static", "auth.index"}
def action_name(endpoint):
    if endpoint in ACTION_NAMES:
        return ACTION_NAMES[endpoint]
    return "写操作 %s %s" % (request.method, request.path)
def extract_target_id():
    """从视图参数中取出被操作对象的主键，取不到则返回 None。"""
    args = request.view_args or {}
    for key in ("pid", "patch_id", "unit_id", "sid", "rid",
    "result_id", "user_id"):
        if key in args:
            try:
                return int(args[key])
            except (TypeError, ValueError):
                return None
    return None
def record(endpoint, status_code):
    """记录一次写操作。失败（4xx/5xx）也记录，便于追溯。"""
    if endpoint in SKIP_ENDPOINTS:
        return
    try:
        detail = "路径 %s" % request.path
        if status_code >= 400:
            detail += "；结果 HTTP %d" % status_code
        db.session.add(OperationLog(
            user_id=current_user.id if
    current_user.is_authenticated else None,
            action=action_name(endpoint),
            target_type=(endpoint or "").split(".")[0] or
    None,
            target_id=extract_target_id(),
            detail=detail,
        ))
```

```
            db.session.commit()
        except Exception:
            # 审计失败不应影响业务响应
            db.session.rollback()
# ==== app/views/auth.py ====
# -*- coding: utf-8 -*-
"""用户与权限管理 —— 登录、注销、用户维护、操作日志。"""
from flask import (Blueprint, render_template, redirect,
    url_for, request,
                   flash, abort)
from flask_login import login_user, logout_user,
    login_required, current_user
from werkzeug.security import check_password_hash,
    generate_password_hash
from app.forms import LoginForm, EmptyForm
from app.models import db, User, OperationLog
bp = Blueprint("auth", __name__)
ROLE_LABELS = {"admin": "管理员", "evaluator": "评价员",
    "viewer": "查看者"}
def require_admin():
    """非管理员直接 403，避免误操作基础数据。"""
    if current_user.role != "admin":
        abort(403)
@bp.route("/")
def index():
    if current_user.is_authenticated:
        return render_template("auth/home.html")
    return redirect(url_for("auth.login"))
@bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("auth.index"))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.d
    ata).first()
        if user and check_password_hash(user.password_hash,
    form.password.data):
            if not user.active:
                flash("该账号已停用，请联系管理员", "error")
                return render_template("auth/login.html",
    form=form)
            login_user(user, remember=form.remember.data)
            # 用 toast 类别：渲染为页面正中的浮层，淡入停留后淡出。
            # 登录成功只是告知性反馈，不需要用户停下来阅读。
            flash("登录成功", "toast")
            return redirect(url_for("auth.index"))
        flash("用户名或密码错误", "error")
    return render_template("auth/login.html", form=form)
@bp.route("/logout")
```

```
@login_required                                                    2151
def logout():                                                      2152
    logout_user()                                                  2153
    flash("已退出登录", "ok")                                      2154
    return redirect(url_for("auth.login"))                         2155
# ---------------------------- 用户维护 ----------------------------2156
    ----                                                           2157
@bp.route("/users")                                                2158
@login_required                                                    2159
def users():                                                       2160
    all_users = User.query.order_by(User.id).all()                 2161
    return render_template("auth/users.html",                      2162
    users=all_users,                                               2163
                                role_labels=ROLE_LABELS,           2164
    form=EmptyForm())                                              2165
@bp.route("/users/add", methods=["GET", "POST"])                   2166
@login_required                                                    2167
def add_user():                                                    2168
    require_admin()                                                2169
    if request.method == "POST":                                   2170
        username = request.form.get("username", "").strip()        2171
        password = request.form.get("password", "")                2172
        role = request.form.get("role", "viewer")                  2173
        if not username or not password:                           2174
            flash("用户名和密码不能为空", "error")                 2175
            return redirect(url_for("auth.add_user"))              2176
        if User.query.filter_by(username=username).first():        2177
            flash("用户名已存在", "error")                         2178
            return redirect(url_for("auth.add_user"))              2179
        db.session.add(User(username=username, role=role,          2180
                            password_hash=generate_password_       2181
    hash(password)))                                               2182
        db.session.commit()                                        2183
        flash("用户已添加", "ok")                                  2184
        return redirect(url_for("auth.users"))                     2185
    return render_template("auth/add_user.html")                   2186
@bp.route("/users/<int:user_id>/edit", methods=["GET",             2187
    "POST"])                                                       2188
@login_required                                                    2189
def edit_user(user_id):                                            2190
    require_admin()                                                2191
    user = db.get_or_404(User, user_id)                            2192
    if request.method == "POST":                                   2193
        role = request.form.get("role", user.role)                 2194
        active = request.form.get("active") == "1"                 2195
        # 保护：不能把最后一个管理员降级或停用，否则无人能管理系统 2196
        if user.role == "admin" and (role != "admin" or not        2197
    active):                                                       2198
            others = User.query.filter(User.role == "admin",       2199
                                       User.id != user.id,         2200
```

```
                                                                    2201
    User.active.is_(True)).count()                                  2202
            if others == 0:                                         2203
                flash("系统至少需保留一名启用状态的管理员", "error")      2204
                return redirect(url_for("auth.edit_user",           2205
    user_id=user.id))                                               2206
        user.role = role                                            2207
        user.active = active                                        2208
        new_password = request.form.get("password",                 2209
    "").strip()                                                     2210
        if new_password:                                            2211
            user.password_hash = generate_password_hash(new_        2212
    password)                                                       2213
        db.session.commit()                                         2214
        flash("用户信息已保存", "ok")                                 2215
        return redirect(url_for("auth.users"))                      2216
    return render_template("auth/edit_user.html", user=user,        2217
                           role_labels=ROLE_LABELS,                 2218
    form=EmptyForm())                                               2219
@bp.route("/users/<int:user_id>/delete", methods=["POST"])          2220
@login_required                                                     2221
def delete_user(user_id):                                           2222
    require_admin()                                                 2223
    user = db.get_or_404(User, user_id)                             2224
    if user.id == current_user.id:                                  2225
        flash("不能删除当前登录的账号", "error")                      2226
        return redirect(url_for("auth.users"))                      2227
    if user.role == "admin":                                        2228
        others = User.query.filter(User.role == "admin",            2229
                                   User.id != user.id,              2230
                                   User.active.is_(True)).co        2231
    unt()                                                           2232
        if others == 0:                                             2233
            flash("系统至少需保留一名启用状态的管理员", "error")        2234
            return redirect(url_for("auth.users"))                  2235
    db.session.delete(user)                                         2236
    db.session.commit()                                             2237
    flash("用户「%s」已删除" % user.username, "ok")                   2238
    return redirect(url_for("auth.users"))                          2239
# ---------------------------- 操作日志 ----------------------------2240
    ----                                                            2241
@bp.route("/logs")                                                  2242
@login_required                                                     2243
def logs():                                                         2244
    page = request.args.get("page", 1, type=int)                    2245
    per_page = 50                                                   2246
    q = OperationLog.query.order_by(OperationLog.id.desc())         2247
    total = q.count()                                               2248
    records = q.offset((page - 1) * per_page).limit(per_page        2249
    ).all()                                                         2250
```

```
    users_map = {u.id: u.username for u in User.query.all()}
    pages = max(1, (total + per_page - 1) // per_page)
    return render_template("auth/logs.html",
    records=records, page=page,
                           pages=pages, total=total,
    users_map=users_map)
# ==== app/views/project.py ====
# -*- coding: utf-8 -*-
"""项目与矿山档案 —— 项目、图斑、参照生态系统、遥感影像挂接。"""
from datetime import datetime
from flask import (Blueprint, render_template, request,
    redirect, url_for,
                   flash, current_app)
from flask_login import login_required, current_user
from app.forms import EmptyForm
from app.models import db, Project, MinePatch,
    ReferenceEcosystem
bp = Blueprint("project", __name__, url_prefix="/project")
@bp.route("/")
@login_required
def index():
    projects = Project.query.order_by(Project.id.desc()).all
    ()
    return render_template("project/index.html",
    projects=projects,
                           form=EmptyForm())
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        p = Project(
            name=request.form.get("name", "").strip(),
            code=request.form.get("code", "").strip() or
    None,
            description=request.form.get("description",
    "").strip() or None,
            created_by=current_user.id,
        )
        if not p.name:
            flash("项目名称不能为空", "error")
        else:
            db.session.add(p)
            db.session.commit()
            flash("项目已创建", "ok")
            return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form,
    project=None)
@bp.route("/<int:pid>/edit", methods=["GET", "POST"])
```

```
@login_required                                                    2301
def edit(pid):                                                     2302
    p = db.get_or_404(Project, pid)                                2303
    form = EmptyForm()                                             2304
    if request.method == "POST" and form.validate_on_submit(       2305
    ):                                                             2306
        p.name = request.form.get("name", "").strip()              2307
        p.code = request.form.get("code", "").strip() or           2308
    None                                                           2309
        p.description = request.form.get("description",            2310
    "").strip() or None                                            2311
        p.status = request.form.get("status", "进行中")             2312
        db.session.commit()                                        2313
        flash("项目已保存", "ok")                                    2314
        return redirect(url_for("project.index"))                  2315
    return render_template("project/edit.html", form=form,         2316
    project=p)                                                     2317
@bp.route("/<int:pid>/delete", methods=["POST"])                   2318
@login_required                                                    2319
def delete(pid):                                                   2320
    p = db.get_or_404(Project, pid)                                2321
    db.session.delete(p)                                           2322
    db.session.commit()                                            2323
    flash("项目已删除", "ok")                                        2324
    return redirect(url_for("project.index"))                      2325
# ---------------------------- 矿山图斑 ----------------------------2326
    ----                                                           2327
@bp.route("/<int:pid>/patches")                                    2328
@login_required                                                    2329
def patches(pid):                                                  2330
    p = db.get_or_404(Project, pid)                                2331
    return render_template("project/patches.html",                 2332
    project=p,                                                     2333
                                  form=EmptyForm())                2334
@bp.route("/patch/<int:patch_id>/delete", methods=["POST"])        2335
@login_required                                                    2336
def patch_delete(patch_id):                                        2337
    """删除矿山图斑，及其微地质单元与指标数据。"""                          2338
    from app.models import MicroUnit, IndicatorData,               2339
    EvaluationResult, \                                            2340
        ScoreDetail, ConstraintCheck, ReferenceEcosystem           2341
    patch = db.get_or_404(MinePatch, patch_id)                     2342
    pid = patch.project_id                                         2343
    unit_ids = [u.id for u in MicroUnit.query.filter_by(patc       2344
    h_id=patch.id).all()]                                          2345
    if unit_ids:                                                   2346
        IndicatorData.query.filter(IndicatorData.unit_id.in_       2347
    (unit_ids)).delete(                                            2348
            synchronize_session=False)                             2349
    for r in EvaluationResult.query.filter_by(patch_id=patch       2350
```

```
    .id).all():
        ScoreDetail.query.filter_by(result_id=r.id).delete(
            synchronize_session=False)
    EvaluationResult.query.filter_by(patch_id=patch.id).dele
    te(
        synchronize_session=False)
    ConstraintCheck.query.filter_by(patch_id=patch.id).delet
    e(
        synchronize_session=False)
    MicroUnit.query.filter_by(patch_id=patch.id).delete(sync
    hronize_session=False)
    ReferenceEcosystem.query.filter_by(patch_id=patch.id).de
    lete(
        synchronize_session=False)
    db.session.delete(patch)
    db.session.commit()
    flash("图斑「%s」及其数据已删除" % patch.name, "ok")
    return redirect(url_for("project.patches", pid=pid))
@bp.route("/<int:pid>/patch/add", methods=["GET", "POST"])
@login_required
def patch_add(pid):
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch = MinePatch(
            project_id=p.id,
            name=request.form.get("name", "").strip(),
            admin_region=fv("admin_region"),
            area_ha=fv("area_ha", float),
            mineral_type=fv("mineral_type"),
            mining_method=fv("mining_method"),
            stop_years=fv("stop_years", float),
            responsible_entity_missing=bool(request.form.get
    ("responsible_entity_missing")),
        )
        if not patch.name:
            flash("图斑名称不能为空", "error")
        else:
            db.session.add(patch)
            db.session.flush()
            #  参照生态系统默认取论文值
            db.session.add(ReferenceEcosystem(
                patch_id=patch.id, coverage_benchmark=75.0,
    shannon_benchmark=0.7))
            db.session.commit()
            flash("图斑已创建", "ok")
```

```
            return redirect(url_for("project.patches",
    pid=p.id))
    return render_template("project/patch_edit.html",
    form=form, project=p, patch=None)
@bp.route("/patch/<int:patch_id>/edit", methods=["GET",
    "POST"])
@login_required
def patch_edit(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch.name = request.form.get("name", "").strip()
        patch.admin_region = fv("admin_region")
        patch.area_ha = fv("area_ha", float)
        patch.mineral_type = fv("mineral_type")
        patch.mining_method = fv("mining_method")
        patch.stop_years = fv("stop_years", float)
        patch.responsible_entity_missing =
    bool(request.form.get("responsible_entity_missing"))
        # 参照生态系统
        ref = ReferenceEcosystem.query.filter_by(patch_id=pa
    tch.id).first()
        if ref is None:
            ref = ReferenceEcosystem(patch_id=patch.id)
            db.session.add(ref)
        ref.coverage_benchmark = fv("coverage_benchmark",
    float) or 75.0
        ref.shannon_benchmark = fv("shannon_benchmark",
    float) or 0.7
        ref.dominant_species = fv("dominant_species")
        patch.image_note = fv("image_note")
        db.session.commit()
        flash("图斑已保存", "ok")
        return redirect(url_for("project.patches",
    pid=patch.project_id))
    return render_template("project/patch_edit.html",
    form=form,
                           project=patch.project,
    patch=patch,
                           reference=ReferenceEcosystem.quer
    y.filter_by(
                               patch_id=patch.id).first())
# ---------------------------- 遥感影像挂接
    ----------------------------
@bp.route("/patch/<int:patch_id>/image", methods=["POST"])
@login_required
```

```
def upload_image(patch_id):                                                    2451
    """上传遥感底图 / 无人机正射影像并挂接到图斑。"""                                  2452
    from app.utils import uploads as up                                        2453
    patch = db.get_or_404(MinePatch, patch_id)                                 2454
    form = EmptyForm()                                                         2455
    if not form.validate_on_submit():                                          2456
        flash("表单校验失败，请重试", "error")                                      2457
        return redirect(url_for("project.patch_edit",                          2458
    patch_id=patch.id))                                                        2459
    rel, err = up.save_upload(request.files.get("image"),                      2460
    "patches",                                                                 2461
                              current_app.config["ALLOWED_IM                   2462
    AGE_EXT"],                                                                 2463
                              current_app.config["UPLOAD_FOL                   2464
    DER"])                                                                     2465
    if err:                                                                    2466
        flash(err, "error")                                                    2467
        return redirect(url_for("project.patch_edit",                          2468
    patch_id=patch.id))                                                        2469
    # 替换旧影像时清理原文件，避免残留                                                 2470
    if patch.image_path:                                                       2471
        up.remove_upload(patch.image_path,                                     2472
    current_app.config["UPLOAD_FOLDER"])                                       2473
    patch.image_path = rel                                                     2474
    patch.image_uploaded_at = datetime.utcnow()                                2475
    note = request.form.get("image_note", "").strip()                          2476
    if note:                                                                   2477
        patch.image_note = note                                                2478
    db.session.commit()                                                        2479
    flash("遥感影像已挂接", "ok")                                                  2480
    return redirect(url_for("project.patch_edit",                              2481
    patch_id=patch.id))                                                        2482
@bp.route("/patch/<int:patch_id>/image/delete",                                2483
    methods=["POST"])                                                          2484
@login_required                                                                2485
def delete_image(patch_id):                                                    2486
    from app.utils import uploads as up                                        2487
    patch = db.get_or_404(MinePatch, patch_id)                                 2488
    form = EmptyForm()                                                         2489
    if form.validate_on_submit():                                              2490
        up.remove_upload(patch.image_path,                                     2491
    current_app.config["UPLOAD_FOLDER"])                                       2492
        patch.image_path = None                                                2493
        patch.image_uploaded_at = None                                         2494
        db.session.commit()                                                    2495
        flash("遥感影像已移除", "ok")                                              2496
    return redirect(url_for("project.patch_edit",                              2497
    patch_id=patch.id))                                                        2498
# ==== app/views/unit.py ====                                                  2499
# -*- coding: utf-8 -*-                                                        2500
```

```
"""微地质单元管理 —— 岩质边坡 / 碎石边坡 / 平台的划分与面积占比。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
    IndicatorData
from app.utils.constants import UNIT_TYPES
bp = Blueprint("unit", __name__, url_prefix="/unit")
def recalc_ratio(patch_id):
    """按面积重算该图斑各单元面积占比。"""
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    total = sum(u.area or 0.0 for u in units)
    if total > 0:
        for u in units:
            u.ratio = round((u.area or 0.0) / total, 6)
    return total
@bp.route("/")
@login_required
def list_patches():
    """微地质单元入口：列出各图斑及其单元划分情况。"""
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    rows = []
    for p in patches:
        units = MicroUnit.query.filter_by(patch_id=p.id).all
    ()
        rows.append({
            "patch": p,
            "count": len(units),
            "ratio_sum": sum(u.ratio or 0.0 for u in units),
        })
    return render_template("unit/list.html", rows=rows)
@bp.route("/<int:patch_id>")
@login_required
def index(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    units = MicroUnit.query.filter_by(patch_id=patch_id).all
    ()
    total = sum(u.area or 0.0 for u in units)
    ratio_sum = sum(u.ratio or 0.0 for u in units)
    return render_template("unit/index.html", patch=patch,
    units=units,
                           total_area=total,
    ratio_sum=ratio_sum,
                           unit_types=UNIT_TYPES,
    form=EmptyForm())
@bp.route("/<int:patch_id>/add", methods=["POST"])
@login_required
```

```
def add(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if form.validate_on_submit():
        unit_type = request.form.get("unit_type")
        if unit_type not in UNIT_TYPES:
            flash("微地质单元类型不正确", "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        # 论文 4.1 节将图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
        # 允许重复会让面积占比与按单元类型组织的指标数据对不上号。
        if MicroUnit.query.filter_by(patch_id=patch.id,
                                     unit_type=unit_type).fi
    rst():
            flash("该图斑已存在「%s」，同一类型只能划分一个" % unit_type,
    "error")
            return redirect(url_for("unit.index",
    patch_id=patch_id))
        area = request.form.get("area", "").strip()
        u = MicroUnit(patch_id=patch.id,
    unit_type=unit_type,
                      area=float(area) if area else None,
                      description=request.form.get("descript
    ion", "").strip() or None)
        db.session.add(u)
        db.session.flush()
        recalc_ratio(patch.id)
        db.session.commit()
        flash("微地质单元已添加", "ok")
    return redirect(url_for("unit.index",
    patch_id=patch_id))
@bp.route("/edit/<int:unit_id>", methods=["GET", "POST"])
@login_required
def edit(unit_id):
    """单个微地质单元的编辑页。"""
    u = db.get_or_404(MicroUnit, unit_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        new_type = request.form.get("unit_type",
    u.unit_type)
        if new_type not in UNIT_TYPES:
            flash("微地质单元类型不正确", "error")
            return redirect(url_for("unit.edit",
    unit_id=u.id))
        # 改类型时同样不允许与本图斑已有的其他单元重复
        dup = MicroUnit.query.filter_by(patch_id=u.patch_id,


    unit_type=new_type).first()
        if dup is not None and dup.id != u.id:
```

```
                flash("该图斑已存在「%s」，同一类型只能划分一个" % new_type,
    "error")
                return redirect(url_for("unit.edit",
    unit_id=u.id))
            u.unit_type = new_type
            area = request.form.get("area", "").strip()
            u.area = float(area) if area else None
            u.description = request.form.get("description",
    "").strip() or None
            recalc_ratio(u.patch_id)
            db.session.commit()
            flash("微地质单元已保存", "ok")
            return redirect(url_for("unit.index",
    patch_id=u.patch_id))
    return render_template("unit/edit.html", unit=u,
    patch=u.patch,
                           unit_types=UNIT_TYPES, form=form)
@bp.route("/delete/<int:unit_id>", methods=["POST"])
@login_required
def delete(unit_id):
    u = db.get_or_404(MicroUnit, unit_id)
    pid = u.patch_id
    name = u.unit_type
    # 一并删除该单元的指标数据，避免残留
    IndicatorData.query.filter_by(unit_id=u.id).delete(synch
    ronize_session=False)
    db.session.delete(u)
    db.session.flush()
    recalc_ratio(pid)
    db.session.commit()
    flash("微地质单元「%s」已删除" % name, "ok")
    return redirect(url_for("unit.index", patch_id=pid))
# ==== app/views/standard.py ====
# -*- coding: utf-8 -*-
"""政策与标准知识库 —— 标准 / 政策文件的结构化规则与省际口径比对。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, Standard, StandardRule
bp = Blueprint("standard", __name__, url_prefix="/standard")
@bp.route("/")
@login_required
def index():
    standards = Standard.query.order_by(Standard.category,
    Standard.province).all()
    local = [s for s in standards if s.category == "地方标准"]
    policy = [s for s in standards if s.category != "地方标准"]
    return render_template("standard/index.html",
    local=local, policy=policy,
```

```
                                    form=EmptyForm())                    2651
@bp.route("/add", methods=["GET", "POST"])                               2652
@login_required                                                          2653
def add():                                                               2654
    form = EmptyForm()                                                   2655
    if request.method == "POST" and form.validate_on_submit(             2656
    ):                                                                   2657
        s = Standard(                                                    2658
            province=request.form.get("province",                        2659
    "").strip(),                                                         2660
            name=request.form.get("name", "").strip(),                   2661
            doc_number=request.form.get("doc_number",                    2662
    "").strip() or None,                                                 2663
            publish_date=request.form.get("publish_date",                2664
    "").strip() or None,                                                 2665
            implement_date=request.form.get("implement_date"             2666
    , "").strip() or None,                                               2667
            category=request.form.get("category", "地方标准"),           2668
            summary=request.form.get("summary", "").strip()              2669
    or None,                                                             2670
        )                                                                2671
        if not s.province or not s.name:                                 2672
            flash("省份与名称不能为空", "error")                          2673
        else:                                                            2674
            db.session.add(s)                                            2675
            db.session.commit()                                          2676
            flash("标准已添加", "ok")                                     2677
            return redirect(url_for("standard.index"))                   2678
    return render_template("standard/edit.html", form=form,              2679
    standard=None)                                                       2680
@bp.route("/<int:sid>/edit", methods=["GET", "POST"])                    2681
@login_required                                                          2682
def edit(sid):                                                           2683
    s = db.get_or_404(Standard, sid)                                     2684
    form = EmptyForm()                                                   2685
    if request.method == "POST" and form.validate_on_submit(             2686
    ):                                                                   2687
        s.province = request.form.get("province",                        2688
    "").strip()                                                          2689
        s.name = request.form.get("name", "").strip()                    2690
        s.doc_number = request.form.get("doc_number",                    2691
    "").strip() or None                                                  2692
        s.publish_date = request.form.get("publish_date",                2693
    "").strip() or None                                                  2694
        s.implement_date = request.form.get("implement_date"             2695
    , "").strip() or None                                                2696
        s.category = request.form.get("category", "地方标准")             2697
        s.summary = request.form.get("summary", "").strip()              2698
    or None                                                              2699
        db.session.commit()                                              2700
```

```
            flash("已保存", "ok")                                    2701
            return redirect(url_for("standard.rules", sid=s.id))     2702
        return render_template("standard/edit.html", form=form,      2703
        standard=s)                                                  2704
@bp.route("/<int:sid>/delete", methods=["POST"])                     2705
@login_required                                                      2706
def delete(sid):                                                     2707
    s = db.get_or_404(Standard, sid)                                 2708
    StandardRule.query.filter_by(standard_id=sid).delete()           2709
    db.session.delete(s)                                             2710
    db.session.commit()                                              2711
    flash("已删除", "ok")                                            2712
    return redirect(url_for("standard.index"))                       2713
@bp.route("/<int:sid>/rules", methods=["GET", "POST"])               2714
@login_required                                                      2715
def rules(sid):                                                      2716
    s = db.get_or_404(Standard, sid)                                 2717
    form = EmptyForm()                                               2718
    if request.method == "POST" and form.validate_on_submit(         2719
    ):                                                               2720
        r = StandardRule(                                            2721
            standard_id=s.id,                                        2722
            item=request.form.get("item", "").strip(),               2723
            operator=request.form.get("operator", ">="),             2724
            threshold=request.form.get("threshold",                  2725
    "").strip() or None,                                             2726
            condition_type=request.form.get("condition_type"         2727
    , "必要条件"),                                                   2728
            region=request.form.get("region", "").strip()            2729
    or None,                                                         2730
            note=request.form.get("note", "").strip() or             2731
    None,                                                            2732
        )                                                            2733
        if not r.item:                                               2734
            flash("判定项不能为空", "error")                         2735
        else:                                                        2736
            db.session.add(r)                                        2737
            db.session.commit()                                      2738
            flash("规则已添加", "ok")                                2739
        return redirect(url_for("standard.rules", sid=s.id))         2740
    rs = StandardRule.query.filter_by(standard_id=s.id).all(         2741
    )                                                                2742
    return render_template("standard/rules.html",                    2743
    form=form, standard=s, rules=rs)                                 2744
@bp.route("/rule/<int:rid>/delete", methods=["POST"])                2745
@login_required                                                      2746
def rule_delete(rid):                                                2747
    r = db.get_or_404(StandardRule, rid)                             2748
    sid = r.standard_id                                              2749
    db.session.delete(r)                                             2750
```

```
    db.session.commit()
    flash("规则已删除", "ok")
    return redirect(url_for("standard.rules", sid=sid))
@bp.route("/compare")
@login_required
def compare():
    """省际口径比对：同一指标在不同省份的阈值横向对比。"""
    rules = StandardRule.query.join(Standard).all()
    groups = {}
    for r in rules:
        groups.setdefault(r.item, []).append(r)
    return render_template("standard/compare.html",
    groups=groups)
# ==== app/views/constraint.py ====
# -*- coding: utf-8 -*-
"""限制性指标判定 —— 一票否决判定。
按论文表 2 与 4.1 节流程，共 5 项：政策、区位约束、土壤污染、地质灾害、耕地属性。
任一项不通过即判定为"人工辅助或生态重建"，不再进入定量评价。
"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, ConstraintCheck
from app.utils.constants import CONSTRAINT_ITEMS
bp = Blueprint("constraint", __name__,
    url_prefix="/constraint")
# 字段名 -> 中文名
FIELDS = [
    ("policy_pass", "政策约束"),
    ("location_pass", "区位约束"),
    ("pollution_pass", "土壤污染"),
    ("geo_pass", "地质灾害"),
    ("farmland_pass", "耕地属性"),
]
@bp.route("/")
@login_required
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    latest = {}
    for p in patches:
        latest[p.id] = ConstraintCheck.query.filter_by(patch
    _id=p.id) \
            .order_by(ConstraintCheck.id.desc()).first()
    return render_template("constraint/index.html",
    patches=patches, latest=latest)
@bp.route("/<int:patch_id>", methods=["GET", "POST"])
@login_required
def check(patch_id):
```

```
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    record = ConstraintCheck.query.filter_by(patch_id=patch_
    id) \
        .order_by(ConstraintCheck.id.desc()).first()
    if request.method == "POST" and form.validate_on_submit(
    ):
        if record is None:
            record = ConstraintCheck(patch_id=patch.id)
            db.session.add(record)
        for field, _ in FIELDS:
            setattr(record, field, request.form.get(field)
    == "1")
        record.passed = all(getattr(record, f) for f, _ in
    FIELDS)
        failed = [name for f, name in FIELDS if not
    getattr(record, f)]
        record.reason = ("全部通过" if record.passed
                         else "未通过：" + "、".join(failed))
        db.session.commit()
        flash("判定结果已保存", "ok")
        return redirect(url_for("constraint.check",
    patch_id=patch.id))
    # 预处理各条件的当前取值，供模板直接使用（Jinja 无 getattr）
    field_states = [(f, name, None if record is None else
    getattr(record, f))
                    for f, name in FIELDS]
    return render_template("constraint/check.html",
    patch=patch, form=form,
                           fields=FIELDS,
    field_states=field_states,
                           record=record)
# ==== app/views/ahp.py ====
# -*- coding: utf-8 -*-
"""指标体系与 AHP 权重 —— 路由与页面逻辑。"""
import json
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from app.forms import AHPSelectForm, EmptyForm
from app.models import db, Indicator, ExpertMatrix,
    WeightScheme
from app.utils.constants import DEFAULT_WEIGHTS, UNIT_TYPES
from app.services import ahp_service as ahp
from app.services import indicator_service as ind_svc
from app.services import weight_service as ws
bp = Blueprint("ahp", __name__, url_prefix="/ahp")
# 一次计算的权重结果对应的指标编码：名称 → 编码
_L1_CODE = {"土壤指标": "soil", "植被指标": "veg", "地形指标":
    "terrain"}
_L2_CODE = {
```

```
    ("土壤指标", "土石比"): "soil_ratio",
    ("土壤指标", "土层厚度"): "thickness",
    ("土壤指标", "pH"): "ph",
    ("土壤指标", "有机质"): "organic_matter",
    ("土壤指标", "污染指标"): "pollution",
    ("植被指标", "植被覆盖度"): "coverage",
    ("植被指标", "生物多样性"): "shannon",
    ("地形指标", "坡度"): "slope",
    ("地形指标", "坡向"): "aspect",
}
def to_code_pairs(unit_type, level, l1, names, weights):
    """把一次 AHP 计算的权重结果转成「指标编码 → 权重」，便于落库。"""
    pairs = []
    for name, w in zip(names, weights):
        if level == 1:
            code = _L1_CODE.get(name)
        else:
            code = _L2_CODE.get((l1, name))
        if code:
            pairs.append((code, w))
    return pairs
def indicator_list(unit_type, level, l1=None):
    """返回 (指标名称列表, 默认权重列表)。
    level=1 时返回一级指标；level=2 时返回指定一级指标下的二级指标。
    """
    weights = DEFAULT_WEIGHTS[unit_type]
    if level == 1:
        names = list(weights["l1"].keys())
        vec = [weights["l1"][n] for n in names]
        return names, vec
    # level == 2
    l2 = weights["l2"][l1]
    names = list(l2.keys())
    vec = [l2[n] for n in names]
    return names, vec
def l1_choices(unit_type):
    """某单元类型的一级指标名称列表（用于二级指标录入时选择父指标）。"""
    return list(DEFAULT_WEIGHTS[unit_type]["l1"].keys())
@bp.route("/", methods=["GET", "POST"])
def index():
    form = AHPSelectForm()
    # 先根据请求中的单元类型填充 l1 选项，再进行表单校验
    unit = request.form.get("unit_type") or
    form.unit_type.choices[0][0]
    form.l1.choices = [(n, n) for n in l1_choices(unit)]
    if form.validate_on_submit():
        unit_type = form.unit_type.data
        level = int(form.level.data)
        if level == 1:
            return redirect(url_for("ahp.matrix",
```

```
    unit_type=unit_type, level=1))
        if not form.l1.data:
            flash("选择「二级指标」时，请同时选择所属的一级指标", "error")
            return render_template("ahp/index.html",
    form=form,
                                   weight_rows=ws.summary())
        return redirect(url_for("ahp.matrix",
    unit_type=unit_type,
                                level=2, l1=form.l1.data))
    return render_template("ahp/index.html", form=form,
                           weight_rows=ws.summary())
@bp.route("/matrix", methods=["GET", "POST"])
def matrix():
    unit_type = request.values.get("unit_type")
    level = int(request.values.get("level", 1))
    l1 = request.values.get("l1")
    if unit_type not in DEFAULT_WEIGHTS:
        flash("无效的微地质单元类型", "error")
        return redirect(url_for("ahp.index"))
    names, vec = indicator_list(unit_type, level, l1)
    n = len(names)
    form = EmptyForm()
    # 预填默认值：论文表 3 权重对应的完全一致矩阵 a_ij = w_i / w_j
    default = [[round(vec[i] / vec[j], 4) for j in
    range(n)] for i in range(n)]
    def compute(upper):
        A = ahp.matrix_from_upper(upper)
        r = ahp.weights_and_consistency(A)
        bad = ahp.find_most_inconsistent(A)
        return {
            "weights": [round(v, 4) for v in r["weights"]],
            "lambda_max": round(r["lambda_max"], 4),
            "ci": r["ci"],
            "cr": r["cr"],
            "consistent": r["consistent"],
            "matrix": [[round(A[i, j], 4) for j in
    range(n)] for i in range(n)],
            "most_inconsistent": bad,
        }
    result = None
    if request.method == "POST" and form.validate_on_submit(
    ):
        action = request.form.get("action", "calc")
        try:
            upper = [float(request.form.get("m_%d_%d" % (i,
    j), "1"))
                     for i in range(n) for j in range(i +
    1, n)]
            result = compute(upper)
        except (ValueError, ZeroDivisionError):
```

```
            flash("矩阵输入有误：请填入正数（对角线自动为 1，下三角自动取倒数）",
    "error")
            return redirect(url_for("ahp.matrix",
    unit_type=unit_type,
                                    level=level, l1=l1))
        if action == "save":
            if not result["consistent"]:
                flash("一致性比例 CR=%.4f ≥ 0.1，判断矩阵不满足一致性要求，"
                      "请调整后再保存" % result["cr"], "error")
            else:
                pairs = to_code_pairs(unit_type, level, l1,
    names,
                                      result["weights"])
                scheme, cnt = ws.save_weights(unit_type,
    pairs)
                flash("已保存并为「%s」生效 %d 项权重（CR=%.4f）。"
                      "此后该单元类型的评价将以此权重计算。"
                      % (unit_type, cnt, result["cr"]),
    "ok")
                return redirect(url_for("ahp.matrix",
    unit_type=unit_type,
                                        level=level, l1=l1))
    elif request.values.get("calc") == "1":
        # 按默认（论文表 3）矩阵直接试算，便于一键查看权重与一致性
        result = compute([default[i][j] for i in range(n)
    for j in range(i + 1, n)])
    _, source_label, custom_scheme = ws.effective_weights(un
    it_type)
    return render_template(
        "ahp/matrix.html",
        form=form, unit_type=unit_type, level=level, l1=l1,
        names=names, vec=vec, default=default,
    result=result,
        source_label=source_label, custom_scheme=custom_sche
    me,
    )
@bp.route("/weights/reset", methods=["POST"])
def weights_reset():
    """删除自定义权重方案，恢复为该单元类型的论文表 3 默认权重。"""
    unit_type = request.form.get("unit_type")
    form = EmptyForm()
    if form.validate_on_submit():
        if unit_type not in DEFAULT_WEIGHTS:
            flash("无效的微地质单元类型", "error")
        else:
            n = ws.reset_weights(unit_type)
            if n:
                flash("已恢复「%s」为论文表 3 默认权重" % unit_type,
    "ok")
            else:
```

```
                flash("「%s」本就是论文表 3 默认权重，无需恢复" % unit_type,
    "ok")
    return redirect(url_for("ahp.index"))
# ---------------------------- 指标维护 ----------------------------
    ----
@bp.route("/indicators")
def indicators():
    """指标定义维护：查看指标树、单位、理想值与取值依据。"""
    if Indicator.query.count() == 0:
        ind_svc.seed_indicators()
    all_ind = Indicator.query.order_by(Indicator.level,
    Indicator.id).all()
    by_parent = {}
    for i in all_ind:
        by_parent.setdefault(i.parent_code, []).append(i)
    return render_template("ahp/indicators.html",
    by_parent=by_parent,
                           all_ind=all_ind,
    unit_types=UNIT_TYPES,
                           type_label=ind_svc.ideal_type_lab
    el,
                           form=EmptyForm())
@bp.route("/indicators/<code>/edit", methods=["GET",
    "POST"])
def indicator_edit(code):
    ind = Indicator.query.filter_by(code=code).first_or_404(
    )
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        name = request.form.get("name", "").strip()
        if not name:
            flash("指标名称不能为空", "error")
            return redirect(url_for("ahp.indicator_edit",
    code=code))
        ind.name = name
        ind.unit = request.form.get("unit", "").strip() or
    None
        ind.ideal_value = request.form.get("ideal_value",
    "").strip() or None
        ind.ideal_type = request.form.get("ideal_type") or
    ind.ideal_type
        units = request.form.getlist("enabled_units")
        ind.enabled_units = ",".join(units) if units else
    None
        ind.is_constraint = request.form.get("is_constraint"
    ) == "1"
        ind.description = request.form.get("description",
    "").strip() or None
        db.session.commit()
```

```
            flash("指标「%s」已保存" % ind.name, "ok")                    3051
            return redirect(url_for("ahp.indicators"))                   3052
        return render_template("ahp/indicator_edit.html",                3053
        ind=ind,                                                         3054
                                    unit_types=UNIT_TYPES,               3055
                                    type_label=ind_svc.ideal_type_lab3056
        el, form=form)                                                   3057
@bp.route("/indicators/reset", methods=["POST"])                         3058
def indicators_reset():                                                  3059
    """按论文取值重置指标定义。"""                                          3060
    form = EmptyForm()                                                   3061
    if form.validate_on_submit():                                        3062
        n = ind_svc.seed_indicators()                                    3063
        flash("已按论文取值重置 %d 项指标定义" % n, "ok")                    3064
    return redirect(url_for("ahp.indicators"))                           3065
# ---------------------------- 多专家矩阵 ----------------------------3066
    -----                                                                3067
def expert_scope():                                                      3068
    unit_type = request.values.get("unit_type") or                       3069
    UNIT_TYPES[0]                                                        3070
    level = int(request.values.get("level", 1))                         3071
    l1 = request.values.get("l1") or None                                3072
    return unit_type, level, l1                                          3073
def expert_query(unit_type, level, l1):                                  3074
    q = ExpertMatrix.query.filter_by(unit_type=unit_type,                3075
    level=level)                                                         3076
    if level == 2:                                                       3077
        q = q.filter_by(l1_name=l1)                                      3078
    else:                                                                3079
        q = q.filter(ExpertMatrix.l1_name.is_(None))                     3080
    return q.order_by(ExpertMatrix.id)                                   3081
def default_scheme(unit_type, level, l1=None):                           3082
    """取（或建）该范围的权重方案，专家矩阵统一挂到它下面。"""                3083
    name = "%s · %s%s" % (unit_type,                                     3084
                          "一级指标" if level == 1 else "二级指标",        3085
                          "(%s)" % l1 if l1 else "")                     3086
    scheme = WeightScheme.query.filter_by(unit_type=unit_typ3087
    e,                                                                   3088
                                          level=level,                   3089
    name=name).first()                                                   3090
    if scheme is None:                                                   3091
        scheme = WeightScheme(unit_type=unit_type,                       3092
    level=level, name=name,                                              3093
                                  is_default=False)                      3094
        db.session.add(scheme)                                           3095
        db.session.flush()                                               3096
    return scheme                                                        3097
@bp.route("/experts", methods=["GET", "POST"])                           3098
def experts():                                                           3099
    """多专家判断矩阵录入与汇总。"""                                        3100
```

```
    unit_type, level, l1 = expert_scope()                          3101
    if unit_type not in DEFAULT_WEIGHTS:                           3102
        flash("无效的微地质单元类型", "error")                      3103
        return redirect(url_for("ahp.index"))                      3104
    names, vec = indicator_list(unit_type, level, l1)              3105
    n = len(names)                                                 3106
    form = EmptyForm()                                             3107
    if request.method == "POST" and form.validate_on_submit(       3108
    ):                                                             3109
        action = request.form.get("action")                        3110
        if action == "add":                                        3111
            expert_name = request.form.get("expert_name",          3112
    "").strip() or "未具名专家"                                     3113
            try:                                                   3114
                upper = [float(request.form.get("m_%d_%d" %        3115
    (i, j), "1")) for i in range(n) for j in range(i               3116
    + 1, n)]                                                       3117
            except ValueError:                                     3118
                flash("矩阵输入有误，请填入正数", "error")           3119
                return redirect(url_for("ahp.experts",             3120
    unit_type=unit_type, level=level, l1=l1))                      3121
            A = ahp.matrix_from_upper(upper)                       3122
            r = ahp.weights_and_consistency(A)                     3123
            scheme = default_scheme(unit_type, level, l1)          3124
            db.session.add(ExpertMatrix(                           3125
                scheme_id=scheme.id,                               3126
                unit_type=unit_type, l1_name=(l1 if level          3127
    == 2 else None),                                               3128
                expert_name=expert_name, level=level,              3129
                matrix_json=json.dumps([list(map(float,            3130
    row)) for row in A]),                                          3131
                cr=round(r["cr"], 6)))                             3132
            db.session.commit()                                    3133
            flash("已录入专家「%s」的判断矩阵（CR=%.4f）" % (expert_name,  3134
    r["cr"]), "ok")                                                3135
        elif action == "save_agg":                                 3136
            # 把汇总权重落库并生效，使多专家结果真正参与评价计算      3137
            weights = _aggregate(unit_type, level, l1)             3138
            if weights is None:                                    3139
                flash("暂无可汇总的专家矩阵", "error")              3140
            elif not weights["consistent"]:                        3141
                flash("汇总矩阵一致性比例 CR=%.4f ≥ 0.1，不满足要求，"  3142
                      "请先检查或调整专家矩阵" % weights["cr"],      3143
    "error")                                                       3144
            else:                                                  3145
                names_, _ = indicator_list(unit_type,              3146
    level, l1)                                                     3147
                pairs = to_code_pairs(unit_type, level, l1,        3148
```

```
    names_,                                                          3151
                                                weights["weights"])  3152
                  _, cnt = ws.save_weights(unit_type, pairs)         3153
                  flash("已保存并为「%s」生效 %d 项汇总权重（CR=%.4f）"  3154
                        % (unit_type, cnt, weights["cr"]),           3155
    "ok")                                                            3156
        return redirect(url_for("ahp.experts",                       3157
    unit_type=unit_type,                                             3158
                                        level=level, l1=l1))         3159
    rows = expert_query(unit_type, level, l1).all()                  3160
    default = [[round(vec[i] / vec[j], 4) for j in                   3161
    range(n)] for i in range(n)]                                     3162
    return render_template("ahp/experts.html",                       3163
    unit_type=unit_type, level=level,                                3164
                                  l1=l1, names=names, rows=rows,     3165
    default=default,                                                 3166
                                  n=n, form=form)                    3167
@bp.route("/experts/<int:mid>/delete", methods=["POST"])             3168
def expert_delete(mid):                                              3169
    row = db.get_or_404(ExpertMatrix, mid)                           3170
    unit_type, level, l1 = row.unit_type, row.level,                 3171
    row.l1_name                                                      3172
    form = EmptyForm()                                               3173
    if form.validate_on_submit():                                    3174
        db.session.delete(row)                                       3175
        db.session.commit()                                          3176
        flash("已删除专家矩阵", "ok")                                3177
    return redirect(url_for("ahp.experts",                           3178
    unit_type=unit_type,                                             3179
                                    level=level, l1=l1))             3180
def _aggregate(unit_type, level, l1):                                3181
    """按几何平均汇总多专家矩阵并做一致性检验。无矩阵时返回 None。"""  3182
    rows = expert_query(unit_type, level, l1).all()                  3183
    mats = []                                                        3184
    for r in rows:                                                   3185
        try:                                                         3186
            mats.append([list(map(float, line))                      3187
                         for line in json.loads(r.matrix_jso3188
    n)])                                                             3189
        except Exception:                                            3190
            continue                                                 3191
    if not mats:                                                     3192
        return None                                                  3193
    agg = ahp.aggregate_matrices(mats)                               3194
    res = ahp.weights_and_consistency(agg)                           3195
    return {                                                         3196
        "count": len(mats),                                          3197
        "weights": res["weights"],                                   3198
        "lambda_max": res["lambda_max"],                             3199
        "ci": res["ci"], "cr": res["cr"],                            3200
```

```
            "consistent": res["consistent"],
            "most_inconsistent": ahp.find_most_inconsistent(agg)
        ,
            "expert_crs": [r.cr for r in rows if r.cr is not
        None],
        }
@bp.route("/experts/aggregate")
def experts_aggregate():
        """按几何平均汇总多专家矩阵，并做一致性检验。"""
        unit_type, level, l1 = expert_scope()
        if unit_type not in DEFAULT_WEIGHTS:
            flash("无效的微地质单元类型", "error")
            return redirect(url_for("ahp.index"))
        names, vec = indicator_list(unit_type, level, l1)
        rows = expert_query(unit_type, level, l1).all()
        result = _aggregate(unit_type, level, l1)
        if result is not None:
            result["weights"] = [round(v, 4) for v in
        result["weights"]]
            result["lambda_max"] = round(result["lambda_max"],
        4)
        return render_template("ahp/experts_aggregate.html",
        unit_type=unit_type,
                               level=level, l1=l1, names=names,
        rows=rows,
                               result=result, form=EmptyForm())
#  ==== app/views/indicator.py ====
#  -*- coding: utf-8 -*-
"""指标数据采集与管理 —— 路由与页面逻辑。
按微地质单元组织录入；系统按单元类型自动决定应录入哪些指标
（岩质边坡无土壤指标、平台无地形指标）。
"""
import io
import json
from flask import (Blueprint, render_template, request,
        redirect, url_for,
                   flash, send_file)
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
        IndicatorData
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
        UNIT_ENABLED_INDICATORS
from app.services import import_service as imp
bp = Blueprint("indicator", __name__,
        url_prefix="/indicator")
#  指标字段定义：字段名 -> （中文名， 单位， 是否可选）
SOIL_FIELDS = [
        ("soil_ratio", "土石比", ""),
        ("thickness", "土层厚度", "cm"),
        ("ph", "pH", ""),
```

```
    ("organic_matter", "有机质", "g/kg"),
]
VEG_FIELDS = [
    ("coverage", "植被覆盖度", "%"),
    ("shannon", "Shannon-Wiener 多样性指数", ""),
]
TERRAIN_FIELDS = [
    ("slope", "坡度", "°"),
    ("aspect", "坡向", ""),
]
def unit_field_config(unit_type):
    """根据单元类型返回需录入的字段分组。"""
    enabled = UNIT_ENABLED_INDICATORS.get(unit_type, ())
    groups = []
    if "土壤指标" in enabled:
        groups.append(("土壤指标", SOIL_FIELDS, True))  # True = 含重金属
    if "植被指标" in enabled:
        groups.append(("植被指标", VEG_FIELDS, False))
    if "地形指标" in enabled:
        groups.append(("地形指标", TERRAIN_FIELDS, False))
    return groups
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).all()
    return render_template("indicator/index.html", patches=patches)
@bp.route("/<int:unit_id>", methods=["GET", "POST"])
def edit(unit_id):
    unit = db.get_or_404(MicroUnit, unit_id)
    ind = IndicatorData.query.filter_by(unit_id=unit.id).first()
    form = EmptyForm()
    groups = unit_field_config(unit.unit_type)
    if request.method == "POST" and form.validate_on_submit():
        def f(name, cast=float):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        heavy = {}
        for m in HEAVY_METALS:
            v = f("hm_" + m)
            if v is not None:
                heavy[m] = v
        if ind is None:
            ind = IndicatorData(unit_id=unit.id)
            db.session.add(ind)
        ind.soil_ratio = f("soil_ratio")
        ind.thickness = f("thickness")
```

```
            ind.ph = f("ph")                                                3301
            ind.organic_matter = f("organic_matter")                        3302
            ind.heavy_metals = json.dumps(heavy,                            3303
        ensure_ascii=False) if heavy else None                              3304
            ind.coverage = f("coverage")                                    3305
            ind.shannon = f("shannon")                                      3306
            ind.slope = f("slope")                                          3307
            ind.aspect = request.form.get("aspect", "").strip()             3308
        or None                                                             3309
            db.session.commit()                                             3310
            flash("指标数据已保存", "ok")                                    3311
            return redirect(url_for("indicator.index"))                     3312
        # 组装当前值供模板回显                                              3313
        current = {}                                                        3314
        if ind:                                                             3315
            current = {                                                     3316
                "soil_ratio": ind.soil_ratio, "thickness":                  3317
        ind.thickness,                                                      3318
                "ph": ind.ph, "organic_matter":                             3319
        ind.organic_matter,                                                 3320
                "coverage": ind.coverage, "shannon":                        3321
        ind.shannon,                                                        3322
                "slope": ind.slope, "aspect": ind.aspect,                   3323
            }                                                               3324
            current["heavy"] = ind.heavy_metals_dict()                      3325
        return render_template("indicator/edit.html",                       3326
                               unit=unit, form=form,                        3327
        groups=groups,                                                      3328
                               heavy_metals=HEAVY_METALS,                   3329
                               aspect_types=ASPECT_TYPES,                   3330
        current=current)                                                    3331
# ------------------------------ 批量导入 ------------------------------3332
        ----                                                                3333
@bp.route("/import", methods=["GET", "POST"])                               3334
def import_data():                                                          3335
        """批量导入指标数据（CSV / Excel）。"""                             3336
        patches = MinePatch.query.order_by(MinePatch.id.desc()).3337
        all()                                                               3338
        form = EmptyForm()                                                  3339
        report = None                                                       3340
        if request.method == "POST" and form.validate_on_submit(3341
        ):                                                                  3342
            patch_id = request.form.get("patch_id", type=int)               3343
            patch = db.session.get(MinePatch, patch_id) if                  3344
        patch_id else None                                                  3345
            if patch is None:                                               3346
                flash("请选择矿山图斑", "error")                             3347
                return redirect(url_for("indicator.import_data")3348
        )                                                                   3349
            file = request.files.get("datafile")                            3350
```

```
    if file is None or not file.filename:
        flash("请选择要导入的文件", "error")
        return redirect(url_for("indicator.import_data"))
    ext = file.filename.rsplit(".", 1)[-1].lower()
    if ext not in ("csv", "xlsx", "xls"):
        flash("仅支持 CSV 或 Excel(.xlsx/.xls)文件", "error")
        return redirect(url_for("indicator.import_data"))
    units = MicroUnit.query.filter_by(patch_id=patch.id).all()
    unit_map = {}
    for u in units:
        unit_map.setdefault(u.unit_type, u)
    if not unit_map:
        flash("该图斑尚未划分微地质单元，请先划分后再导入", "error")
        return redirect(url_for("indicator.import_data"))
    try:
        parsed, errors = imp.parse_upload(file.filename, file.stream, unit_map)
    except Exception as e:
        flash("文件解析失败：%s" % e, "error")
        return redirect(url_for("indicator.import_data"))
    saved = 0
    for rec in parsed:
        unit = unit_map[rec["unit_type"]]
        ind = IndicatorData.query.filter_by(unit_id=unit.id).first()
        if ind is None:
            ind = IndicatorData(unit_id=unit.id)
            db.session.add(ind)
        f = rec["fields"]
        for key in ("soil_ratio", "thickness", "ph", "organic_matter",
                    "coverage", "shannon", "slope", "aspect"):
            if key in f:
                setattr(ind, key, f[key])
        if rec["heavy_metals"]:
            ind.heavy_metals = json.dumps(rec["heavy_metals"],
                                          ensure_ascii=False)
        saved += 1
    db.session.commit()
    report = {"patch": patch.name, "saved": saved, "errors": errors,
              "total": len(parsed) + len(
```

```
                              [e for e in errors if e.startswith("第 3401
    ")])}                                                             3402
        if saved:                                                     3403
            flash("已导入 %d 个微地质单元的指标数据" % saved, "ok")      3404
        else:                                                         3405
            flash("没有可导入的数据行，请检查文件内容", "error")         3406
    return render_template("indicator/import.html",                   3407
    patches=patches,                                                  3408
                                    form=form, report=report,         3409
    headers=imp.HEADERS)                                              3410
@bp.route("/import/template")                                         3411
def import_template():                                                3412
    """下载导入模板（CSV，含一行示例）。"""                              3413
    csv_text = imp.build_template_csv()                               3414
    data = csv_text.encode("utf-8-sig")       # 带 BOM，便于          3415
    Excel 正确识别中文                                                 3416
    return send_file(io.BytesIO(data), as_attachment=True,            3417
                     download_name="指标数据导入模板.csv",              3418
                     mimetype="text/csv")                             3419
# ==== app/views/evaluate.py ====                                     3420
# -*- coding: utf-8 -*-                                               3421
"""适宜性定量评价引擎 —— 路由与页面逻辑。                              3422
从数据库加载某图斑的微地质单元与指标数据，调用评价引擎计算，          3423
展示各单元得分、最终评价因子 F、适宜性等级与推荐措施。                3424
入口处按论文 4.1 节的流程顺序检查前置条件（见 app/services/workflow.py），3425
前置步骤未完成时不予计算，避免在条件不具备时产出评价结论。            3426
"""                                                                   3427
import json                                                           3428
from flask import Blueprint, render_template, request,                3429
    redirect, url_for, flash                                          3430
from app.forms import EmptyForm                                       3431
from app.models import (db, MinePatch, MicroUnit,                     3432
    IndicatorData,                                                    3433
                        EvaluationResult, ScoreDetail)                3434
from app.services import evaluation_service as ev                     3435
from app.services import workflow as wf                               3436
bp = Blueprint("evaluate", __name__, url_prefix="/evaluate")          3437
def load_patch_input(patch_id):                                       3438
    """从数据库组装评价引擎输入。"""                                    3439
    units = MicroUnit.query.filter_by(patch_id=patch_id).all3440
    ()                                                                3441
    result = []                                                       3442
    for u in units:                                                   3443
        ind = IndicatorData.query.filter_by(unit_id=u.id).fi3444
    rst()                                                             3445
        data = {                                                      3446
            "soil_ratio": ind.soil_ratio if ind else None,            3447
            "thickness": ind.thickness if ind else None,              3448
            "ph": ind.ph if ind else None,                            3449
            "organic_matter": ind.organic_matter if ind               3450
```

```
        else None,
            "heavy_metals": ind.heavy_metals_dict() if ind
        else {},
            "coverage": ind.coverage if ind else None,
            "shannon": ind.shannon if ind else None,
            "slope": ind.slope if ind else None,
            "aspect": ind.aspect if ind else None,
        }
        result.append({
            "unit_id": u.id,
            "unit_type": u.unit_type,
            "ratio": u.ratio if u.ratio is not None else
        0.0,
            "data": data,
        })
    return result
def _round(v, n=6):
    return None if v is None else round(v, n)
def load_weights(units):
    """解析各单元类型当前生效的权重与来源说明。
    权重可来自 AHP 计算并保存的生效方案；未配置时用论文表 3 的默认值。
    来源一并记录，便于报告标注与事后复核。
    """
    from app.services import weight_service as ws
    weights, sources = {}, {}
    for u in units:
        ut = u["unit_type"]
        if ut not in weights:
            w, label, _ = ws.effective_weights(ut)
            weights[ut] = w
            sources[ut] = label
    return weights, sources
def pack_units(unit_results, sources=None):
    """把评价结果整理为可持久化的 JSON。
    完整保留各单元得分、一级指标得分与逐条二级指标明细，
    使结果页与报告能够复现「已记录的那一次评价」，
    而不是每次打开时重新推算。
    """
    sources = sources or {}
    out = []
    for u in unit_results:
        out.append({
            "unit_type": u["unit_type"],
            "ratio": _round(u["ratio"]),
            "unit_score": _round(u["unit_score"]),
            "weighted": _round(u["weighted"]),
            "weight_source": sources.get(u["unit_type"]),
            "l1_scores": {k: _round(v) for k, v in
        u["l1_scores"].items()},
            "details": [{
```

```
                    "l1": d["l1"], "l2": d["l2"],
                    "raw": None if d["raw"] is None else
        str(d["raw"]),
                    "std_value": _round(d["std_value"]),
                    "weight": _round(d["weight"]),
                    "contribution": _round(d["contribution"]),
                } for d in u["details"]],
            })
        return out
def save_evaluation(patch_id, units):
    """执行评价并把结果与逐层明细落库，返回评价记录。
    权重取各单元类型当前生效的方案（AHP 计算结果），未配置时用
    论文表 3 的默认权重；来源一并记录。结果 JSON 保留各单元得分、
    一级指标得分与逐条二级指标明细，结果页与报告据此复现
    记录下来的那次评价，不再重新推算。
    """
    weights, sources = load_weights(units)
    res = ev.score_mine(units, weights)
    saved = EvaluationResult(
        patch_id=patch_id,
        final_score=round(res["final_score"], 4),
        grade=res["grade"],
        measures=res["grade_info"]["measure"],
        unit_scores=json.dumps(pack_units(res["unit_results"
    ], sources),
                               ensure_ascii=False),
    )
    db.session.add(saved)
    db.session.flush()        # 先取得 result.id，再写明细
    unit_ids = {u["unit_type"]: u["unit_id"] for u in units}
    for u in res["unit_results"]:
        for d in u["details"]:
            db.session.add(ScoreDetail(
                result_id=saved.id,
                unit_id=unit_ids.get(u["unit_type"]),
                l1_name=d["l1"], l2_name=d["l2"],
                raw_value=None if d["raw"] is None else
    str(d["raw"]),
                std_value=d["std_value"],
    weight=d["weight"],
                contribution=d["contribution"]))
    db.session.commit()
    return saved
def block_reasons(progress):
    """把未满足的前置条件转成可读的拦截原因。"""
    return ["%s：%s" % (b["label"], b["reason"]) for b in
    progress["blocking"]]
@bp.route("/")
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
```

```
    all()
    latest = {}
    progress = {}
    for p in patches:
        latest[p.id] = EvaluationResult.query.filter_by(patch_id=p.id) \
            .order_by(EvaluationResult.id.desc()).first()
        progress[p.id] = wf.patch_progress(p.id)
    return render_template("evaluate/index.html",
    patches=patches,
                           latest=latest, progress=progress)
@bp.route("/<int:patch_id>", methods=["GET", "POST"])
def run(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    progress = wf.patch_progress(patch_id)
    if request.method == "POST" and form.validate_on_submit():
        if not progress["ready"]:
            for reason in block_reasons(progress):
                flash("无法执行评价 —— " + reason, "error")
            return redirect(url_for("evaluate.run",
    patch_id=patch_id))
        units = load_patch_input(patch_id)
        saved = save_evaluation(patch.id, units)
        return redirect(url_for("evaluate.result",
    result_id=saved.id))
    # 展示将采用的权重来源，便于在执行前确认
    units_now = load_patch_input(patch_id)
    _, sources = load_weights(units_now) if units_now else
    ({}, {})
    return render_template("evaluate/run.html",
    patch=patch, form=form,
                           progress=progress,
    weight_sources=sources)
@bp.route("/result/<int:result_id>")
def result(result_id):
    r = db.get_or_404(EvaluationResult, result_id)
    patch = db.session.get(MinePatch, r.patch_id)
    unit_scores = json.loads(r.unit_scores) if
    r.unit_scores else []
    return render_template("evaluate/result.html", r=r,
    patch=patch,
                           unit_scores=unit_scores)
# ==== app/views/report.py ====
# -*- coding: utf-8 -*-
"""报告与结果输出 —— 得分明细、图表、Word / PDF 导出。"""
import io
import json
from datetime import datetime
```

```
from flask import Blueprint, render_template, send_file,
    flash, redirect, url_for
from app.models import db, MinePatch, ReferenceEcosystem
from app.services import report_service as rs
from app.services import workflow as wf
from app.utils import charts
from app.utils import browser
from app.utils.constants import grade_of
bp = Blueprint("report", __name__, url_prefix="/report")
def _prepare(patch_id):
    """加载图斑并取出已记录的评价结果，供明细页与导出复用。
    报告是评价结论的呈现，因此这里读取最近一次保存的评价记录，
    而不是重新计算：评价完成后再改动指标数据，也不会让报告与
    系统记录的评价结论对不上。没有任何评价记录时，返回流程检查
    结果，由调用方拦截并提示先去执行评价。
    """
    patch = db.get_or_404(MinePatch, patch_id)
    prog = wf.report_progress(patch_id)
    if not prog["ready"]:
        return patch, None, None, None, prog
    rec = prog["result"]
    result = {
        "final_score": rec.final_score,
        "grade": rec.grade,
        "grade_info": grade_of(rec.final_score),
        "unit_results": json.loads(rec.unit_scores) if
    rec.unit_scores else [],
    }
    reference = ReferenceEcosystem.query.filter_by(patch_id=
    patch_id).first()
    return patch, reference, result, rec, prog
def _deny(progress, patch_id, endpoint="report.detail"):
    """前置条件未满足时的统一处理：逐条提示并退回报告列表。"""
    for b in progress["blocking"]:
        flash("暂不能生成报告 —— %s：%s" % (b["label"],
    b["reason"]), "error")
    if progress["blocking"]:
        flash("请先完成上述步骤后再生成报告。", "error")
    return redirect(url_for("report.index"))
def _charts(result):
    """生成两张图表：单元得分对比 + 一级指标平均得分分布。"""
    units = [(u["unit_type"], u["unit_score"],
              "%.4f" % u["unit_score"]) for u in
    result["unit_results"]]
    chart_units = charts.hbars(units)
    # 各一级指标在各单元上的平均得分，反映其对评价结果的实际影响
    acc, cnt = {}, {}
    for u in result["unit_results"]:
        for k, v in (u.get("l1_scores") or {}).items():
            acc[k] = acc.get(k, 0.0) + v
```

```
            cnt[k] = cnt.get(k, 0) + 1                                  3651
    items = [(k, acc[k] / cnt[k], "%.4f" % (acc[k] /                    3652
    cnt[k])) for k in acc]                                              3653
    items.sort(key=lambda x: -x[1])                                     3654
    chart_weight = charts.donut(items)                                  3655
    return chart_units, chart_weight                                    3656
@bp.route("/")                                                          3657
def index():                                                            3658
    patches = MinePatch.query.order_by(MinePatch.id.desc()).            3659
    all()                                                               3660
    progress = {p.id: wf.report_progress(p.id) for p in                 3661
    patches}                                                            3662
    return render_template("report/index.html",                         3663
    patches=patches,                                                    3664
                                progress=progress)                      3665
@bp.route("/<int:patch_id>")                                            3666
def detail(patch_id):                                                   3667
    patch, reference, result, record, prog =                            3668
    _prepare(patch_id)                                                  3669
    if result is None:                                                  3670
        return _deny(prog, patch_id)                                    3671
    chart_units, chart_weight = _charts(result)                         3672
    return render_template("report/detail.html",                        3673
    patch=patch, result=result,                                         3674
                                record=record,                          3675
    reference=reference,                                                3676
                                chart_units=chart_units,                3677
    chart_weight=chart_weight)                                          3678
@bp.route("/<int:patch_id>/word")                                       3679
def word(patch_id):                                                     3680
    patch, reference, result, record, prog =                            3681
    _prepare(patch_id)                                                  3682
    if result is None:                                                  3683
        return _deny(prog, patch_id)                                    3684
    buf = rs.build_report(patch, result)                                3685
    filename = "自然恢复适宜性评价报告_%s.docx" % patch.name              3686
    return send_file(buf, as_attachment=True,                           3687
    download_name=filename,                                             3688
                          mimetype="application/vnd.openxmlformat       3689
    s-officedocument."                                                  3690
                                    "wordprocessingml.document")        3691
@bp.route("/<int:patch_id>/pdf")                                        3692
def pdf(patch_id):                                                      3693
    """导出 PDF 报告：服务端渲染打印模板，再交浏览器无头打印。"""                3694
    patch, reference, result, record, prog =                            3695
    _prepare(patch_id)                                                  3696
    if result is None:                                                  3697
        return _deny(prog, patch_id)                                    3698
    chart_units, chart_weight = _charts(result)                         3699
    html = render_template(                                             3700
```

```
            "report/print.html", patch=patch,                          3701
        reference=reference, result=result,                            3702
            chart_units=chart_units, chart_weight=chart_weight,        3703
            generated_at=datetime.now().strftime("%Y-%m-%d             3704
        %H:%M"))                                                       3705
        data, err = browser.html_to_pdf(html)                          3706
        if err:                                                        3707
            flash("PDF 导出失败：%s" % err, "error")                    3708
            return redirect(url_for("report.detail",                   3709
        patch_id=patch_id))                                            3710
        return send_file(io.BytesIO(data), as_attachment=True,         3711
                         download_name="自然恢复适宜性评价报告_%s.pdf" %    3712
        patch.name,                                                    3713
                         mimetype="application/pdf")                   3714
# ==== app/templates/base.html ====                                    3715
<!DOCTYPE html>                                                        3716
<html lang="zh-CN">                                                    3717
<head>                                                                 3718
  <meta charset="UTF-8">                                               3719
  <meta name="viewport" content="width=device-width,                   3720
    initial-scale=1.0">                                                3721
  <link rel="icon" href="{{ url_for('static',                          3722
    filename='favicon.ico') }}">                                       3723
  <title>{% block title %}历史遗留废弃矿山自然恢复适宜性评价系统{% endblock     3724
    %}</title>                                                         3725
  <style>                                                              3726
    /* ---------- 设计变量：单一强调色 + 中性灰阶 ---------- */                3727
    :root {                                                            3728
      --accent: #1f3a5f;                                               3729
      --accent-hover: #2a4d7c;                                         3730
      --accent-soft: #eef2f7;                                          3731
      --danger: #b3261e;                                               3732
      --danger-soft: #fdecea;                                          3733
      --ok: #1e6b3a;                                                   3734
      --ok-soft: #eaf6ef;                                              3735
      --ink: #23262b;                                                  3736
      --ink-soft: #6b7280;                                             3737
      --line: #dfe3e8;                                                 3738
      --surface: #ffffff;                                              3739
      --canvas: #f5f6f8;                                               3740
      --radius: 6px;                                                   3741
      --shadow: 0 1px 2px rgba(0, 0, 0, .05);                          3742
      --header-h: 56px;                                                3743
      --nav-w: 208px;                                                  3744
    }                                                                  3745
    * { box-sizing: border-box; }                                      3746
    html, body { margin: 0; padding: 0; }                              3747
    body {                                                             3748
      font-family: "Microsoft YaHei", "PingFang SC",                   3749
    "SimSun", sans-serif;                                              3750
```

```
    font-size: 14px;
    line-height: 1.6;
    color: var(--ink);
    background: var(--canvas);
}
/* ---------- 跳到主要内容（键盘可达性） ---------- */
.skip-link {
    position: absolute;
    left: 12px;
    top: -60px;
    z-index: 30;
    padding: 8px 14px;
    background: var(--accent);
    color: #fff;
    border-radius: var(--radius);
    text-decoration: none;
    transition: top .15s ease-out;
}
.skip-link:focus { top: 12px; }
/* ---------- 顶部标题栏 ---------- */
.app-header {
    display: flex;
    align-items: center;
    gap: 12px;
    height: var(--header-h);
    padding: 0 20px;
    background: var(--accent);
    color: #fff;
}
.app-header__title {
    font-size: 16px;
    font-weight: 600;
    letter-spacing: .2px;
}
.app-header__version {
    padding: 1px 8px;
    font-size: 12px;
    border: 1px solid rgba(255, 255, 255, .45);
    border-radius: 10px;
    font-variant-numeric: tabular-nums;
}
.app-header__spacer { flex: 1; }
.app-header__user {
    display: flex;
    align-items: center;
    gap: 10px;
    font-size: 13px;
}
.app-header__user a {
    color: #d6e3f4;
```

```
    text-decoration: none;
    padding: 4px 8px;
    border-radius: 4px;
}
.app-header__user a:hover,
.app-header__user a:focus-visible {
    color: #fff;
    background: rgba(255, 255, 255, .14);
}
/* ---------- 主体：左侧导航 + 内容区 ---------- */
.app-body {
    display: flex;
    align-items: stretch;
    min-height: calc(100dvh - var(--header-h));
}
.app-nav {
    flex: 0 0 var(--nav-w);
    width: var(--nav-w);
    padding: 14px 10px;
    background: var(--surface);
    border-right: 1px solid var(--line);
}
.app-nav__group {
    margin: 0 0 4px;
    padding: 10px 10px 6px;
    font-size: 12px;
    color: var(--ink-soft);
}
.app-nav ul {
    margin: 0;
    padding: 0;
    list-style: none;
}
.app-nav a {
    display: block;
    padding: 9px 12px;
    margin-bottom: 2px;
    border-radius: var(--radius);
    color: var(--ink);
    text-decoration: none;
    border-left: 3px solid transparent;
}
.app-nav a:hover { background: var(--accent-soft); }
.app-nav a:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 1px;
}
.app-nav a[aria-current="page"] {
    background: var(--accent-soft);
    border-left-color: var(--accent);
```

```
  color: var(--accent);
  font-weight: 600;
}
.app-main {
  flex: 1;
  min-width: 0;
  padding: 22px 26px 40px;
}
.container { max-width: 1040px; }
/* ---------- 卡片 ---------- */
.card {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 22px 24px;
  margin-bottom: 18px;
}
.card > h2 {
  margin: 0 0 14px;
  font-size: 16px;
  font-weight: 600;
  color: var(--accent);
  text-wrap: balance;
}
.card > h3 {
  margin: 20px 0 10px;
  font-size: 14px;
  font-weight: 600;
  color: var(--accent);
}
/* ---------- 表格 ---------- */
table {
  width: 100%;
  border-collapse: collapse;
  margin: 10px 0 16px;
  font-variant-numeric: tabular-nums;
}
caption {
  text-align: left;
  color: var(--ink-soft);
  font-size: 12px;
  padding-bottom: 6px;
}
th, td {
  border: 1px solid var(--line);
  padding: 8px 12px;
  text-align: center;
  vertical-align: middle;
}
```

```
th {
  background: var(--accent-soft);
  font-weight: 600;
  white-space: nowrap;
}
tbody tr:hover { background: #fafbfc; }
/* ---------- 表单控件 ---------- */
label { display: inline-block; }
.field { margin-bottom: 12px; }
.field__label {
  display: block;
  margin-bottom: 4px;
  color: var(--ink-soft);
  font-size: 13px;
}
input[type="text"], input[type="number"],
input[type="password"],
select, textarea {
  padding: 6px 9px;
  font-family: inherit;
  font-size: 14px;
  color: var(--ink);
  background: #fff;
  border: 1px solid #c6ccd4;
  border-radius: 4px;
  max-width: 100%;
}
input[type="number"] {
  width: 104px;
  text-align: center;
  font-variant-numeric: tabular-nums;
}
textarea { width: 100%; min-height: 64px; resize:
vertical; }
input:focus-visible, select:focus-visible,
textarea:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 1px;
  border-color: var(--accent);
}
.checkbox {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  cursor: pointer;
  user-select: none;
}
.checkbox input { width: 16px; height: 16px; margin: 0;
cursor: pointer; }
/* ---------- 按钮 ---------- */
```

```
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 8px 18px;
  font-family: inherit;
  font-size: 14px;
  line-height: 1.4;
  color: #fff;
  background: var(--accent);
  border: 1px solid transparent;
  border-radius: 4px;
  cursor: pointer;
  text-decoration: none;
}
.btn:hover { background: var(--accent-hover); }
.btn:focus-visible { outline: 2px solid var(--accent);
outline-offset: 2px; }
.btn--ghost {
  color: var(--ink);
  background: #fff;
  border-color: #c6ccd4;
}
.btn--ghost:hover { background: #f1f3f6; }
.btn--danger {
  color: var(--danger);
  background: #fff;
  border-color: #e6b4b0;
}
.btn--danger:hover { background: var(--danger-soft); }
.btn--sm { padding: 4px 12px; font-size: 13px; }
.btn:disabled { opacity: .5; cursor: not-allowed; }
.actions { display: flex; flex-wrap: wrap; align-items:
center; gap: 8px; }
.actions form { display: inline; }
/* ---------- 提示条 ---------- */
.flash {
  padding: 10px 14px;
  margin-bottom: 14px;
  border-radius: var(--radius);
  border-left: 4px solid;
}
.flash.error {
  color: var(--danger);
  background: var(--danger-soft);
  border-left-color: var(--danger);
}
.flash.ok {
  color: var(--ok);
```

```
    background: var(--ok-soft);                                          4001
    border-left-color: var(--ok);                                        4002
}                                                                        4003
/* ---------- 居中浮层提示（登录成功、各类操作完成） ----------          4004
   外层 toast-stack 固定在页面正中心并纵向排列，                         4005
   内层每条各自淡入 → 停留 → 淡出，多条提示会依次排开而不重叠。          4006
   半透明底 + 背景模糊做出毛玻璃质感，不做成实心块；                     4007
   pointer-events: none 让它不拦截鼠标，出现期间也能正常操作。 */       4008
.toast-stack {                                                           4009
    position: fixed;                                                     4010
    top: 50%;                                                            4011
    left: 50%;                                                           4012
    z-index: 100;                                                        4013
    display: flex;                                                       4014
    flex-direction: column;                                              4015
    align-items: center;                                                 4016
    gap: 10px;                                                           4017
    transform: translate(-50%, -50%);                                    4018
    pointer-events: none;                                                4019
}                                                                        4020
.toast-center {                                                          4021
    max-width: min(560px, 78vw);                                         4022
    padding: 13px 28px;                                                  4023
    border-radius: 999px;                                                4024
    background: rgba(31, 58, 95, .62);                /*                 4025
半透明，非实体 */                                                        4026
    color: #fff;                                                         4027
    font-size: 16px;                                                     4028
    font-weight: 600;                                                    4029
    line-height: 1.65;                                                   4030
    letter-spacing: .03em;                                               4031
    text-align: center;                                                  4032
    text-wrap: pretty;                                                   4033
    border: 1px solid rgba(255, 255, 255, .28);                          4034
    backdrop-filter: blur(9px) saturate(1.3);                            4035
    -webkit-backdrop-filter: blur(9px) saturate(1.3);                    4036
    box-shadow: 0 10px 34px rgba(19, 36, 58, .18);                       4037
    animation: toast-fade 2.6s ease-in-out forwards;                     4038
}                                                                        4039
@keyframes toast-fade {                                                  4040
    0%          { opacity: 0; transform: scale(.90); }                   4041
    14%, 74%    { opacity: 1; transform: scale(1); }                     4042
    100%        { opacity: 0; transform: scale(1); }                     4043
}                                                                        4044
/* 系统设置里要求「减少动态效果」时，去掉缩放，只做淡入淡出 */          4045
@media (prefers-reduced-motion: reduce) {                                4046
    .toast-center { animation: toast-fade-plain 2.6s                     4047
linear forwards; }                                                       4048
    @keyframes toast-fade-plain {                                        4049
        0%          { opacity: 0; }                                      4050
```

```
      14%, 74% { opacity: 1; }
      100%     { opacity: 0; }
    }
}
/* 不支持背景模糊时，加深底色保证文字仍然清楚 */
@supports not (backdrop-filter: blur(1px)) {
  .toast-center { background: rgba(31, 58, 95, .88); }
}
/* ---------- 文字工具类 ---------- */
.muted { color: var(--ink-soft); font-size: 12.5px; }
.mono { font-family: Consolas, "Courier New",
monospace; font-variant-numeric: tabular-nums; }
.num { font-variant-numeric: tabular-nums; }
.warn { color: var(--danger); font-weight: 600; }
.ok { color: var(--ok); font-weight: 600; }
.nowrap { white-space: nowrap; }
/* 表格操作列里常把删除按钮放在 <form> 中。form 默认是块级元素，
   会把「删除」挤到下一行，导致操作列参差不齐，这里让它留在行内。 */
.nowrap form { display: inline; margin: 0; }
.stack { display: flex; flex-direction: column; gap:
14px; }
/* 状态标签：不依赖颜色单独表意 */
.tag {
  display: inline-block;
  padding: 1px 9px;
  font-size: 12.5px;
  border-radius: 10px;
  border: 1px solid;
  white-space: nowrap;
}
.tag--ok { color: var(--ok); background:
var(--ok-soft); border-color: #b8ddc6; }
.tag--warn { color: var(--danger); background:
var(--danger-soft); border-color: #eec4c0; }
.tag--muted { color: var(--ink-soft); background:
#f1f3f6; border-color: #dde1e6; }
/* ---------- 登录页 ---------- */
/* 背景：淡雅远山（SVG）+ 渐变兜底；卡片垂直水平都居中 */
.body--login {
  background:
    url("{{ url_for('static', filename='login-bg.svg')
}}")
    center / cover no-repeat fixed,
    linear-gradient(180deg, #f6f9fc 0%, #e6eef5 100%);
}
.body--login .app-main {
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 28px 20px;
```

```
}
.body--login .container { max-width: none; width: 100%;
}
.login-wrap {
  display: flex;
  justify-content: center;
  width: 100%;
}
.login-card {
  width: 100%;
  max-width: 400px;
  padding: 30px 32px 26px;
  border-radius: 10px;
  box-shadow: 0 14px 44px rgba(19, 36, 58, .13);
}
.login-card h2 {
  margin: 0 0 6px;
  text-align: center;
  font-size: 20px;
}
.login-card__sub {
  margin: 0 0 22px;
  text-align: center;
  font-size: 12.5px;
  color: var(--ink-soft);
}
.login-card .field__label { font-size: 14px; color:
var(--ink); }
/* ---------- 首页功能块 ---------- */
.tile-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill,
minmax(232px, 1fr));
  gap: 12px;
}
.tile {
  display: block;
  padding: 16px;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  color: var(--ink);
  text-decoration: none;
}
.tile:hover { border-color: var(--accent); background:
#fbfcfe; }
.tile:focus-visible { outline: 2px solid var(--accent);
outline-offset: 2px; }
.tile__title { font-weight: 600; color: var(--accent);
margin-bottom: 5px; }
```

```
.tile__desc { font-size: 12.5px; color: var(--ink-soft); text-wrap: pretty; }
/* ---------- 结果摘要 ---------- */
.stat-row {
  display: flex;
  flex-wrap: wrap;
  gap: 28px;
  margin: 4px 0 6px;
}
.stat__label { font-size: 12.5px; color: var(--ink-soft); }
.stat__value { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }
/* ---------- 流程步骤条 ---------- */
.steps { display: flex; flex-wrap: wrap; gap: 8px; margin: 4px 0 18px; }
.step {
  display: flex; align-items: center; gap: 7px;
  padding: 8px 12px; font-size: 13px;
  background: #fff; border: 1px solid var(--line);
  border-radius: var(--radius);
}
.step__no {
  display: inline-flex; align-items: center; justify-content: center;
  width: 20px; height: 20px; border-radius: 50%;
  background: #eef1f5; color: var(--ink-soft);
  font-size: 12px; font-variant-numeric: tabular-nums;
}
.step__name { font-weight: 600; }
.step__state { font-size: 12.5px; color: var(--ink-soft); }
.step--done { border-color: #b8ddc6; background: var(--ok-soft); }
.step--done .step__no { background: var(--ok); color: #fff; }
.step--done .step__state { color: var(--ok); }
.step--fail { border-color: #eec4c0; background: var(--danger-soft); }
.step--fail .step__no { background: var(--danger); color: #fff; }
.step--fail .step__state { color: var(--danger); font-weight: 600; }
.step--next { border-color: var(--accent); }
.step--next .step__no { background: var(--accent); color: #fff; }
/* ---------- 提示块 ---------- */
.notice {
  padding: 14px 16px; margin: 0 0 16px;
  border-radius: var(--radius); border-left: 4px solid;
```

```
}
.notice--warn {
  background: var(--danger-soft);
  border-left-color: var(--danger);
  color: #7a1c17;
}
.notice__title { margin: 0 0 8px; font-weight: 600; }
.notice__list { margin: 0; padding-left: 20px;
line-height: 1.9; }
.notice__list a { color: var(--accent); }
.notice__hint { margin: 8px 0 0; font-size: 12.5px;
color: var(--ink-soft); }
/* ---------- 图斑面板（按图斑分块展示，如指标数据采集） ---------- */
.patch-panel {
  border: 1px solid var(--line);
  border-radius: var(--radius);
  background: var(--surface);
  padding: 14px 16px;
  margin: 0 0 14px;
}
.patch-panel:last-child { margin-bottom: 0; }
.patch-panel__head {
  display: flex;
  align-items: baseline;
  flex-wrap: wrap;
  gap: 6px 14px;
  margin: 0 0 12px;
  padding-bottom: 9px;
  border-bottom: 1px solid var(--line);
}
.patch-panel__name {
  margin: 0;
  font-size: 15px;
  font-weight: 600;
}
.patch-panel__meta {
  font-size: 12.5px;
  color: var(--ink-soft);
}
.patch-panel__meta a { color: var(--accent); }
.patch-panel__tail { margin-left: auto; }
.patch-panel .notice { margin: 0; }
.patch-panel table { margin: 0; }
/* ---------- 窄屏适配 ---------- */
@media (max-width: 860px) {
  .app-body { flex-direction: column; }
  .app-nav {
    flex: none;
    width: auto;
    border-right: none;
```

```
            border-bottom: 1px solid var(--line);                    4251
            padding: 8px 10px;                                       4252
          }                                                          4253
          .app-nav ul { display: flex; flex-wrap: wrap; gap:         4254
        4px; }                                                       4255
          .app-nav a {                                               4256
            margin-bottom: 0;                                        4257
            border-left: none;                                       4258
            border-bottom: 3px solid transparent;                    4259
          }                                                          4260
          .app-nav a[aria-current="page"] { border-left: none;       4261
        border-bottom-color: var(--accent); }                        4262
          .app-nav__group { display: none; }                         4263
          .app-main { padding: 16px 14px 32px; }                     4264
        }                                                            4265
    </style>                                                         4266
</head>                                                              4267
<body class="{% if not current_user.is_authenticated               4268
      %}body--login{% endif %}">                                     4269
    <a class="skip-link" href="#main">跳到主要内容</a>                4270
    {# 取一次即可（同一请求内重复调用返回相同内容）。                  4271
       类别为 toast 的渲染成居中浮层，其余仍是页面顶部的提示条 ——      4272
       错误提示需要停下来细读，不适合做成会消失的浮层。 #}             4273
    {% set flash_msgs = get_flashed_messages(with_categories=t       4274
       rue) %}                                                       4275
    {# 类别为 toast / ok 的渲染成页面正中的浮层（操作完成的即时反馈）；  4276
       其余（尤其 error）仍是页面顶部的提示条 ——                     4277
       错误提示需要停下来细读，不适合做成会消失的浮层。               4278
       没有浮层时这个容器是空的，不占任何位置。 #}                    4279
    <div class="toast-stack">                                        4280
      {% for category, message in flash_msgs if category in          4281
      ("toast", "ok") %}                                             4282
      <div class="toast-center" role="status"                        4283
      aria-live="polite">{{ message }}</div>                         4284
      {% endfor %}                                                   4285
    </div>                                                           4286
    <header class="app-header">                                      4287
      <span class="app-header__title">历史遗留废弃矿山自然恢复适宜性评价系统</spa4288
      n>                                                             4289
      <span class="app-header__version">V1.0</span>                  4290
      <span class="app-header__spacer"></span>                       4291
      {% if current_user.is_authenticated %}                         4292
      <span class="app-header__user">                                4293
        <span>{{ current_user.username }} ({{                        4294
      current_user.role }})</span>                                   4295
        <a href="/logout">退出</a>                                   4296
      </span>                                                        4297
      {% endif %}                                                    4298
    </header>                                                        4299
    <div class="app-body">                                           4300
```

```
    {% if current_user.is_authenticated %}
    <nav class="app-nav" aria-label="功能导航">
      <ul>
        <li>
          <a href="/"{% if home_active %}
    aria-current="page"{% endif %}>首页</a>
        </li>
      </ul>
      {% for g in nav_groups %}
      <p class="app-nav__group">{{ g.title }}</p>
      <ul>
        {% for label, href, active in g.links %}
        <li>
          <a href="{{ href }}"{% if active %}
    aria-current="page"{% endif %}>{{ label }}</a>
        </li>
        {% endfor %}
      </ul>
      {% endfor %}
    </nav>
    {% endif %}
    <main id="main" class="app-main">
      <div class="container">
        {% for category, message in flash_msgs if category
    not in ("toast", "ok") %}
        <p class="flash {{ category }}" role="status">{{
    message }}</p>
        {% endfor %}
        {% block content %}{% endblock %}
      </div>
    </main>
  </div>
</body>
</html>
#  ==== app/templates/_macros.html ====
{#  共用宏：流程状态面板  #}
{#  评价流程步骤条：按论文  4．1  节顺序显示各前置步骤的完成情况
    顺序为：限制性条件判定  →  划分微地质单元  →  录入指标数据  →  执行评价。
    论文原文：「先进行政策、位置约束地质灾害判断……对于满足限制性条件的
    废弃矿山，按照地形地貌划分……三种地质微单元类型」，故判定在前。  #}
{% macro workflow_steps(progress) %}
<div class="steps">
  <div class="step {% if progress.constraint.passed
    %}step--done{% elif progress.constraint.done
    %}step--fail{% else %}step--todo{% endif %}">
    <span class="step__no">1</span>
    <span class="step__name">限制性条件判定</span>
    <span class="step__state">
      {% if progress.constraint.passed %}已通过
      {% elif progress.constraint.done %}未通过
```

```
        {% else %}待完成{% endif %}
      </span>
    </div>
    <div class="step {% if progress.units.done %}step--done{%
      else %}step--todo{% endif %}">
      <span class="step__no">2</span>
      <span class="step__name">划分微地质单元</span>
      <span class="step__state">
        {% if progress.units.done %}已完成{% else %}待完成{% endif
      %}
      </span>
    </div>
    <div class="step {% if progress.data.ready %}step--done{%
      else %}step--todo{% endif %}">
      <span class="step__no">3</span>
      <span class="step__name">录入指标数据</span>
      <span class="step__state">
        {% if progress.data.ready %}已完成{% else %}待完成{% endif
      %}
      </span>
    </div>
    <div class="step step--next">
      <span class="step__no">4</span>
      <span class="step__name">执行适宜性评价</span>
      <span class="step__state">当前步骤</span>
    </div>
  </div>
{% endmacro %}
{# 前置条件未满足时的提示块，逐条给出原因与去处理入口 #}
{% macro workflow_blocking(progress) %}
{% if progress.blocking %}
<div class="notice notice--warn">
  <p class="notice__title">尚不能执行本步骤，请先完成以下前置工作：</p>
  <ol class="notice__list">
    {% for b in progress.blocking %}
    <li>
      <strong>{{ b.label }}</strong> —— {{ b.reason }}
      <a href="{{ b.fix_url }}">{{ b.fix_label }}</a>
    </li>
    {% endfor %}
  </ol>
  <p class="notice__hint">
    说明：本系统按论文 4.1 节的评价流程设置前置检查，
    避免在前置条件不具备时产出不合理的评价结论。
  </p>
</div>
{% endif %}
{% endmacro %}
{# 一行式状态摘要，用于列表页 #}
{% macro workflow_badge(progress) %}
```

```
{% if progress.ready %}                                                    4401
<span class="tag tag--ok">可执行评价</span>                                  4402
{% elif progress.blocking %}                                               4403
<span class="tag tag--warn">{{ progress.blocking[0].label                  4404
    }}</span>                                                              4405
<span class="muted">{{ progress.blocking[0].reason }}</span>               4406
{% endif %}                                                                4407
{% endmacro %}                                                             4408
{# 层级引导：本系统是「项目 → 图斑」两级结构，评价针对的是图斑而非项目。                  4409
   下游各页按图斑组织数据，故只要存在「还没建图斑的项目」，就提示下一步该做什么。 #}      4410
{% macro patch_guide(projects_without_patch) %}                            4411
{% if projects_without_patch %}                                            4412
<div class="notice notice--warn">                                          4413
  <p class="notice__title">有 {{ projects_without_patch|lengt              4414
    h }} 个项目下还没有图斑，本页不会出现它们。</p>                                4415
  <p>                                                                      4416
    本系统是「项目 → 矿山图斑」两级结构：项目是归档单位，而                          4417
    <strong>限制性条件判定、微地质单元划分、指标数据与适宜性评价都是针对图斑进行的</strong>。  4418
  </p>                                                                     4419
  <ul class="notice__list">                                                4420
    {% for p in projects_without_patch %}                                  4421
    <li>                                                                   4422
      {{ p.name }}                                                         4423
      —— <a href="/project/{{ p.id }}/patch/add">为它新建图斑</a>           4424
    </li>                                                                  4425
    {% endfor %}                                                           4426
  </ul>                                                                    4427
  <p class="notice__hint">                                                 4428
    也可以先进入 <a href="/project/">项目与矿山档案</a>，                        4429
    点该项目右侧的「＋ 新建图斑」，建好之后本页就会出现该图斑。                       4430
  </p>                                                                     4431
</div>                                                                     4432
{% endif %}                                                                4433
{% endmacro %}                                                             4434
# ==== app/templates/error.html ====                                       4435
{% extends "base.html" %}                                                  4436
{% block title %}{{ code }} · {{ message }}{% endblock %}                  4437
{% block content %}                                                        4438
<div class="card" style="max-width:560px;">                                4439
  <h2>{{ code }} · {{ message }}</h2>                                      4440
  <p class="muted">                                                        4441
    请返回上一页重试；若确认该操作应当可用，请检查当前账号的角色权限。                    4442
  </p>                                                                     4443
  <p class="actions" style="margin-top:16px;">                             4444
    <a class="btn" href="/">返回首页</a>                                     4445
  </p>                                                                     4446
</div>                                                                     4447
{% endblock %}                                                             4448
# ==== app/templates/auth/login.html ====                                  4449
{% extends "base.html" %}                                                  4450
```

```
{% block title %}登录 · 历史遗留废弃矿山自然恢复适宜性评价系统{% endblock %}
{% block content %}
<div class="login-wrap">
  <div class="card login-card">
    <h2>用户登录</h2>
    <p class="login-card__sub">历史遗留废弃矿山自然恢复适宜性评价系统 V1.0</p>
    <form method="post" novalidate>
      {{ form.hidden_tag() }}
      <div class="field">
        <label class="field__label" for="{{ form.username.id }}">用户名</label>
        {{ form.username(id=form.username.id, autocomplete="username") }}
      </div>
      <div class="field">
        <label class="field__label" for="{{ form.password.id }}">密码</label>
        {{ form.password(id=form.password.id, autocomplete="current-password") }}
      </div>
      <div class="field">
        <label class="checkbox" for="{{ form.remember.id }}">
          {{ form.remember(id=form.remember.id) }}
          <span>记住我（下次自动登录，一个月内有效）</span>
        </label>
      </div>
      <p style="margin:18px 0 0;">{{ form.submit(class_="btn", style="width:100%;") }}</p>
    </form>
    <p class="muted" style="margin:14px 0 0;">演示账号：admin / admin123</p>
  </div>
</div>
{% endblock %}
# ==== app/templates/auth/home.html ====
{% extends "base.html" %}
{% block title %}首页{% endblock %}
{% block content %}
<div class="card">
  <h2>系统功能</h2>
  <p class="muted">
    本系统以论文《历史遗留废弃矿山自然恢复适宜性评价指标体系建设及典型案例》
    建立的评价方法为算法内核。下方按论文 4.1 节的评价流程顺序排列，建议依序使用；
    系统会在每一步检查前置条件，前置步骤未完成时不会放行后续计算。
  </p>
</div>
<div class="card">
  <h2>评价流程</h2>
```

```
    <div class="tile-grid">
      <a class="tile" href="/project/">
        <div class="tile__title">项目与矿山档案</div>
        <div class="tile__desc">建立评价项目、录入矿山图斑、挂接遥感影像、设置参照生态系统</div>
      </a>
      <a class="tile" href="/standard/">
        <div class="tile__title">政策与标准知识库</div>
        <div class="tile__desc">收录 １１ 项地方标准与政策文件，结构化判定规则，支持省际口径比对</div>
      </a>
      <a class="tile" href="/constraint/">
        <div class="tile__title">限制性指标判定</div>
        <div class="tile__desc">政策、区位约束、土壤污染、地质灾害、耕地属性五项一票否决</div>
      </a>
      <a class="tile" href="/unit/">
        <div class="tile__title">微地质单元</div>
        <div class="tile__desc">划分岩质边坡、碎石边坡、平台三类单元，自动计算面积占比</div>
      </a>
      <a class="tile" href="/ahp/">
        <div class="tile__title">指标体系与权重</div>
        <div class="tile__desc">指标定义维护、判断矩阵录入、权重计算与一致性检验、多专家汇总</div>
      </a>
      <a class="tile" href="/indicator/">
        <div class="tile__title">指标数据采集</div>
        <div class="tile__desc">按微地质单元录入土壤、植被、地形指标，支持 ＣＳＶ / Ｅｘｃｅｌ 批量导入</div>
      </a>
      <a class="tile" href="/evaluate/">
        <div class="tile__title">适宜性定量评价</div>
        <div class="tile__desc">指标标准化、分单元赋分、面积加权合成、适宜性等级判定</div>
      </a>
      <a class="tile" href="/report/">
        <div class="tile__title">报告与结果输出</div>
        <div class="tile__desc">结果图表、逐层得分明细，导出 ＰＤＦ 与 Ｗｏｒｄ 评价报告</div>
      </a>
    </div>
  </div>
  <div class="card">
    <h2>系统管理</h2>
    <div class="tile-grid">
      <a class="tile" href="/users">
        <div class="tile__title">用户与操作日志</div>
        <div class="tile__desc">用户维护与角色权限；全部写操作自动留痕，可查询追溯</div>
      >
```

```
      </a>
    </div>
</div>
{% endblock %}
# ==== app/templates/auth/users.html ====
{% extends "base.html" %}
{% block title %}用户与权限管理{% endblock %}
{% block content %}
<div class="card">
    <h2>用户管理</h2>
    <p class="muted">
      系统设有管理员、评价员、查看者三种角色：管理员可维护用户与基础数据，
      评价员可建项目、录数据、执行评价，查看者仅可查阅。
    </p>
    <p class="actions" style="margin-bottom:12px;">
      {% if current_user.role == "admin" %}
      <a class="btn btn--sm" href="/users/add">新增用户</a>
      {% endif %}
      <a class="btn btn--sm btn--ghost"
      href="/logs">查看操作日志</a>
    </p>
    {% if current_user.role != "admin" %}
    <p class="muted">
      当前角色为「{{ role_labels.get(current_user.role,
      current_user.role) }}」，
      仅管理员可新增、编辑或删除用户。
    </p>
    {% endif %}
    <table>
      <caption class="muted">共 {{ users|length }}
      个用户</caption>
      <thead>
        <tr>
          <th scope="col">ID</th>
          <th scope="col">用户名</th>
          <th scope="col">角色</th>
          <th scope="col">状态</th>
          {% if current_user.role == "admin" %}<th
      scope="col">操作</th>{% endif %}
        </tr>
      </thead>
      <tbody>
        {% for u in users %}
        <tr>
          <td class="num">{{ u.id }}</td>
          <td>
            {{ u.username }}
            {% if u.id == current_user.id %}<span
      class="muted">（当前登录）</span>{% endif %}
          </td>
```

```
            <td>{{ role_labels.get(u.role, u.role) }}</td>
            <td>
              {% if u.active %}<span class="tag
      tag--ok">启用</span>
              {% else %}<span class="tag tag--muted">停用</span>{%
      endif %}
            </td>
            {% if current_user.role == "admin" %}
            <td class="nowrap">
              <a href="/users/{{ u.id }}/edit">编辑</a>
               | 
              <form method="post" action="/users/{{ u.id
      }}/delete"
                    onsubmit="return confirm('确定删除用户「{{
      u.username }}」吗？\n该操作无法恢复。');">
                {{ form.hidden_tag() }}
                <button type="submit" class="btn btn--sm
      btn--danger"
                        {% if u.id == current_user.id %}disabled
                        title="不能删除当前登录的账号"{% endif
      %}>删除</button>
              </form>
            </td>
            {% endif %}
          </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
    {% endblock %}
    # ==== app/templates/auth/add_user.html ====
    {% extends "base.html" %}
    {% block title %}新增用户{% endblock %}
    {% block content %}
    <div class="card" style="max-width:560px;">
      <h2>新增用户</h2>
      <form method="post">
        {{ form.hidden_tag() }}
        <div class="field">
          <label class="field__label" for="username">用户名</label>
          <input id="username" type="text" name="username"
        required
                 autocomplete="off" style="width:280px;">
        </div>
        <div class="field">
          <label class="field__label" for="password">密码</label>
          <input id="password" type="password" name="password"
        required
                 autocomplete="new-password"
        style="width:280px;">
```

```
      </div>                                                                    4651
      <div class="field">                                                       4652
        <label class="field__label" for="role">角色</label>                     4653
        <select id="role" name="role">                                          4654
          <option value="viewer">查看者（仅可查阅）</option>                    4655
          <option value="evaluator">评价员（可建项目、录数据、执行评价）</option4656
      >                                                                         4657
          <option value="admin">管理员（可维护用户与基础数据）</option>         4658
        </select>                                                               4659
      </div>                                                                    4660
      <p class="actions" style="margin-top:18px;">                              4661
        <button type="submit" class="btn">保存</button>                        4662
        <a class="btn btn--ghost" href="/users">返回</a>                        4663
      </p>                                                                      4664
    </form>                                                                     4665
</div>                                                                          4666
{% endblock %}                                                                  4667
# ==== app/templates/auth/edit_user.html ====                                   4668
{% extends "base.html" %}                                                       4669
{% block title %}编辑用户 · {{ user.username }}{% endblock %}                   4670
{% block content %}                                                             4671
<div class="card" style="max-width:560px;">                                     4672
  <h2>编辑用户 · {{ user.username }}</h2>                                       4673
  <form method="post">                                                          4674
    {{ form.hidden_tag() }}                                                     4675
    <div class="field">                                                         4676
      <label class="field__label" for="role">角色</label>                       4677
      <select id="role" name="role">                                            4678
        {% for code, label in role_labels.items() %}                            4679
        <option value="{{ code }}" {% if user.role == code                      4680
    %}selected{% endif %}>{{ label }}</option>                                  4681
        {% endfor %}                                                            4682
      </select>                                                                 4683
    </div>                                                                      4684
    <div class="field">                                                         4685
      <label class="checkbox" for="active">                                     4686
        <input id="active" type="checkbox" name="active"                        4687
    value="1"                                                                   4688
               {% if user.active %}checked{% endif %}>                          4689
        <span>启用该账号</span>                                                 4690
      </label>                                                                  4691
      <p class="muted" style="margin:6px 0 0;">                                 4692
        停用后该账号无法登录，但其历史操作记录会保留。                          4693
        系统至少需保留一名启用状态的管理员。                                    4694
      </p>                                                                      4695
    </div>                                                                      4696
    <div class="field">                                                         4697
      <label class="field__label" for="password">重置密码（留空则不修改4698
    ）</label>                                                                  4699
      <input id="password" type="password" name="password"                      4700
```

```
                autocomplete="new-password"                                    4701
      style="width:280px;"                                                     4702
                placeholder="留空表示保持原密码">                                4703
      </div>                                                                   4704
      <p class="actions" style="margin-top:18px;">                             4705
        <button type="submit" class="btn">保存</button>                        4706
        <a class="btn btn--ghost" href="/users">返回</a>                       4707
      </p>                                                                     4708
    </form>                                                                    4709
</div>                                                                         4710
{% endblock %}                                                                 4711
# ==== app/templates/auth/logs.html ====                                       4712
{% extends "base.html" %}                                                      4713
{% block title %}操作日志{% endblock %}                                         4714
{% block content %}                                                            4715
<div class="card">                                                             4716
  <h2>操作日志</h2>                                                            4717
  <p class="muted">                                                            4718
    系统自动记录所有写操作（新建、编辑、删除、导入、执行评价等），                4719
    便于追溯数据变更。失败的操作也会记录并标注 HTTP 状态码。                     4720
  </p>                                                                         4721
  <p class="actions" style="margin-bottom:12px;">                              4722
    <a class="btn btn--sm btn--ghost"                                          4723
    href="/users">返回用户管理</a>                                              4724
  </p>                                                                         4725
  {% if not records %}                                                         4726
  <p class="muted">暂无操作记录。</p>                                           4727
  {% else %}                                                                   4728
  <table>                                                                      4729
    <caption class="muted">共 {{ total }} 条记录，当前第 {{ page }}            4730
    / {{ pages }} 页</caption>                                                 4731
    <thead>                                                                    4732
      <tr>                                                                     4733
        <th scope="col">时间</th>                                              4734
        <th scope="col">操作人</th>                                            4735
        <th scope="col">操作</th>                                              4736
        <th scope="col">对象</th>                                              4737
        <th scope="col">说明</th>                                              4738
      </tr>                                                                    4739
    </thead>                                                                   4740
    <tbody>                                                                    4741
      {% for r in records %}                                                   4742
      <tr>                                                                     4743
        <td class="mono nowrap">{{ r.created_at.strftime("%Y                   4744
    -%m-%d %H:%M:%S") if r.created_at else "—" }}</td>                         4745
        <td>{{ users_map.get(r.user_id, "—") }}</td>                           4746
        <td style="text-align:left;">{{ r.action }}</td>                       4747
        <td>                                                                   4748
          {% if r.target_type %}{{ r.target_type }}{% if                       4749
    r.target_id %}#{{ r.target_id }}{% endif %}                                4750
```

```
              {% else %}—{% endif %}                                    4751
            </td>                                                       4752
            <td style="text-align:left;">{{ r.detail or "—"           4753
      }}</td>                                                           4754
        </tr>                                                           4755
        {% endfor %}                                                    4756
      </tbody>                                                          4757
   </table>                                                             4758
   {% if pages > 1 %}                                                   4759
   <nav class="actions" aria-label="日志分页">                          4760
      {% if page > 1 %}                                                 4761
      <a class="btn btn--sm btn--ghost" href="/logs?page={{             4762
      page - 1 }}">上一页</a>                                           4763
      {% endif %}                                                       4764
      <span class="muted">第 {{ page }} / {{ pages }} 页</span>         4765
      {% if page < pages %}                                             4766
      <a class="btn btn--sm btn--ghost" href="/logs?page={{             4767
      page + 1 }}">下一页</a>                                           4768
      {% endif %}                                                       4769
   </nav>                                                               4770
   {% endif %}                                                          4771
   {% endif %}                                                          4772
</div>                                                                  4773
{% endblock %}                                                          4774
# ==== app/templates/project/index.html ====                            4775
{% extends "base.html" %}                                               4776
{% block title %}项目与矿山档案{% endblock %}                           4777
{% block content %}                                                     4778
<div class="card">                                                      4779
   <h2>项目与矿山档案</h2>                                              4780
   <p class="muted">                                                    4781
      项目是归档单位，用于组织同一矿区的多个图斑；                     4782
      <strong>限制性条件判定、微地质单元划分、指标数据与适宜性评价都是针对图斑进行的</strong>。  4783
      因此新建项目后，请继续在该项目下建立图斑，后续模块才可操作。    4784
   </p>                                                                 4785
   <p class="actions" style="margin-bottom:12px;">                      4786
      <a class="btn btn--sm" href="/project/add">新建项目</a>           4787
   </p>                                                                 4788
   {% if not projects %}                                                4789
   <p class="muted">                                                    4790
      暂无项目。点击上方「新建项目」开始建立评价项目，                 4791
      或运行 <span class="mono">python seed_demo.py</span>             4792
      载入论文案例。                                                    4793
   </p>                                                                 4794
   {% else %}                                                           4795
   <table>                                                              4796
      <caption class="muted">共 {{ projects|length }}                   4797
      个项目</caption>                                                  4798
      <thead>                                                           4799
        <tr>                                                            4800
```

```
            <th scope="col">项目名称</th>                                    4801
            <th scope="col">编号</th>                                        4802
            <th scope="col">状态</th>                                        4803
            <th scope="col">图斑数</th>                                      4804
            <th scope="col">操作</th>                                        4805
          </tr>                                                              4806
        </thead>                                                             4807
        <tbody>                                                              4808
          {% for p in projects %}                                            4809
          <tr>                                                               4810
            <td style="text-align:left;">{{ p.name }}</td>                   4811
            <td class="mono">{{ p.code or "—" }}</td>                        4812
            <td><span class="tag tag--muted">{{ p.status                     4813
        }}</span></td>                                                       4814
            <td class="num">                                                 4815
              {% if p.patches.count() %}                                     4816
                {{ p.patches.count() }}                                      4817
              {% else %}                                                     4818
                <span class="tag tag--warn">0</span>                         4819
              {% endif %}                                                    4820
            </td>                                                            4821
            <td class="nowrap">                                              4822
              <a href="/project/{{ p.id }}/patches">                         4823
                {% if p.patches.count() %}矿山图斑{% else %}＋               4824
        新建图斑{% endif %}                                                  4825
              </a>                                                           4826
               | <a href="/project/{{ p.id                         4827
        }}/edit">编辑</a>                                                    4828
               |                                                   4829
              <form method="post" action="/project/{{ p.id                   4830
        }}/delete"                                                           4831
                    onsubmit="return confirm('确定删除项目「{{ p.name        4832
        }}」吗？\n该项目下的图斑、微地质单元与评价数据将一并删除，且无法恢复。');">  4833
                {{ form.hidden_tag() }}                                      4834
                <button type="submit" class="btn btn--sm                     4835
        btn--danger">删除</button>                                           4836
              </form>                                                        4837
            </td>                                                            4838
          </tr>                                                              4839
          {% endfor %}                                                       4840
        </tbody>                                                             4841
      </table>                                                               4842
      {% endif %}                                                            4843
    </div>                                                                   4844
    {% endblock %}                                                           4845
    # ==== app/templates/project/edit.html ====                              4846
    {% extends "base.html" %}                                                4847
    {% block title %}{{ "编辑项目" if project else "新建项目" }}{%           4848
        endblock %}                                                          4849
    {% block content %}                                                      4850
```

```
<div class="card" style="max-width:640px;">
  <h2>{{ "编辑项目" if project else "新建项目" }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="name">项目名称</label>
      <input id="name" type="text" name="name"
    style="width:100%;" required
             value="{{ project.name if project else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="code">项目编号</label>
      <input id="code" type="text" name="code"
    style="width:240px;"
             value="{{ project.code or '' if project else
    '' }}">
    </div>
    {% if project %}
    <div class="field">
      <label class="field__label" for="status">状态</label>
      <select id="status" name="status">
        {% for s in ["进行中", "已完成", "已归档"] %}
        <option value="{{ s }}" {% if project.status == s
    %}selected{% endif %}>{{ s }}</option>
        {% endfor %}
      </select>
    </div>
    {% endif %}
    <div class="field">
      <label class="field__label" for="description">项目说明</la
    bel>
      <textarea id="description" name="description">{{
    project.description or '' if project else ''
    }}</textarea>
    </div>
    <p class="actions" style="margin-top:18px;">
      <button type="submit" class="btn">保存</button>
      <a class="btn btn--ghost" href="/project/">返回</a>
    </p>
  </form>
</div>
{% endblock %}
# ==== app/templates/project/patches.html ====
{% extends "base.html" %}
{% block title %}矿山图斑 · {{ project.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>矿山图斑 · {{ project.name }}</h2>
  <p class="actions" style="margin-bottom:12px;">
    <a class="btn btn--sm" href="/project/{{ project.id
```

```
    }}/patch/add">新建图斑</a>                                          4901
    <a class="btn btn--sm btn--ghost"                                   4902
    href="/project/">返回项目列表</a>                                   4903
</p>                                                                    4904
{% if not project.patches.count() %}                                    4905
<p class="muted">该项目下暂无图斑。点击上方「新建图斑」建立第一个矿山图斑。</p>   4906
{% else %}                                                              4907
<table>                                                                 4908
    <caption class="muted">共 {{ project.patches.count() }}             4909
    个图斑</caption>                                                    4910
    <thead>                                                             4911
      <tr>                                                              4912
        <th scope="col">图斑名称</th>                                   4913
        <th scope="col">行政区</th>                                     4914
        <th scope="col">面积（公顷）</th>                               4915
        <th scope="col">开采矿种</th>                                   4916
        <th scope="col">微地质单元</th>                                 4917
        <th scope="col">操作</th>                                       4918
      </tr>                                                             4919
    </thead>                                                            4920
    <tbody>                                                             4921
      {% for p in project.patches %}                                    4922
      <tr>                                                              4923
        <td style="text-align:left;">{{ p.name }}</td>                  4924
        <td>{{ p.admin_region or "—" }}</td>                            4925
        <td class="num">{{ p.area_ha if p.area_ha is not                4926
    none else "—" }}</td>                                               4927
        <td>{{ p.mineral_type or "—" }}</td>                            4928
        <td class="num">{{ p.units.count() }}</td>                       4929
        <td class="nowrap">                                             4930
          <a href="/unit/{{ p.id }}">微地质单元</a>                     4931
           | <a href="/constraint/{{ p.id                     4932
    }}">限制性判定</a>                                                 4933
           | <a href="/project/patch/{{ p.id                  4934
    }}/edit">编辑</a>                                                  4935
           |                                                  4936
          <form method="post" action="/project/patch/{{                 4937
    p.id }}/delete"                                                     4938
                onsubmit="return confirm('确定删除图斑「{{ p.name       4939
    }}」吗？\n该图斑的微地质单元、指标数据与评价结果将一并删除，且无法恢复。');">  4940
            {{ form.hidden_tag() }}                                     4941
            <button type="submit" class="btn btn--sm                    4942
    btn--danger">删除</button>                                          4943
          </form>                                                       4944
        </td>                                                           4945
      </tr>                                                             4946
      {% endfor %}                                                      4947
    </tbody>                                                            4948
</table>                                                                4949
{% endif %}                                                             4950
```

```
</div>                                                                  4951
{% endblock %}                                                          4952
# ==== app/templates/project/patch_edit.html ====                       4953
{% extends "base.html" %}                                               4954
{% block title %}{{ "编辑图斑" if patch else "新建图斑" }}{%            4955
    endblock %}                                                         4956
{% block content %}                                                     4957
<div class="card" style="max-width:720px;">                             4958
  <h2>{{ "编辑图斑" if patch else "新建图斑" }} · {{ project.name       4959
    }}</h2>                                                             4960
  <form method="post">                                                  4961
    {{ form.hidden_tag() }}                                             4962
    <div class="field">                                                 4963
      <label class="field__label" for="patch-name">图斑名称</lab4964
    el>                                                                 4965
      <input id="patch-name" type="text" name="name"                    4966
    style="width:100%;" required                                        4967
             value="{{ patch.name if patch else '' }}">                 4968
    </div>                                                              4969
    <div class="field">                                                 4970
      <label class="field__label" for="admin-region">所在行政区</ 4971
    label>                                                              4972
      <input id="admin-region" type="text"                              4973
    name="admin_region" style="width:100%;"                             4974
             value="{{ patch.admin_region or '' if patch                4975
    else '' }}">                                                        4976
    </div>                                                              4977
    <div class="field">                                                 4978
      <label class="field__label" for="area-ha">图斑面积（公顷）</la4979
    bel>                                                                4980
      <input id="area-ha" type="number" step="any"                      4981
    name="area_ha"                                                      4982
             value="{{ patch.area_ha if patch and                       4983
    patch.area_ha is not none else '' }}">                              4984
    </div>                                                              4985
    <div class="field">                                                 4986
      <label class="field__label" for="mineral-type">开采矿种</l4987
    abel>                                                               4988
      <input id="mineral-type" type="text"                              4989
    name="mineral_type" style="width:240px;"                            4990
             value="{{ patch.mineral_type or '' if patch                4991
    else '' }}">                                                        4992
    </div>                                                              4993
    <div class="field">                                                 4994
      <label class="field__label" for="mining-method">开采方式</ 4995
    label>                                                              4996
      <input id="mining-method" type="text"                             4997
    name="mining_method" style="width:240px;"                           4998
             value="{{ patch.mining_method or '' if patch               4999
    else '' }}">                                                        5000
```

```
</div>
<div class="field">
  <label class="field__label" for="stop-years">停产年限</label>
  <input id="stop-years" type="number" step="any" name="stop_years"
         value="{{ patch.stop_years if patch and patch.stop_years is not none else '' }}">
</div>
<div class="field">
  <label class="checkbox" for="responsible-missing">
    <input id="responsible-missing" type="checkbox" name="responsible_entity_missing" value="1"
           {% if not patch or patch.responsible_entity_missing %}checked{% endif %}>
    <span>责任主体缺失</span>
  </label>
</div>
<h3>参照生态系统（指标标准化基准）</h3>
<p class="muted">
  论文 4.2.2 节取值：植被覆盖度 75%、Shannon-Wiener 多样性指数 0.7。
  该基准与图斑绑定管理，可按不同参照生态系统分别设定。
</p>
<div class="field">
  <label class="field__label" for="coverage-benchmark">植被覆盖度基准（%）</label>
  <input id="coverage-benchmark" type="number" step="any" name="coverage_benchmark"
         value="{{ reference.coverage_benchmark if reference else 75 }}">
</div>
<div class="field">
  <label class="field__label" for="shannon-benchmark">Shannon-Wiener 多样性指数基准</label>
  <input id="shannon-benchmark" type="number" step="any" name="shannon_benchmark"
         value="{{ reference.shannon_benchmark if reference else 0.7 }}">
</div>
<div class="field">
  <label class="field__label" for="dominant-species">建群种</label>
  <input id="dominant-species" type="text" name="dominant_species" style="width:100%;"
         value="{{ reference.dominant_species or '' if reference else '' }}">
</div>
<p class="actions" style="margin-top:18px;">
  <button type="submit" class="btn">保存</button>
  <a class="btn btn--ghost" href="/project/{{
```

```
      project.id }}/patches">返回</a>
      </p>
    </form>
</div>
{% if patch %}
<div class="card" style="max-width:720px;">
    <h2>遥感影像挂接</h2>
    <p class="muted">
      挂接遥感底图或无人机正射影像，用于辅助微地质单元划分与植被覆盖度判读。
      支持 PNG、JPG、GIF、WebP、BMP、TIFF，单个文件不超过 32 MB。
    </p>
    {% if patch.image_path %}
    <figure style="margin:0 0 16px;">
      <img src="{{ url_for('static', filename=patch.image_path
      ) }}"
           alt="图斑「{{ patch.name }}」已挂接的遥感影像"
           style="max-width:100%;border:1px solid
      var(--line);border-radius:6px;">
      <figcaption class="muted" style="margin-top:6px;">
        {{ patch.image_note or "遥感影像" }}
        {% if patch.image_uploaded_at %}
        · 上传于 {{ patch.image_uploaded_at.strftime("%Y-%m-%d
      %H:%M") }}
        {% endif %}
      </figcaption>
    </figure>
    <form method="post" action="/project/patch/{{ patch.id
      }}/image/delete"
          onsubmit="return confirm('确定移除该遥感影像吗？');">
      {{ form.hidden_tag() }}
      <button type="submit" class="btn btn--sm
      btn--danger">移除影像</button>
    </form>
    <hr style="margin:16px 0;border:none;border-top:1px solid
      var(--line);">
    {% endif %}
    <form method="post" action="/project/patch/{{ patch.id
      }}/image"
          enctype="multipart/form-data">
      {{ form.hidden_tag() }}
      <div class="field">
        <label class="field__label" for="image-file">
          {{ "替换影像文件" if patch.image_path else "选择影像文件" }}
        </label>
        <input id="image-file" type="file" name="image"
               accept=".png,.jpg,.jpeg,.gif,.webp,.bmp,.tif,.t
      iff" required>
      </div>
      <div class="field">
        <label class="field__label" for="image-note">影像说明（如来源、
```

```
      时相）</label>                                                                5101
        <input id="image-note" type="text" name="image_note"                        5102
      style="width:100%;"                                                           5103
                placeholder="如：2025 年 6 月无人机正射影像"                          5104
                value="{{ patch.image_note or '' }}">                               5105
      </div>                                                                        5106
      <p class="actions">                                                           5107
        <button type="submit" class="btn">{{ "替换影像" if                          5108
      patch.image_path else "上传并挂接" }}</button>                                5109
      </p>                                                                          5110
    </form>                                                                         5111
</div>                                                                              5112
{% endif %}                                                                         5113
{% endblock %}                                                                      5114
# ==== app/templates/unit/list.html ====                                            5115
{% extends "base.html" %}                                                           5116
{% from "_macros.html" import patch_guide %}                                        5117
{% block title %}微地质单元{% endblock %}                                           5118
{% block content %}                                                                 5119
<div class="card">                                                                  5120
  <h2>微地质单元</h2>                                                               5121
  <p class="muted">                                                                 5122
    按论文 4.1 节将矿山图斑划分为岩质边坡、碎石边坡、平台三类微地质单元。          5123
    面积占比由面积自动计算，即论文式（9） 中决定各单元权重的系数。                5124
  </p>                                                                              5125
  {{ patch_guide(projects_without_patch) }}                                         5126
  {% if not rows %}                                                                 5127
  <p class="muted">                                                                 5128
    暂无矿山图斑。请先到「项目与矿山档案」建立图斑。                               5129
  </p>                                                                              5130
  {% else %}                                                                        5131
  <table>                                                                           5132
    <thead>                                                                         5133
      <tr>                                                                          5134
        <th scope="col">所属项目</th>                                               5135
        <th scope="col">矿山图斑</th>                                               5136
        <th scope="col">已划分单元</th>                                             5137
        <th scope="col">占比合计</th>                                               5138
        <th scope="col">操作</th>                                                   5139
      </tr>                                                                         5140
    </thead>                                                                        5141
    <tbody>                                                                         5142
      {% for r in rows %}                                                           5143
      <tr>                                                                          5144
        <td>{{ r.patch.project.name if r.patch.project else                         5145
    "—" }}</td>                                                                     5146
        <td style="text-align:left;">{{ r.patch.name }}</td>                        5147
        <td class="num">{{ r.count }} 个</td>                                       5148
        <td class="num">                                                            5149
          {% if r.count %}                                                          5150
```

```
                {{ "%.2f%%"|format(r.ratio_sum * 100) }}
                {% if r.ratio_sum < 0.999 or r.ratio_sum >
    1.001 %}
                <span class="warn">（不等于 100%）</span>
                {% endif %}
              {% else %}—{% endif %}
            </td>
            <td class="nowrap">
              <a href="/unit/{{ r.patch.id }}">划分与维护</a>
               | <a href="/indicator/">录入指标数据</a>
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
      {% endif %}
</div>
{% endblock %}
# ==== app/templates/unit/index.html ====
{% extends "base.html" %}
{% block title %}微地质单元 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>微地质单元 · {{ patch.name }}</h2>
  <p class="muted">
    按论文 4.1 节将矿山图斑划分为岩质边坡、碎石边坡、平台三类微地质单元。
    面积占比由面积自动计算，即论文式（9） 中的权重系数。
  </p>
  {% if units %}
  <table>
    <caption class="muted">
      共 {{ units|length }} 个单元；
      面积合计 <span class="num">{{ "%.4f"|format(total_area)
    }}</span>；
      占比合计 <span class="num">{{ "%.2f%%"|format(ratio_sum *
    100) }}</span>
      {% if ratio_sum and (ratio_sum < 0.999 or ratio_sum >
    1.001) %}
      <span class="warn">（占比合计不等于 100%，请检查面积录入）</span>
      {% endif %}
    </caption>
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">面积</th>
        <th scope="col">面积占比</th>
        <th scope="col">说明</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
```

```
    <tbody>
      {% for u in units %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="num">{{ u.area if u.area is not none else "—" }}</td>
        <td class="num">
          {% if u.ratio is not none %}{{ "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}
        </td>
        <td style="text-align:left;">{{ u.description or "—" }}</td>
        <td class="nowrap">
          <a href="/unit/edit/{{ u.id }}">编辑</a>
           | 
          <form method="post" action="/unit/delete/{{ u.id }}"
                onsubmit="return confirm('确定删除微地质单元「{{ u.unit_type }}」吗？\n该单元已录入的指标数据将一并删除，且无法恢复。');">
            {{ form.hidden_tag() }}
            <button type="submit" class="btn btn--sm btn--danger">删除</button>
          </form>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% else %}
  <p class="muted">尚未划分微地质单元。请在下方添加。</p>
  {% endif %}

  <h3>添加微地质单元</h3>
  <form method="post" action="/unit/{{ patch.id }}/add">
    {{ form.hidden_tag() }}
    <div class="actions">
      <label class="field__label" for="add-unit-type" style="margin:0;">类型</label>
      <select id="add-unit-type" name="unit_type">
        {% for t in unit_types %}<option value="{{ t }}">{{ t }}</option>{% endfor %}
      </select>
      <label class="field__label" for="add-unit-area" style="margin:0;">面积</label>
      <input id="add-unit-area" type="number" step="any" name="area">
      <label class="field__label" for="add-unit-desc" style="margin:0;">说明</label>
      <input id="add-unit-desc" type="text" name="description" style="width:220px;">
      <button type="submit" class="btn btn--sm">添加</button>
```

```
      </div>
    </form>
    <p class="actions" style="margin-top:20px;">
      <a class="btn btn--sm btn--ghost" href="/project/{{
      patch.project_id }}/patches">返回图斑列表</a>
      <a class="btn btn--sm btn--ghost"
      href="/indicator/">去录入指标数据</a>
    </p>
</div>
{% endblock %}
# ==== app/templates/unit/edit.html ====
{% extends "base.html" %}
{% block title %}编辑微地质单元{% endblock %}
{% block content %}
<div class="card">
    <h2>编辑微地质单元 · {{ patch.name }}</h2>
    <p class="muted">修改面积后，系统会自动重算本图斑各单元的面积占比。</p>
    <form method="post">
      {{ form.hidden_tag() }}
      <div class="field">
        <label class="field__label" for="unit-type">微地质单元类型</l
      abel>
        <select id="unit-type" name="unit_type">
          {% for t in unit_types %}
          <option value="{{ t }}" {% if unit.unit_type == t
        %}selected{% endif %}>{{ t }}</option>
          {% endfor %}
        </select>
      </div>
      <div class="field">
        <label class="field__label" for="unit-area">面积</label>
        <input id="unit-area" type="number" step="any"
      name="area"
               value="{{ unit.area if unit.area is not none
      else '' }}">
      </div>
      <div class="field">
        <label class="field__label" for="unit-desc">说明</label>
        <input id="unit-desc" type="text" name="description"
      style="width:420px;"
               value="{{ unit.description or '' }}">
      </div>
      <p class="actions" style="margin-top:16px;">
        <button type="submit" class="btn">保存</button>
        <a class="btn btn--ghost" href="/unit/{{
      unit.patch_id }}">返回</a>
      </p>
    </form>
</div>
{% endblock %}
```

```
# ==== app/templates/standard/index.html ====
{% extends "base.html" %}
{% block title %}政策与标准知识库{% endblock %}
{% block content %}
<div class="card">
  <h2>政策与标准知识库</h2>
  <p class="muted">
    收录论文表 1 梳理的地方标准与政策文件，并将其判定条件结构化为可执行规则，
    支持同一指标在不同省份的阈值横向比对。
  </p>
  <p class="actions" style="margin-bottom:14px;">
    <a class="btn btn--sm" href="/standard/add">添加标准</a>
    <a class="btn btn--sm btn--ghost"
    href="/standard/compare">省际口径比对</a>
  </p>
  <h3>地方标准</h3>
  {% if local %}
  <table>
    <thead>
      <tr>
        <th scope="col">省份</th>
        <th scope="col">名称</th>
        <th scope="col">编号</th>
        <th scope="col">规则数</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for s in local %}
      <tr>
        <td>{{ s.province }}</td>
        <td style="text-align:left;">{{ s.name }}</td>
        <td class="mono">{{ s.doc_number or "—" }}</td>
        <td class="num">{{ s.rules|length }}</td>
        <td class="nowrap">
          <a href="/standard/{{ s.id }}/rules">规则</a>
           | <a href="/standard/{{ s.id
    }}/edit">编辑</a>
           | 
          <form method="post" action="/standard/{{ s.id
    }}/delete"
                onsubmit="return confirm('确定删除「{{
    s.province }} {{ s.name }}」吗？\n该标准下的 {{ s.rules|length
    }} 条规则将一并删除，且无法恢复。');">
            {{ form.hidden_tag() }}
            <button type="submit" class="btn btn--sm
    btn--danger">删除</button>
          </form>
        </td>
      </tr>
```

```
        {% endfor %}
      </tbody>
    </table>
    {% else %}
    <p class="muted">暂无地方标准。运行 <span class="mono">python seed_demo.py</span> 可载入论文表 1 的 11 项标准与政策文件。</p>
    {% endif %}
    <h3>政策文件</h3>
    {% if policy %}
    <table>
      <thead>
        <tr>
          <th scope="col">省份</th>
          <th scope="col">名称</th>
          <th scope="col">文号</th>
          <th scope="col">规则数</th>
          <th scope="col">操作</th>
        </tr>
      </thead>
      <tbody>
        {% for s in policy %}
        <tr>
          <td>{{ s.province }}</td>
          <td style="text-align:left;">{{ s.name }}</td>
          <td class="mono">{{ s.doc_number or "—" }}</td>
          <td class="num">{{ s.rules|length }}</td>
          <td class="nowrap">
            <a href="/standard/{{ s.id }}/rules">规则</a>
             | <a href="/standard/{{ s.id
      }}/edit">编辑</a>
             | 
            <form method="post" action="/standard/{{ s.id
      }}/delete"
                  onsubmit="return confirm('确定删除「{{
      s.province }} {{ s.name }}」吗？\n该文件下的 {{ s.rules|length
      }} 条规则将一并删除，且无法恢复。');">
              {{ form.hidden_tag() }}
              <button type="submit" class="btn btn--sm
      btn--danger">删除</button>
            </form>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% else %}
    <p class="muted">暂无政策文件。</p>
    {% endif %}
</div>
{% endblock %}
```

```
# ==== app/templates/standard/edit.html ====
{% extends "base.html" %}
{% block title %}{{ "编辑标准" if standard else "添加标准" }}{% endblock %}
{% block content %}
<div class="card" style="max-width:800px;">
  <h2>{{ "编辑标准" if standard else "添加标准" }}</h2>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="province">省份</label>
      <input id="province" type="text" name="province"
    style="width:160px;" required
             value="{{ standard.province if standard else ''
    }}">
    </div>
    <div class="field">
      <label class="field__label" for="std-name">名称</label>
      <input id="std-name" type="text" name="name"
    style="width:100%;" required
             value="{{ standard.name if standard else ''
    }}">
    </div>
    <div class="field">
      <label class="field__label" for="doc-number">标准编号 /
    文号</label>
      <input id="doc-number" type="text" name="doc_number"
    style="width:320px;"
             value="{{ standard.doc_number or '' if
    standard else '' }}">
    </div>
    <div class="field">
      <label class="field__label" for="category">类别</label>
      <select id="category" name="category">
        {% for c in ["地方标准", "政策文件"] %}
        <option value="{{ c }}" {% if standard and
    standard.category == c %}selected{% endif %}>{{ c
    }}</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="publish-date">发布时间</l
    abel>
      <input id="publish-date" type="text"
    name="publish_date" style="width:200px;"
             placeholder="如 2023-03-30"
             value="{{ standard.publish_date or '' if
    standard else '' }}">
    </div>
```

```
      <div class="field">
        <label class="field__label" for="implement-date">实施时间<
      /label>
        <input id="implement-date" type="text"
      name="implement_date" style="width:200px;"
               placeholder="如 2023-04-30（未明确可留空）"
               value="{{ standard.implement_date or '' if
      standard else '' }}">
      </div>
      <div class="field">
        <label class="field__label" for="summary">特点 /
      判定标准</label>
        <textarea id="summary" name="summary"
      style="min-height:96px;">{{ standard.summary or '' if
      standard else '' }}</textarea>
      </div>
      <p class="actions" style="margin-top:18px;">
        <button type="submit" class="btn">保存</button>
        {% if standard %}
        <form method="post" action="/standard/{{ standard.id
      }}/delete" style="display:inline;"
              onsubmit="return confirm('确定删除「{{
      standard.province }} {{ standard.name
      }}」吗？\n该标准下的规则将一并删除，且无法恢复。');">
          {{ form.hidden_tag() }}
          <button type="submit" class="btn
      btn--danger">删除该标准</button>
        </form>
        {% endif %}
        <a class="btn btn--ghost" href="/standard/">返回</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/standard/rules.html ====
{% extends "base.html" %}
{% block title %}结构化规则 · {{ standard.province }}{% endblock
  %}
{% block content %}
<div class="card">
  <h2>结构化规则 · {{ standard.province }} {{ standard.name
    }}</h2>
  <p class="muted">
    将标准条文拆解为「判定项 + 比较符 + 阈值 + 条件类型」的可执行规则，
    使同一指标在各省的阈值差异可自动比对。
  </p>
  {% if rules %}
  <table>
    <caption class="muted">共 {{ rules|length }}
    条规则</caption>
```

```
    <thead>
      <tr>
        <th scope="col">判定项</th>
        <th scope="col">条件</th>
        <th scope="col">条件类型</th>
        <th scope="col">适用分区</th>
        <th scope="col">备注</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rules %}
      <tr>
        <td style="text-align:left;">{{ r.item }}</td>
        <td class="mono nowrap">{{ r.operator }} {{
    r.threshold or "" }}</td>
        <td>{{ r.condition_type }}</td>
        <td>{{ r.region or "—" }}</td>
        <td style="text-align:left;">{{ r.note or "—"
    }}</td>
        <td>
          <form method="post" action="/standard/rule/{{
    r.id }}/delete"
                onsubmit="return confirm('确定删除规则「{{ r.item
    }} {{ r.operator }} {{ r.threshold or '' }}」吗?');">
            {{ form.hidden_tag() }}
            <button type="submit" class="btn btn--sm
    btn--danger">删除</button>
          </form>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% else %}
  <p class="muted">该标准暂无规则。请在下方添加第一条规则。</p>
  {% endif %}
  <h3>添加规则</h3>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="actions">
      <div>
        <label class="field__label" for="r-item">判定项</label>
        <input id="r-item" type="text" name="item"
    placeholder="如：植被覆盖度" required>
      </div>
      <div>
        <label class="field__label" for="r-op">比较符</label>
        <select id="r-op" name="operator">
          {% for o in [">=", ">", "<=", "<", "=="]
```

```
      %}<option value="{{ o }}">{{ o }}</option>{% endfor %}
        </select>
      </div>
      <div>
        <label class="field__label" for="r-thr">阈值</label>
        <input id="r-thr" type="text" name="threshold"
      placeholder="如：50" style="width:110px;">
      </div>
      <div>
        <label class="field__label" for="r-cond">条件类型</label
      >
        <select id="r-cond" name="condition_type">
          <option value="必要条件">必要条件</option>
          <option value="充分条件">充分条件</option>
        </select>
      </div>
      <div>
        <label class="field__label" for="r-region">适用分区</lab
      el>
        <input id="r-region" type="text" name="region"
      placeholder="如宁夏分区" style="width:130px;">
      </div>
      <div>
        <label class="field__label" for="r-note">备注</label>
        <input id="r-note" type="text" name="note"
      style="width:220px;">
      </div>
      <div>
        <button type="submit" class="btn">添加规则</button>
      </div>
    </div>
  </form>
  <p class="actions" style="margin-top:20px;">
    <a class="btn btn--sm btn--ghost"
    href="/standard/">返回标准列表</a>
    <a class="btn btn--sm btn--ghost"
    href="/standard/compare">省际口径比对</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/standard/compare.html ====
{% extends "base.html" %}
{% block title %}省际口径比对{% endblock %}
{% block content %}
<div class="card">
  <h2>省际口径比对</h2>
  <p class="muted">
    同一判定项在不同省份的阈值横向对比，避免人工翻阅多份标准文件。
  </p>
  {% if not groups %}
```

```
  <p class="muted">                                                                5601
    暂无规则数据。请先在「标准知识库」中为标准录入规则，                                5602
    或运行 <span class="mono">python seed_demo.py</span> 载入论文表                5603
    1 数据。                                                                       5604
  </p>                                                                             5605
  {% else %}                                                                       5606
  {% for item, rs in groups.items() %}                                             5607
  <h3>{{ item }}</h3>                                                              5608
  <table>                                                                          5609
    <thead>                                                                        5610
      <tr>                                                                         5611
        <th scope="col">省份</th>                                                  5612
        <th scope="col">标准 / 文件</th>                                           5613
        <th scope="col">条件</th>                                                  5614
        <th scope="col">条件类型</th>                                              5615
        <th scope="col">备注</th>                                                  5616
      </tr>                                                                        5617
    </thead>                                                                       5618
    <tbody>                                                                        5619
      {% for r in rs %}                                                            5620
      <tr>                                                                         5621
        <td>{{ r.standard.province }}</td>                                         5622
        <td style="text-align:left;">{{ r.standard.name                            5623
    }}</td>                                                                        5624
        <td class="mono nowrap">{{ r.operator }} {{                                5625
    r.threshold or "" }}</td>                                                      5626
        <td>{{ r.condition_type }}</td>                                            5627
        <td style="text-align:left;">{{ r.note or "—"                              5628
    }}</td>                                                                        5629
      </tr>                                                                        5630
      {% endfor %}                                                                 5631
    </tbody>                                                                       5632
  </table>                                                                         5633
  {% endfor %}                                                                     5634
  {% endif %}                                                                      5635
  <p class="actions" style="margin-top:16px;">                                     5636
    <a class="btn btn--ghost" href="/standard/">返回标准列表</a>                    5637
  </p>                                                                             5638
</div>                                                                             5639
{% endblock %}                                                                     5640
# ==== app/templates/constraint/index.html ====                                    5641
{% extends "base.html" %}                                                          5642
{% from "_macros.html" import patch_guide %}                                       5643
{% block title %}限制性指标判定{% endblock %}                                      5644
{% block content %}                                                                5645
<div class="card">                                                                 5646
  <h2>限制性指标判定</h2>                                                          5647
  <p class="muted">                                                                5648
    按论文表 2 与 4.1 节流程，对每项限制性条件逐项判定。                           5649
    任一项不通过即判定为「人工辅助或生态重建」，不再进入定量评价。                 5650
```

```
  </p>
  {{ patch_guide(projects_without_patch) }}
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先在「项目与矿山档案」中建立图斑。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">判定结果</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in patches %}
      {% set r = latest.get(p.id) %}
      <tr>
        <td>{{ p.project.name if p.project else "—" }}</td>
        <td style="text-align:left;">{{ p.name }}</td>
        <td style="text-align:left;">
          {% if r %}
            {% if r.passed %}
            <span class="tag tag--ok">通过</span>
            {% else %}
            <span class="tag tag--warn">不通过</span>
            {% endif %}
            <span class="muted">{{ r.reason }}</span>
          {% else %}
          <span class="tag tag--muted">未判定</span>
          {% endif %}
        </td>
        <td><a href="/constraint/{{ p.id }}">执行判定</a></td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/constraint/check.html ====
{% extends "base.html" %}
{% block title %}限制性条件判定 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:760px;">
  <h2>限制性条件判定 · {{ patch.name }}</h2>
  <p class="muted">
    五项条件全部通过方可进入定量评价；任一项不通过即判定为「人工辅助或生态重建」，
    不再进行后续计算。
  </p>
```

```
  <form method="post">
    {{ form.hidden_tag() }}
    {% for field, name, val in field_states %}
    <fieldset style="border:1px solid
    var(--line);border-radius:6px;padding:10px
    14px;margin:0 0 10px;">
      <legend style="padding:0 6px;font-weight:600;">{{
    name }}</legend>
      <div class="actions">
        <label class="checkbox">
          <input type="radio" name="{{ field }}" value="1"
                 {% if val is none or val %}checked{% endif
    %}>
          <span>通过</span>
        </label>
        <label class="checkbox">
          <input type="radio" name="{{ field }}" value="0"
                 {% if val is not none and not val
    %}checked{% endif %}>
          <span>不通过</span>
        </label>
      </div>
    </fieldset>
    {% endfor %}
    <p class="actions" style="margin-top:16px;">
      <button type="submit" class="btn">保存判定</button>
      <a class="btn btn--ghost" href="/constraint/">返回</a>
    </p>
  </form>
  {% if record %}
  <div style="margin-top:18px;padding:14px;border-radius:6px
    ;border-left:4px solid;
                {% if record.passed %}background:var(--ok-soft
    );border-left-color:var(--ok);
                {% else %}background:var(--danger-soft);border
    -left-color:var(--danger);{% endif %}">
    <p style="margin:0 0 4px;font-weight:600;">
      判定结论：
      {% if record.passed %}
      <span class="ok">通过，可进入定量评价</span>
      {% else %}
      <span class="warn">不通过 —— 建议生态重建或人工辅助</span>
      {% endif %}
    </p>
    <p class="muted" style="margin:0;">{{ record.reason
    }}</p>
  </div>
  {% endif %}
</div>
{% endblock %}
```

```
# ==== app/templates/ahp/index.html ====
{% extends "base.html" %}
{% block title %}AHP 权重计算{% endblock %}
{% block content %}
<div class="card" style="max-width:720px;">
  <h2>AHP 权重计算 · 选择指标层级</h2>
  <p class="muted">
    选择微地质单元类型与指标层级后，系统按论文表 3 确定判断矩阵的规模。
    岩质边坡不设土壤指标、平台不设地形指标，系统会相应调整矩阵所包含的指标。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="{{ form.unit_type.id }}">微地质单元类型</label>
      {{ form.unit_type(id=form.unit_type.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.level.id }}">指标层级</label>
      {{ form.level(id=form.level.id) }}
    </div>
    <div class="field">
      <label class="field__label" for="{{ form.l1.id }}">
        一级指标（层级选「二级指标」时需选择）
      </label>
      {{ form.l1(id=form.l1.id) }}
    </div>
    <p class="actions" style="margin-top:16px;">
      {{ form.submit(class_="btn") }}
      <a class="btn btn--ghost" href="/ahp/indicators">指标定义维护</a>
      <a class="btn btn--ghost" href="/ahp/experts">多专家矩阵</a>
    </p>
  </form>
</div>
<div class="card">
  <h2>各微地质单元当前生效权重</h2>
  <p class="muted">
    适宜性评价按此处列出的权重计算。默认采用论文表 3 的取值；
    若在本系统中用层次分析法算出了新权重并保存，则改按新权重计算。
  </p>
  <table>
    <thead>
      <tr>
        <th scope="col">微地质单元</th>
        <th scope="col">当前权重来源</th>
        <th scope="col">操作</th>
      </tr>
```

```
      </thead>
      <tbody>
        {% for r in weight_rows %}
        <tr>
          <td style="text-align:left;">{{ r.unit_type }}</td>
          <td style="text-align:left;">
            {% if r.custom %}
            <span class="tag tag--warn">{{ r.label }}</span>
            {% else %}
            <span class="tag tag--muted">{{ r.label }}</span>
            {% endif %}
          </td>
          <td class="nowrap">
            <a href="/ahp/matrix?unit_type={{ r.unit_type
      }}&level=1">一级指标权重</a>
             | 
            <a href="/ahp/matrix?unit_type={{ r.unit_type
      }}&level=1&calc=1">查看一致性</a>
            {% if r.custom %}
             | 
            <form method="post" action="/ahp/weights/reset"
      style="display:inline;"
                  onsubmit="return confirm('确定把「{{
      r.unit_type }}」恢复为论文表 3 的默认权重吗？');">
              <input type="hidden" name="csrf_token"
      value="{{ csrf_token() }}">
              <input type="hidden" name="unit_type" value="{{
      r.unit_type }}">
              <button type="submit" class="btn btn--sm
      btn--ghost">恢复默认</button>
            </form>
            {% endif %}
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
</div>
{% endblock %}
# ==== app/templates/ahp/matrix.html ====
{% extends "base.html" %}
{% block title %}判断矩阵录入{% endblock %}
{% block content %}
<div class="card">
  <h2>
    判断矩阵录入
    · {{ unit_type }}
    · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{%
    endif %}
  </h2>
```

```
<p class="muted">
  采用 1-9 标度：1 同等重要、3 稍微重要、5 明显重要、7 强烈重要、9 极端重要，
  2、4、6、8 为相邻判断的中间值。只需录入上三角，对角线为 1，下三角自动取倒数。
  矩阵已按论文表 3 的权重预填。
</p>
<form method="post">
  {{ form.hidden_tag() }}
  <table style="width:auto;">
    <thead>
      <tr>
        <th></th>
        {% for name in names %}<th scope="col">{{ name }}</th>{% endfor %}
      </tr>
    </thead>
    <tbody>
      {% for i in range(names|length) %}
      <tr>
        <th scope="row">{{ names[i] }}</th>
        {% for j in range(names|length) %}
          {% if i == j %}
          <td class="muted num">1</td>
          {% elif i < j %}
          <td>
            <label class="muted" style="position:absolute;left:-9999px;"
                   for="m-{{ i }}-{{ j }}">{{ names[i] }} 相对 {{ names[j] }} 的重要性</label>
            <input id="m-{{ i }}-{{ j }}" type="number" step="0.01" min="0.01"
                   name="m_{{ i }}_{{ j }}" value="{{ default[i][j] }}">
          </td>
          {% else %}
          <td class="muted num">{{ default[i][j] }}</td>
          {% endif %}
        {% endfor %}
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p class="actions">
    <button type="submit" class="btn" name="action" value="calc">计算权重</button>
    {% if result and result.consistent %}
    <button type="submit" class="btn" name="action" value="save">保存并生效</button>
    {% endif %}
    <a class="btn btn--ghost" href="/ahp/">重新选择</a>
  </p>
```

```
</form>
<p class="muted" style="margin-top:10px;">
  当前「{{ unit_type }}」生效权重：<strong>{{ source_label }}</strong>。
  点击「保存并生效」后，本单元类型的适宜性评价将采用此处算出的权重；
  如需回到论文取值，可在 <a href="/ahp/">权重首页</a> 恢复默认。
</p>
{% if result %}
<hr style="margin:22px 0;border:none;border-top:1px solid var(--line);">
<h3>计算结果</h3>
<table style="width:auto;">
  <thead>
    <tr>
      <th scope="col">指标</th>
      {% for name in names %}<th scope="col">{{ name }}</th>{% endfor %}
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">权重</th>
      {% for w in result.weights %}<td class="mono">{{ w }}</td>{% endfor %}
    </tr>
  </tbody>
</table>
<div class="stat-row">
  <div>
    <div class="stat__label">最大特征值 λmax</div>
    <div class="stat__value mono">{{ result.lambda_max }}</div>
  </div>
  <div>
    <div class="stat__label">一致性指标 CI</div>
    <div class="stat__value mono">{{ "%.4f"|format(result.ci) }}</div>
  </div>
  <div>
    <div class="stat__label">一致性比例 CR</div>
    <div class="stat__value mono">{{ "%.4f"|format(result.cr) }}</div>
  </div>
  <div>
    <div class="stat__label">一致性检验（要求 CR &lt; 0.1）</div>
    <div class="stat__value">
      {% if result.consistent %}<span class="ok">通过</span>
      {% else %}<span class="warn">不通过</span>{% endif %}
    </div>
  </div>
```

```
  </div>
  {% if not result.consistent and result.most_inconsistent %}
  <p style="margin-top:14px;padding:12px;border-radius:6px;background:var(--danger-soft);
            border-left:4px solid var(--danger);">
    <strong class="warn">建议优先调整：</strong>
    第 {{ result.most_inconsistent.i + 1 }} 行、第 {{ result.most_inconsistent.j + 1 }} 列的比较
    （当前 <span class="mono num">{{ result.most_inconsistent.value|round(3) }}</span>，
    理想约 <span class="mono num">{{ result.most_inconsistent.expected|round(3) }}</span>）
  </p>
  {% endif %}
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/ahp/indicators.html ====
{% extends "base.html" %}
{% block title %}指标定义维护{% endblock %}
{% block content %}
<div class="card">
  <h2>指标体系维护</h2>
  <p class="muted">
    按论文表 2 与表 3 建立的指标树。此处可核对指标名称、单位、标准化类型、
    理想取值与取值依据，并按需调整启用该指标的微地质单元。
    计算时的权威取值仍以 <span class="mono">app/utils/constants.py</span> 为准，
    本页用于展示与核对。
  </p>
  <p class="actions" style="margin-bottom:14px;">
    <a class="btn btn--sm btn--ghost"
    href="/ahp/">返回权重计算</a>
    <a class="btn btn--sm btn--ghost"
    href="/ahp/experts">多专家矩阵</a>
    <form method="post" action="/ahp/indicators/reset"
    style="display:inline;"
          onsubmit="return confirm('将按论文取值重置全部指标定义，自定义修改会丢失。
    确定继续？');">
      {{ form.hidden_tag() }}
      <button type="submit" class="btn btn--sm
    btn--danger">按论文取值重置</button>
    </form>
  </p>
  {% for level, title in [(1, "一级指标"), (2, "二级指标"), (3,
    "三级指标")] %}
  {% set level_items = all_ind|selectattr("level",
    "equalto", level)|list %}
  {% if level_items %}
```

```
  <h3>{{ title }} ({{ level_items|length }} 项)</h3>
  <table>
    <thead>
      <tr>
        <th scope="col">指标</th>
        <th scope="col">上级指标</th>
        <th scope="col">单位</th>
        <th scope="col">标准化类型</th>
        <th scope="col">理想取值</th>
        <th scope="col">启用单元</th>
        <th scope="col">限制性</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for i in level_items %}
      <tr>
        <td style="text-align:left;">
          <span class="mono muted">{{ i.code
    }}</span><br>{{ i.name }}
        </td>
        <td>{{ (all_ind|selectattr("code", "equalto",
    i.parent_code)|list|first).name if i.parent_code else
    "—" }}</td>
        <td>{{ i.unit or "—" }}</td>
        <td>{{ type_label(i.ideal_type) }}</td>
        <td class="mono">{{ i.ideal_value or "—" }}</td>
        <td style="text-align:left;">{{ i.enabled_units or
    "—" }}</td>
        <td>
          {% if i.is_constraint %}<span class="tag
    tag--warn">是</span>
          {% else %}<span class="tag tag--muted">否</span>{%
    endif %}
        </td>
        <td><a href="/ahp/indicators/{{ i.code
    }}/edit">编辑</a></td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
  {% endfor %}
</div>
{% endblock %}
# ==== app/templates/ahp/indicator_edit.html ====
{% extends "base.html" %}
{% block title %}编辑指标 · {{ ind.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:680px;">
```

```
<h2>编辑指标 · {{ ind.name }}</h2>
<p class="muted">
  指标编码 <span class="mono">{{ ind.code }}</span>、层级 {{ ind.level }} 级，
  由论文指标体系确定，不可修改。
</p>
<form method="post">
  {{ form.hidden_tag() }}
  <div class="field">
    <label class="field__label" for="name">指标名称</label>
    <input id="name" type="text" name="name" style="width:100%;" required
           value="{{ ind.name }}">
  </div>
  <div class="field">
    <label class="field__label" for="unit">计量单位</label>
    <input id="unit" type="text" name="unit" style="width:180px;"
           value="{{ ind.unit or '' }}">
  </div>
  <div class="field">
    <label class="field__label" for="ideal-type">标准化类型</label>
    <select id="ideal-type" name="ideal_type">
      {% for code, label in [("benefit","越大越优"),("cost","越小越优"),("mid","适中型"),("discrete","离散赋值")] %}
      <option value="{{ code }}" {% if ind.ideal_type == code %}selected{% endif %}>{{ label }}</option>
      {% endfor %}
    </select>
  </div>
  <div class="field">
    <label class="field__label" for="ideal-value">理想取值</label>
    <input id="ideal-value" type="text" name="ideal_value" style="width:100%;"
           value="{{ ind.ideal_value or '' }}">
    <p class="muted" style="margin:4px 0 0;">
      如土石比 0.8、土层厚度 20、pH 5~9、植被覆盖度 75。
    </p>
  </div>
  <fieldset style="border:1px solid var(--line);border-radius:6px;padding:10px 14px;margin:0 0 12px;">
    <legend style="padding:0 6px;font-weight:600;">启用该指标的微地质单元</legend>
    {% for t in unit_types %}
    <label class="checkbox" style="margin-right:18px;">
      <input type="checkbox" name="enabled_units" value="{{ t }}"
```

```
                    {% if t in ind.units_list() %}checked{%                    6101
      endif %}>                                                                6102
            <span>{{ t }}</span>                                               6103
          </label>                                                             6104
          {% endfor %}                                                         6105
          <p class="muted" style="margin:8px 0 0;">                            6106
            论文设定：岩质边坡不启用土壤指标，平台不启用地形指标。                6107
          </p>                                                                 6108
      </fieldset>                                                              6109
      <div class="field">                                                      6110
          <label class="checkbox" for="is-constraint">                         6111
            <input id="is-constraint" type="checkbox"                          6112
      name="is_constraint" value="1"                                           6113
                    {% if ind.is_constraint %}checked{% endif %}>              6114
            <span>同时作为限制性指标（参与一票否决判定）</span>                    6115
          </label>                                                             6116
      </div>                                                                   6117
      <div class="field">                                                      6118
          <label class="field__label" for="description">取值依据说明</        6119
      label>                                                                   6120
          <textarea id="description" name="description"                        6121
      style="min-height:88px;">{{ ind.description or ''                        6122
      }}</textarea>                                                            6123
      </div>                                                                   6124
      <p class="actions" style="margin-top:16px;">                             6125
          <button type="submit" class="btn">保存</button>                      6126
          <a class="btn btn--ghost" href="/ahp/indicators">返回指标列          6127
      表</a>                                                                   6128
      </p>                                                                     6129
    </form>                                                                    6130
</div>                                                                         6131
{% endblock %}                                                                 6132
# ==== app/templates/ahp/experts.html ====                                     6133
{% extends "base.html" %}                                                      6134
{% block title %}多专家判断矩阵{% endblock %}                                  6135
{% block content %}                                                            6136
<div class="card">                                                             6137
    <h2>多专家判断矩阵 · 录入与汇总</h2>                                       6138
    <p class="muted">                                                          6139
      论文明确权重由「组织 10 位专家对各项指标权重进行打分」确定。              6140
      此页支持多位专家分别录入判断矩阵，再用几何平均汇总，                      6141
      保持矩阵互反性，并重新做一致性检验。                                      6142
    </p>                                                                       6143
    <form method="get" class="actions" style="margin-bottom:14                 6144
      px;">                                                                    6145
      <div>                                                                    6146
          <label class="field__label" for="unit-type">微地质单元类型</l       6147
      abel>                                                                    6148
          <select id="unit-type" name="unit_type"                              6149
      onchange="this.form.submit()">                                           6150
```

```
        {% for t in ["岩质边坡", "碎石边坡", "平台"] %}
        <option value="{{ t }}" {% if unit_type == t
    %}selected{% endif %}>{{ t }}</option>
        {% endfor %}
      </select>
    </div>
    <div>
      <label class="field__label" for="level">指标层级</label>
      <select id="level" name="level"
    onchange="this.form.submit()">
        <option value="1" {% if level == 1 %}selected{%
    endif %}>一级指标</option>
        <option value="2" {% if level == 2 %}selected{%
    endif %}>二级指标</option>
      </select>
    </div>
  </form>
  <p class="muted">
    当前范围：<strong>{{ unit_type }}</strong> ·
    {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif
    %}。
    若选二级指标，请先到
    <a href="/ahp/">权重计算</a> 选择一级指标后进入本页。
  </p>
  {% if rows %}
  <table>
    <caption class="muted">已录入 {{ rows|length }}
    位专家的矩阵</caption>
    <thead>
      <tr>
        <th scope="col">专家</th>
        <th scope="col">矩阵规模</th>
        <th scope="col">该矩阵 CR</th>
        <th scope="col">一致性</th>
        <th scope="col">录入时间</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for r in rows %}
      <tr>
        <td>{{ r.expert_name }}</td>
        <td class="num">{{ n }} × {{ n }}</td>
        <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is
    not none else "—" }}</td>
        <td>
          {% if r.cr is not none and r.cr < 0.1 %}<span
    class="tag tag--ok">通过</span>
          {% else %}<span class="tag tag--warn">不通过</span>{%
    endif %}
```

```
        </td>
        <td class="mono nowrap">{{ r.created_at.strftime("%Y
    -%m-%d %H:%M") if r.created_at else "—" }}</td>
        <td>
          <form method="post" action="/ahp/experts/{{ r.id
    }}/delete"
                onsubmit="return confirm('确定删除专家「{{
    r.expert_name }}」的矩阵吗？');">
            {{ form.hidden_tag() }}
            <button type="submit" class="btn btn--sm
    btn--danger">删除</button>
          </form>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p class="actions" style="margin-bottom:10px;">
    <a class="btn" href="/ahp/experts/aggregate?unit_type={{
    unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{%
    endif %}">
      汇总计算（几何平均）
    </a>
  </p>
  {% else %}
  <p class="muted">尚未录入任何专家矩阵。请在下方录入第一位专家的判断矩阵。</p>
  {% endif %}
  <h3>录入一位专家的判断矩阵</h3>
  <form method="post">
    {{ form.hidden_tag() }}
    <input type="hidden" name="action" value="add">
    <input type="hidden" name="unit_type" value="{{
    unit_type }}">
    <input type="hidden" name="level" value="{{ level }}">
    {% if l1 %}<input type="hidden" name="l1" value="{{ l1
    }}">{% endif %}
    <div class="field">
      <label class="field__label" for="expert-name">专家姓名</la
    bel>
      <input id="expert-name" type="text" name="expert_name"
             style="width:220px;" placeholder="如：张工">
    </div>
    <table style="width:auto;">
      <thead>
        <tr>
          <th></th>
          {% for name in names %}<th scope="col">{{ name
    }}</th>{% endfor %}
        </tr>
      </thead>
```

```
        <tbody>
          {% for i in range(n) %}
          <tr>
            <th scope="row">{{ names[i] }}</th>
            {% for j in range(n) %}
              {% if i == j %}
              <td class="muted num">1</td>
              {% elif i < j %}
              <td>
                <label class="muted" style="position:absolute;left:-9999px;"
                       for="e-{{ i }}-{{ j }}">{{ names[i] }} 相对 {{ names[j] }} 的重要性</label>
                <input id="e-{{ i }}-{{ j }}" type="number" step="0.01" min="0.01"
                       name="m_{{ i }}_{{ j }}" value="{{ default[i][j] }}">
              </td>
              {% else %}
              <td class="muted num">{{ default[i][j] }}</td>
              {% endif %}
            {% endfor %}
          </tr>
          {% endfor %}
        </tbody>
      </table>
      <p class="actions" style="margin-top:12px;">
        <button type="submit" class="btn">录入该专家矩阵</button>
        <a class="btn btn--ghost" href="/ahp/indicators">指标维护</a>
        <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/ahp/experts_aggregate.html ====
{% extends "base.html" %}
{% block title %}多专家矩阵汇总{% endblock %}
{% block content %}
<div class="card">
  <h2>多专家矩阵汇总 · {{ unit_type }} · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif %}</h2>
  <p class="muted">
    对各位专家的判断矩阵按对应元素取几何平均：
    a<sub>ij</sub> = (a<sub>ij</sub><sup>(1)</sup> × … ×
    a<sub>ij</sub><sup>(k)</sup>)<sup>1/k</sup>。
    该方式可保持矩阵的互反性，汇总后重新计算权重与一致性比例。
  </p>
  {% if not result %}
  <p class="muted">
```

```
    该范围内尚未录入专家矩阵。                                                6301
    <a href="/ahp/experts?unit_type={{ unit_type                              6302
    }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif                       6303
    %}">去录入</a>                                                            6304
</p>                                                                          6305
{% else %}                                                                    6306
<div class="stat-row">                                                        6307
    <div>                                                                     6308
        <div class="stat__label">参与汇总的专家</div>                          6309
        <div class="stat__value">{{ result.count }} 位</div>                  6310
    </div>                                                                    6311
    <div>                                                                     6312
        <div class="stat__label">λmax</div>                                   6313
        <div class="stat__value mono">{{ result.lambda_max                    6314
    }}</div>                                                                  6315
    </div>                                                                    6316
    <div>                                                                     6317
        <div class="stat__label">CI</div>                                     6318
        <div class="stat__value mono">{{                                      6319
    "%.4f"|format(result.ci) }}</div>                                         6320
    </div>                                                                    6321
    <div>                                                                     6322
        <div class="stat__label">CR（要求 &lt; 0.1）</div>                     6323
        <div class="stat__value mono">{{                                      6324
    "%.4f"|format(result.cr) }}</div>                                         6325
    </div>                                                                    6326
    <div>                                                                     6327
        <div class="stat__label">一致性检验</div>                              6328
        <div class="stat__value">                                             6329
            {% if result.consistent %}<span class="ok">通过</span>            6330
            {% else %}<span class="warn">不通过</span>{% endif %}             6331
        </div>                                                                6332
    </div>                                                                    6333
</div>                                                                        6334
<h3>汇总后权重</h3>                                                           6335
<table style="width:auto;">                                                   6336
    <thead>                                                                   6337
        <tr>                                                                  6338
            <th scope="col">指标</th>                                         6339
            {% for name in names %}<th scope="col">{{ name                    6340
    }}</th>{% endfor %}                                                       6341
        </tr>                                                                 6342
    </thead>                                                                  6343
    <tbody>                                                                   6344
        <tr>                                                                  6345
            <th scope="row">权重</th>                                         6346
            {% for w in result.weights %}<td class="mono">{{ w                6347
    }}</td>{% endfor %}                                                       6348
        </tr>                                                                 6349
    </tbody>                                                                  6350
```

```
</table>
<h3>各专家矩阵的一致性</h3>
<table style="width:auto;">
  <thead>
    <tr><th scope="col">专家</th><th scope="col">该矩阵
  CR</th><th scope="col">判定</th></tr>
  </thead>
  <tbody>
    {% for r in rows %}
    <tr>
      <td>{{ r.expert_name }}</td>
      <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is
  not none else "—" }}</td>
      <td>
        {% if r.cr is not none and r.cr < 0.1 %}<span
  class="ok">通过</span>
        {% else %}<span class="warn">不通过</span>{% endif %}
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% if not result.consistent and result.most_inconsistent
  %}
<p style="padding:12px;border-radius:6px;background:var(--
  danger-soft);
          border-left:4px solid var(--danger);">
  <strong class="warn">汇总矩阵一致性不通过，建议优先调整：</strong>
  第 {{ result.most_inconsistent.i + 1 }} 行、第 {{
  result.most_inconsistent.j + 1 }} 列的比较
  （当前 <span class="mono">{{ result.most_inconsistent.value
  |round(3) }}</span>，
  理想约 <span class="mono">{{ result.most_inconsistent.expec
  ted|round(3) }}</span>）。
  该偏差通常来自个别专家打分，可先核对一致性较差的专家矩阵。
</p>
{% endif %}
{% endif %}
<p class="actions" style="margin-top:16px;">
  <form method="post"
        action="/ahp/experts?unit_type={{ unit_type
  }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif %}"
        style="display:inline;"
        onsubmit="return confirm('保存后，「{{ unit_type
  }}」的适宜性评价将改用这组汇总权重计算。确定继续吗？');">
    {{ form.hidden_tag() }}
    <input type="hidden" name="action" value="save_agg">
    <button type="submit" class="btn" {% if not
  result.consistent %}disabled{% endif %}>
      保存为生效权重
```

```
        </button>
      </form>
      <a class="btn btn--ghost" href="/ahp/experts?unit_type={{ unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif %}">返回录入</a>
      <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
    </p>
    {% if not result.consistent %}
    <p class="muted" style="margin-top:8px;">
      汇总矩阵一致性未通过，暂不能保存为生效权重。
    </p>
    {% endif %}
</div>
{% endblock %}
# ==== app/templates/indicator/index.html ====
{% extends "base.html" %}
{% from "_macros.html" import patch_guide %}
{% block title %}指标数据采集{% endblock %}
{% block content %}
<div class="card">
    <h2>指标数据采集与管理</h2>
    <p class="muted">
      按微地质单元组织录入。系统按单元类型自动决定应录入的指标：
      岩质边坡不启用土壤指标、平台不启用地形指标。
    </p>
    <p class="actions" style="margin-bottom:12px;">
      <a class="btn btn--sm" href="/indicator/import">批量导入（CSV / Excel）</a>
    </p>
    {{ patch_guide(projects_without_patch) }}
    {% if not patches %}
    <p class="muted">
      暂无矿山图斑。请先在「项目与矿山档案」中建立图斑，
      或运行 <span class="mono">python seed_demo.py</span>
      载入论文案例。
    </p>
    {% else %}
    {% for p in patches %}
    <div class="patch-panel">
      <div class="patch-panel__head">
        <h3 class="patch-panel__name">{{ p.name }}</h3>
        <span class="patch-panel__meta">
          所属项目：{{ p.project.name if p.project else "—" }}
        </span>
        <span class="patch-panel__meta">
          微地质单元：{{ p.units.count() }} 个
        </span>
        {% if p.units.count() %}
        <span class="patch-panel__meta patch-panel__tail">
          <a href="/indicator/import">批量导入本图斑数据</a>
```

```
        </span>                                                                  6451
        {% endif %}                                                              6452
    </div>                                                                       6453
    {% if not p.units.count() %}                                                 6454
    <div class="notice notice--warn">                                            6455
        <p class="notice__title">该图斑尚未划分微地质单元，暂不能录入指标数据。</p>    6456
        <p style="margin:0;">                                                    6457
            指标数据是按微地质单元（岩质边坡 / 碎石边坡 / 平台）分别组织的，          6458
            请先完成划分，再回到本页录入。                                         6459
        </p>                                                                     6460
        <p style="margin:10px 0 0;">                                             6461
            <a class="btn btn--sm" href="/unit/{{ p.id                           6462
    }}">去划分微地质单元</a>                                                     6463
        </p>                                                                     6464
    </div>                                                                       6465
    {% else %}                                                                   6466
    <table>                                                                      6467
        <thead>                                                                  6468
            <tr>                                                                 6469
                <th scope="col">微地质单元</th>                                  6470
                <th scope="col">面积占比</th>                                    6471
                <th scope="col">数据状态</th>                                    6472
                <th scope="col">操作</th>                                        6473
            </tr>                                                                6474
        </thead>                                                                 6475
        <tbody>                                                                  6476
            {% for u in p.units %}                                               6477
            <tr>                                                                 6478
                <td>{{ u.unit_type }}</td>                                       6479
                <td class="num">                                                 6480
                    {% if u.ratio is not none %}{{                               6481
    "%.2f%%"|format(u.ratio * 100) }}{% else %}—{% endif %}                      6482
                </td>                                                            6483
                <td>                                                             6484
                    {% if u.indicator_data %}                                    6485
                    <span class="tag tag--ok">已录入</span>                      6486
                    {% else %}                                                   6487
                    <span class="tag tag--muted">未录入</span>                   6488
                    {% endif %}                                                  6489
                </td>                                                            6490
                <td><a href="/indicator/{{ u.id }}">录入 /                       6491
    编辑</a></td>                                                                6492
            </tr>                                                                6493
            {% endfor %}                                                         6494
        </tbody>                                                                 6495
    </table>                                                                     6496
    {% endif %}                                                                  6497
</div>                                                                           6498
{% endfor %}                                                                     6499
{% endif %}                                                                      6500
```

```
</div>
{% endblock %}
# ==== app/templates/indicator/edit.html ====
{% extends "base.html" %}
{% block title %}指标数据录入 · {{ unit.unit_type }}{% endblock %}
{% block content %}
<div class="card" style="max-width:820px;">
  <h2>指标数据录入 · {{ unit.unit_type }}</h2>
  <p class="muted">
    输入框留空表示该指标「不适用」，与「实测值为 0」含义不同，计算时会被跳过。
    岩质边坡不设土壤指标、平台不设地形指标，系统已按单元类型隐藏不相关字段。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    {% for group_name, fields, has_hm in groups %}
    <h3>{{ group_name }}</h3>
    {% for fname, cname, unit_ in fields %}
    <div class="field">
      <label class="field__label" for="f-{{ fname }}">
        {{ cname }}{% if unit_ %} ({{ unit_ }}){% endif %}
      </label>
      {% if fname == "aspect" %}
      <select id="f-{{ fname }}" name="aspect">
        <option value="">— 不适用 —</option>
        {% for a in aspect_types %}
        <option value="{{ a }}" {% if current.get("aspect")
    == a %}selected{% endif %}>{{ a }}</option>
        {% endfor %}
      </select>
      {% else %}
      <input id="f-{{ fname }}" type="number" step="any"
    name="{{ fname }}"
             value="{{ current.get(fname) if
    current.get(fname) is not none else '' }}">
      {% endif %}
    </div>
    {% endfor %}
    {% endfor %}
    {% for group_name, fields, has_hm in groups %}
    {% if has_hm %}
    <h3>重金属含量（mg/kg）</h3>
    <p class="muted">未检测的金属可留空，系统按论文注释视为未受污染处理。</p>
    <table style="width:auto;">
      <thead>
        <tr>
          {% for m in heavy_metals %}<th scope="col">{{ m
    }}</th>{% endfor %}
        </tr>
      </thead>
      <tbody>
```

```
            <tr>                                                                    6551
              {% for m in heavy_metals %}                                           6552
              <td>                                                                  6553
                <label class="muted" style="position:absolute;                      6554
    left:-9999px;"                                                                  6555
                       for="hm-{{ m }}">{{ m }} 含量</label>                        6556
                <input id="hm-{{ m }}" type="number"                                6557
    step="any" name="hm_{{ m }}"                                                    6558
                       value="{{ current.get('heavy',                               6559
    {}).get(m) if current.get('heavy', {}).get(m) is not                            6560
    none else '' }}">                                                               6561
              </td>                                                                 6562
              {% endfor %}                                                          6563
            </tr>                                                                   6564
          </tbody>                                                                  6565
        </table>                                                                    6566
        {% endif %}                                                                 6567
    {% endfor %}                                                                    6568
    <p class="actions" style="margin-top:18px;">                                    6569
      <button type="submit" class="btn">保存</button>                               6570
      <a class="btn btn--ghost" href="/indicator/">返回</a>                         6571
    </p>                                                                            6572
  </form>                                                                           6573
</div>                                                                              6574
{% endblock %}                                                                      6575
# ==== app/templates/indicator/import.html ====                                     6576
{% extends "base.html" %}                                                           6577
{% block title %}批量导入指标数据{% endblock %}                                     6578
{% block content %}                                                                 6579
<div class="card" style="max-width:860px;">                                         6580
  <h2>批量导入指标数据</h2>                                                         6581
  <p class="muted">                                                                 6582
    支持 CSV 与 Excel（.xlsx/.xls）。文件每行对应一个微地质单元，                  6583
    系统按「单元类型」匹配图斑内已划分的单元后写入数据。                           6584
    先下载模板按格式填写，可减少出错。                                             6585
  </p>                                                                              6586
  <p class="actions" style="margin-bottom:14px;">                                   6587
    <a class="btn btn--sm btn--ghost"                                               6588
    href="/indicator/import/template">下载导入模板（CSV）</a>                       6589
    <a class="btn btn--sm btn--ghost"                                               6590
    href="/indicator/">返回指标数据</a>                                             6591
  </p>                                                                              6592
  <form method="post" enctype="multipart/form-data">                                6593
    {{ form.hidden_tag() }}                                                         6594
    <div class="field">                                                             6595
      <label class="field__label" for="patch-id">目标矿山图斑</lab                  6596
    el>                                                                             6597
      <select id="patch-id" name="patch_id" required>                               6598
        <option value="">— 请选择 —</option>                                        6599
        {% for p in patches %}                                                      6600
```

```
            <option value="{{ p.id }}">{{ p.name }} ({{                6601
        p.units.count() }} 个单元)</option>                                6602
            {% endfor %}                                                   6603
          </select>                                                        6604
        </div>                                                             6605
        <div class="field">                                                6606
          <label class="field__label" for="datafile">数据文件</label      6607
        >                                                                  6608
          <input id="datafile" type="file" name="datafile"                 6609
                 accept=".csv,.xlsx,.xls" required>                        6610
        </div>                                                             6611
        <p class="actions" style="margin-top:16px;">                       6612
          <button type="submit" class="btn">开始导入</button>              6613
        </p>                                                               6614
      </form>                                                              6615
      <h3>模板列说明</h3>                                                  6616
      <p class="muted">                                                    6617
        共 {{ headers|length }} 列。留空表示该指标「不适用」。             6618
        坡向须填「阳坡 / 阴阳坡 / 阴坡」之一。未检测的重金属留空即可       6619
        （按论文注释视为未受污染）。                                       6620
      </p>                                                                 6621
      <p class="mono muted" style="line-height:1.9;">{{                   6622
        headers|join(" , ") }}</p>                                         6623
      <h3>导入规则</h3>                                                    6624
      <ul class="muted" style="padding-left:20px;line-height:1.9         6625
        ;">                                                                6626
        <li>「单元类型」必须与图斑内已划分的单元一致，否则该行会被跳过并在结果中提示。</li>   6627
        <li>同一单元类型若在文件中出现多行，以最后一行为准。</li>          6628
        <li>已有数据的单元会被覆盖更新；没有数据的单元将新建记录。</li>    6629
        <li>数值列请填纯数字，不要带单位（如填 <span class="mono">10</span> 而非   6630
        <span class="mono">10cm</span>）。</li>                            6631
      </ul>                                                                6632
    </div>                                                                 6633
    {% if report %}                                                        6634
    <div class="card" style="max-width:860px;">                            6635
      <h2>导入结果</h2>                                                    6636
      <div class="stat-row">                                               6637
        <div>                                                              6638
          <div class="stat__label">目标图斑</div>                          6639
          <div style="font-size:15px;">{{ report.patch }}</div>            6640
        </div>                                                             6641
        <div>                                                              6642
          <div class="stat__label">成功导入</div>                          6643
          <div class="stat__value ok">{{ report.saved }}                   6644
        个单元</div>                                                       6645
        </div>                                                             6646
        <div>                                                              6647
          <div class="stat__label">问题行</div>                            6648
          <div class="stat__value {% if report.errors %}warn{%             6649
        endif %}">{{ report.errors|length }}</div>                         6650
```

```
      </div>                                                                    6651
    </div>                                                                      6652
    {% if report.errors %}                                                      6653
    <h3>未导入的行</h3>                                                         6654
    <ul class="muted" style="padding-left:20px;line-height:1.9                  6655
      ;">                                                                       6656
      {% for e in report.errors %}<li>{{ e }}</li>{% endfor %}                  6657
    </ul>                                                                       6658
    {% else %}                                                                  6659
    <p class="ok" style="margin-top:12px;">全部数据行均已成功导入。</p>         6660
    {% endif %}                                                                 6661
    <p class="actions" style="margin-top:16px;">                                6662
      <a class="btn btn--ghost" href="/indicator/">查看指标数据</a>             6663
    </p>                                                                        6664
</div>                                                                          6665
{% endif %}                                                                     6666
{% endblock %}                                                                  6667
# ==== app/templates/evaluate/index.html ====                                   6668
{% extends "base.html" %}                                                       6669
{% from "_macros.html" import patch_guide %}                                    6670
{% block title %}适宜性定量评价{% endblock %}                                   6671
{% block content %}                                                             6672
<div class="card">                                                              6673
    <h2>适宜性定量评价</h2>                                                     6674
    <p class="muted">                                                           6675
      系统按论文式（8） 对各项指标做标准化，按式（9） 分单元赋分并按面积占比加权合成，  6676
      最后按论文表 4 判定适宜性等级。未满足前置条件的图斑不能执行评价，        6677
      下表会列出所缺步骤。                                                      6678
    </p>                                                                        6679
    {{ patch_guide(projects_without_patch) }}                                   6680
    {% if not patches %}                                                        6681
    <p class="muted">暂无矿山图斑。请先到「项目与矿山档案」建立图斑。</p>      6682
    {% else %}                                                                  6683
    <table>                                                                     6684
      <thead>                                                                   6685
        <tr>                                                                    6686
          <th scope="col">所属项目</th>                                         6687
          <th scope="col">矿山图斑</th>                                         6688
          <th scope="col">前置条件</th>                                         6689
          <th scope="col">最近评价结果</th>                                     6690
          <th scope="col">操作</th>                                             6691
        </tr>                                                                   6692
      </thead>                                                                  6693
      <tbody>                                                                   6694
        {% for p in patches %}                                                  6695
        {% set pr = progress.get(p.id) %}                                       6696
        <tr>                                                                    6697
          <td>{{ p.project.name if p.project else "—" }}</td>                   6698
          <td style="text-align:left;">{{ p.name }}</td>                        6699
          <td style="text-align:left;">                                         6700
```

```
            {% if pr and pr.ready %}
            <span class="tag tag--ok">已具备</span>
            {% elif pr %}
            <span class="tag tag--warn">缺：{{ pr.blocking[0].label }}</span>
            <span class="muted">{{ pr.blocking[0].reason }}</span>
            {% endif %}
          </td>
          <td>
            {% if latest.get(p.id) %}
            F = <span class="mono num">{{ latest[p.id].final_score }}</span>,
            <span class="tag tag--ok">{{ latest[p.id].grade }}</span>
            {% else %}
            <span class="tag tag--muted">未评价</span>
            {% endif %}
          </td>
          <td class="nowrap">
            <a href="/evaluate/{{ p.id }}">执行评价</a>
            {% if latest.get(p.id) %}
             | <a href="/evaluate/result/{{ latest[p.id].id }}">查看结果</a>
            {% endif %}
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% endif %}
</div>
{% endblock %}
# ==== app/templates/evaluate/run.html ====
{% extends "base.html" %}
{% import "_macros.html" as ui %}
{% block title %}执行评价 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:880px;">
    <h2>执行适宜性评价 · {{ patch.name }}</h2>
    <p class="muted">
      系统按论文式（8） 对各项指标做标准化，按式（9） 分单元赋分并按面积占比加权合成，
      最后按论文表 4 判定适宜性等级。评价结果会保存，可随时回看。
    </p>
    {{ ui.workflow_steps(progress) }}
    {{ ui.workflow_blocking(progress) }}
    {% if progress.ready %}
    <form method="post">
      {{ form.hidden_tag() }}
      <p class="actions">
```

```
        <button type="submit" class="btn">开始评价</button>
        <a class="btn btn--ghost" href="/evaluate/">返回</a>
      </p>
    </form>
    {% else %}
    <p class="actions">
      <button type="button" class="btn" disabled>开始评价</button>
      <a class="btn btn--ghost" href="/evaluate/">返回</a>
    </p>
    <p class="muted">前置步骤全部完成后，该按钮才可用。</p>
    {% endif %}
</div>
<div class="card" style="max-width:880px;">
    <h2>当前流程状态</h2>
    <p class="muted">按论文 4.1 节的顺序列出，限制性条件判定在前。</p>
    <table>
      <thead>
        <tr><th scope="col">步骤</th><th scope="col">状态</th><th
      scope="col">说明</th></tr>
      </thead>
      <tbody>
        <tr>
          <td>限制性条件判定</td>
          <td>
            {% if progress.constraint.passed %}<span
      class="tag tag--ok">已通过</span>
            {% elif progress.constraint.done %}<span
      class="tag tag--warn">未通过</span>
            {% else %}<span class="tag tag--muted">待完成</span>{
      % endif %}
          </td>
          <td style="text-align:left;">{{
      progress.constraint.detail }}</td>
        </tr>
        <tr>
          <td>划分微地质单元</td>
          <td>
            {% if progress.units.done %}<span class="tag
      tag--ok">已完成</span>
            {% else %}<span class="tag tag--muted">待完成</span>{
      % endif %}
          </td>
          <td style="text-align:left;">{{
      progress.units.detail }}</td>
        </tr>
        <tr>
          <td>录入指标数据</td>
          <td>
            {% if progress.data.ready %}<span class="tag
      tag--ok">已完成</span>
```

```
            {% else %}<span class="tag tag--muted">待完成</span>{
    % endif %}
          </td>
          <td style="text-align:left;">{{
    progress.data.detail }}</td>
        </tr>
      </tbody>
    </table>
</div>
{% if weight_sources %}
<div class="card" style="max-width:880px;">
    <h2>本次将采用的指标权重</h2>
    <p class="muted">
      权重决定各指标对评价结果的影响程度。默认采用论文表 3 的取值；
      若已用层次分析法算出新权重并保存，则按新权重计算。
    </p>
    <table style="max-width:720px;">
      <thead>
        <tr><th scope="col">微地质单元</th><th
    scope="col">权重来源</th></tr>
      </thead>
      <tbody>
        {% for ut, label in weight_sources.items() %}
        <tr>
          <td style="text-align:left;">{{ ut }}</td>
          <td style="text-align:left;">
            {% if "自定义" in label %}
            <span class="tag tag--warn">{{ label }}</span>
            {% else %}
            <span class="tag tag--muted">{{ label }}</span>
            {% endif %}
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
</div>
{% endif %}
{% endblock %}
# ==== app/templates/evaluate/result.html ====
{% extends "base.html" %}
{% block title %}评价结果{% endblock %}
{% block content %}
<div class="card">
    <h2>适宜性评价结果 · {{ patch.name if patch else "" }}</h2>
    <div class="stat-row">
      <div>
        <div class="stat__label">最终评价因子 F</div>
        <div class="stat__value mono">{{ r.final_score
    }}</div>
```

```
      </div>                                                                    6851
      <div>                                                                     6852
        <div class="stat__label">适宜性等级</div>                               6853
        <div class="stat__value ok">{{ r.grade }}</div>                         6854
      </div>                                                                    6855
      <div style="flex:1;min-width:240px;">                                     6856
        <div class="stat__label">推荐措施</div>                                 6857
        <div style="font-size:15px;">{{ r.measures }}</div>                     6858
      </div>                                                                    6859
    </div>                                                                      6860
    {% if unit_scores %}                                                        6861
    <h3>各微地质单元得分</h3>                                                   6862
    <table style="max-width:860px;">                                            6863
      <thead>                                                                   6864
        <tr>                                                                    6865
          <th scope="col">微地质单元</th>                                       6866
          <th scope="col">面积占比</th>                                         6867
          <th scope="col">单元评价因子 F′</th>                                  6868
          <th scope="col">加权贡献</th>                                         6869
          <th scope="col">权重来源</th>                                         6870
        </tr>                                                                   6871
      </thead>                                                                  6872
      <tbody>                                                                   6873
        {% for u in unit_scores %}                                              6874
        <tr>                                                                    6875
          <td>{{ u.unit_type }}</td>                                            6876
          <td class="num">{{ "%.2f%%"|format(u.ratio * 100)                     6877
      }}</td>                                                                   6878
          <td class="mono">{{ u.unit_score }}</td>                              6879
          <td class="mono">{{ "%.4f"|format(u.ratio *                           6880
      u.unit_score) }}</td>                                                     6881
          <td class="muted">{{ u.weight_source or "论文表 3                     6882
      默认权重" }}</td>                                                         6883
        </tr>                                                                   6884
        {% endfor %}                                                            6885
      </tbody>                                                                  6886
    </table>                                                                    6887
    {% endif %}                                                                 6888
    <p class="actions" style="margin-top:18px;">                                6889
      <a class="btn btn--ghost" href="/report/{{ r.patch_id                     6890
      }}">查看得分明细</a>                                                      6891
      <a class="btn btn--ghost" href="/evaluate/">返回评价列表</a>              6892
    </p>                                                                        6893
</div>                                                                          6894
{% endblock %}                                                                  6895
# ==== app/templates/report/index.html ====                                     6896
{% extends "base.html" %}                                                       6897
{% from "_macros.html" import patch_guide %}                                    6898
{% block title %}报告输出{% endblock %}                                         6899
{% block content %}                                                             6900
```

```
<div class="card">
  <h2>报告与结果输出</h2>
  <p class="muted">
    可查看逐层得分明细与结果图表，并导出 PDF 或 Word 评价报告。
    报告是评价结论的呈现，因此仅在评价所依赖的前置步骤全部完成后才可生成。
  </p>
  {{ patch_guide(projects_without_patch) }}
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先到「项目与矿山档案」建立图斑。</p>
  {% else %}
  <table>
    <thead>
      <tr>
        <th scope="col">所属项目</th>
        <th scope="col">矿山图斑</th>
        <th scope="col">可生成报告</th>
        <th scope="col">操作</th>
      </tr>
    </thead>
    <tbody>
      {% for p in patches %}
      {% set pr = progress.get(p.id) %}
      <tr>
        <td>{{ p.project.name if p.project else "—" }}</td>
        <td style="text-align:left;">{{ p.name }}</td>
        <td style="text-align:left;">
          {% if pr and pr.ready %}
          <span class="tag tag--ok">可以</span>
          {% elif pr %}
          <span class="tag tag--warn">暂不可</span>
          <span class="muted">{{ pr.blocking[0].reason
    }}</span>
          {% endif %}
        </td>
        <td class="nowrap">
          {% if pr and pr.ready %}
          <a href="/report/{{ p.id }}">查看得分明细</a>
           | <a href="/report/{{ p.id }}/pdf">导出
    PDF</a>
           | <a href="/report/{{ p.id }}/word">导出
    Word</a>
          {% else %}
          <span class="muted">请先完成前置步骤</span>
          {% endif %}
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% endif %}
```

```
</div>
{% endblock %}
# ==== app/templates/report/detail.html ====
{% extends "base.html" %}
{% block title %}得分明细 · {{ patch.name }}{% endblock %}
{% block content %}
<div class="card">
  <h2>得分明细 · {{ patch.name }}</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">最终评价因子 F</div>
      <div class="stat__value mono">{{
    "%.4f"|format(result.final_score) }}</div>
    </div>
    <div>
      <div class="stat__label">适宜性等级</div>
      <div class="stat__value ok">{{ result.grade }}</div>
    </div>
    <div style="flex:1;min-width:240px;">
      <div class="stat__label">推荐措施</div>
      <div style="font-size:15px;">{{
    result.grade_info.measure }}</div>
    </div>
  </div>
  <p class="muted" style="margin-top:12px;">
    本页与导出报告均取自
    <strong>{% if record %}{{ record.created_at.strftime("%Y
    -%m-%d %H:%M:%S") }}{% endif %}
    记录的那次评价结果</strong>（评价记录编号 {{ record.id if record else
    "—" }}），
    不会因事后修改指标数据而变化。
  </p>
  <p class="actions" style="margin-top:16px;">
    <a class="btn" href="/report/{{ patch.id }}/pdf">导出 PDF
    报告</a>
    <a class="btn btn--ghost" href="/report/{{ patch.id
    }}/word">导出 Word 报告</a>
    <a class="btn btn--ghost" href="/report/">返回</a>
  </p>
</div>
<div class="card">
  <h2>结果图表</h2>
  <h3>各微地质单元得分对比</h3>
  <div>{{ chart_units|safe }}</div>
  <h3>各一级指标平均得分</h3>
  <p class="muted">反映各一级指标对评价结果的实际影响，便于识别关键控制项。</p>
  <div>{{ chart_weight|safe }}</div>
</div>
{% for u in result.unit_results %}
<div class="card">
```

```
  <h2>{{ u.unit_type }} · 占比 {{ "%.2f%%"|format(u.ratio *
    100) }} · 单元得分 {{ "%.4f"|format(u.unit_score) }}</h2>
  {% if u.weight_source %}
  <p class="muted" style="margin-top:-6px;">权重来源：{{
    u.weight_source }}</p>
  {% endif %}
  <table>
    <thead>
      <tr>
        <th scope="col">一级指标</th>
        <th scope="col">二级指标</th>
        <th scope="col">实测值</th>
        <th scope="col">标准化值</th>
        <th scope="col">权重</th>
        <th scope="col">贡献</th>
      </tr>
    </thead>
    <tbody>
      {% for d in u.details %}
      <tr>
        <td>{{ d.l1 }}</td>
        <td style="text-align:left;">{{ d.l2 }}</td>
        <td class="mono" style="text-align:left;">{{ "—" if
    d.raw is none else d.raw }}</td>
        <td class="mono">{{ "—" if d.std_value is none else
    "%.4f"|format(d.std_value) }}</td>
        <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
        <td class="mono">{{ "—" if d.contribution is none
    else "%.4f"|format(d.contribution) }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  <p class="muted" style="margin:0;">
    一级指标得分：
    {% for k, v in u.l1_scores.items() %}{{ k }} = <span
    class="num">{{ "%.4f"|format(v) }}</span>{% if not
    loop.last %};{% endif %}{% endfor %}
  </p>
</div>
{% endfor %}
{% endblock %}
# ==== app/templates/report/print.html ====
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<title>自然恢复适宜性评价报告 · {{ patch.name }}</title>
<style>
  @page { size: A4; margin: 18mm 16mm; }
```

```
  * { box-sizing: border-box; }
  body {
    margin: 0; color: #23262b; background: #fff;
    font-family: "Microsoft YaHei", "SimSun", sans-serif;
    font-size: 10.5pt; line-height: 1.65;
  }
  h1 { font-size: 17pt; text-align: center; margin: 0 0
    6px; }
  h2 { font-size: 13pt; color: #1f3a5f; margin: 20px 0 8px;
       border-bottom: 1px solid #dfe3e8; padding-bottom:
    4px; }
  h3 { font-size: 11.5pt; color: #1f3a5f; margin: 16px 0
    6px; }
  .sub { text-align: center; color: #6b7280; font-size:
    9.5pt; margin-bottom: 18px; }
  table { width: 100%; border-collapse: collapse; margin:
    8px 0 14px;
          font-variant-numeric: tabular-nums; }
  th, td { border: 1px solid #c9ccd1; padding: 4px 8px;
    text-align: center; }
  th { background: #eef2f7; font-weight: 600; }
  .left { text-align: left; }
  .mono { font-family: Consolas, "Courier New", monospace; }
  .kv { width: auto; }
  .kv th { width: 30%; text-align: left; }
  .kv td { text-align: left; }
  ul { margin: 6px 0 12px; padding-left: 20px; }
  .note { color: #6b7280; font-size: 9.5pt; }
  .chart { margin: 10px 0 16px; }
  .sig { margin-top: 28px; font-size: 10pt; }
  .avoid-break { break-inside: avoid; }
  svg { max-width: 100%; }
</style>
</head>
<body>
  <h1>历史遗留废弃矿山自然恢复适宜性评价报告</h1>
  <p class="sub">
    {{ patch.name }}
    {% if patch.admin_region %} · {{ patch.admin_region
    }}{% endif %}
    <br>历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ１.０ ·
    生成时间 {{ generated_at }}
  </p>
  <h2>一、图斑基本信息</h2>
  <table class="kv">
    <tr><th>图斑名称</th><td>{{ patch.name }}</td></tr>
    <tr><th>所在行政区</th><td>{{ patch.admin_region or "—"
    }}</td></tr>
    <tr><th>图斑面积（公顷）</th><td>{{ patch.area_ha if
    patch.area_ha is not none else "—" }}</td></tr>
```

```
    <tr><th>开采矿种</th><td>{{ patch.mineral_type or "—"                    7101
    }}</td></tr>                                                            7102
    <tr><th>开采方式</th><td>{{ patch.mining_method or "—"                   7103
    }}</td></tr>                                                            7104
    <tr><th>责任主体缺失</th><td>{{ "是" if patch.responsible_entit          7105
    y_missing else "否" }}</td></tr>                                        7106
    {% if reference %}                                                      7107
    <tr><th>参照生态系统</th>                                               7108
        <td>植被覆盖度基准 {{ reference.coverage_benchmark }}%,             7109
            多样性指数基准 {{ reference.shannon_benchmark                   7110
    }}</td></tr>                                                            7111
    {% endif %}                                                             7112
</table>                                                                    7113
<h2>二、评价结论</h2>                                                       7114
<table class="kv">                                                          7115
    <tr><th>最终评价因子 F</th><td class="mono"><strong>{{                  7116
    "%.4f"|format(result.final_score) }}</strong></td></tr>                 7117
    <tr><th>适宜性等级</th><td><strong>{{ result.grade                      7118
    }}</strong></td></tr>                                                   7119
    <tr><th>判定依据</th><td>{{ result.grade_info.meaning                   7120
    }}</td></tr>                                                            7121
    <tr><th>推荐措施</th><td>{{ result.grade_info.measure                   7122
    }}</td></tr>                                                            7123
</table>                                                                    7124
<h2>三、各微地质单元得分</h2>                                               7125
<table>                                                                     7126
    <thead>                                                                 7127
      <tr><th>微地质单元</th><th>面积占比</th><th>单元评价因子               7128
    F'</th><th>加权贡献</th></tr>                                           7129
    </thead>                                                                7130
    <tbody>                                                                 7131
      {% for u in result.unit_results %}                                    7132
      <tr>                                                                  7133
        <td>{{ u.unit_type }}</td>                                          7134
        <td class="mono">{{ "%.2f%%"|format(u.ratio * 100)                  7135
    }}</td>                                                                 7136
        <td class="mono">{{ "%.4f"|format(u.unit_score)                     7137
    }}</td>                                                                 7138
        <td class="mono">{{ "%.4f"|format(u.weighted)                       7139
    }}</td>                                                                 7140
      </tr>                                                                 7141
      {% endfor %}                                                          7142
    </tbody>                                                                7143
</table>                                                                    7144
<div class="chart">{{ chart_units|safe }}</div>                             7145
<h2>四、一级指标权重分布</h2>                                               7146
<div class="chart">{{ chart_weight|safe }}</div>                            7147
<h2>五、得分明细</h2>                                                       7148
{% for u in result.unit_results %}                                          7149
<h3>{{ u.unit_type }}（占比 {{ "%.2f%%"|format(u.ratio *                    7150
```

```
      100) }}，单元得分 {{ "%.4f"|format(u.unit_score) }}）</h3>
    <table>
      <thead>
        <tr>
          <th>一级指标</th><th>二级指标</th><th>实测值</th>
          <th>标准化值</th><th>权重</th><th>贡献</th>
        </tr>
      </thead>
      <tbody>
        {% for d in u.details %}
        <tr>
          <td>{{ d.l1 }}</td>
          <td class="left">{{ d.l2 }}</td>
          <td class="mono left">{{ "—" if d.raw is none else
      d.raw }}</td>
          <td class="mono">{{ "—" if d.std_value is none else
      "%.4f"|format(d.std_value) }}</td>
          <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
          <td class="mono">{{ "—" if d.contribution is none
      else "%.4f"|format(d.contribution) }}</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    <p class="note">
      一级指标得分：
      {% for k, v in u.l1_scores.items() %}{{ k }} = {{
      "%.4f"|format(v) }}{% if not loop.last %}；{% endif %}{%
      endfor %}
    </p>
    {% endfor %}
    <h2>六、评价依据说明</h2>
    <ul class="note">
      <li>指标标准化按论文式（8） 执行，按指标性质分为越大越优、越小越优、
          适中型与离散赋值四类。</li>
      <li>单元评价因子按论文式（9） 由二级指标权重加权求得；
          整矿评价因子由各微地质单元按面积占比加权合成。</li>
      <li>适宜性等级按论文表 4 划分：F&lt;0.3 不适宜、0.3≤F≤0.7 基本适宜、F≥0.7
      适宜。</li>
      <li>指标权重取自论文表 3，按岩质边坡、碎石边坡、平台三类微地质单元分别确定。</li>
    </ul>
    <p class="sig">
      评价人（签字）：＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿　　日期：＿＿＿＿＿＿＿＿＿＿＿＿
    </p>
  </body>
</html>
# ==== seed_demo.py ====
# -*- coding: utf-8 -*-
"""内置玉田县案例：把论文第 5 节案例数据写入数据库。
用于功能验证与操作演示。数据逐字取自论文表 5 与第 5 节。
```

```
运行：python seed_demo.py                    幂等，只补写缺失的演示数据，
                                                 不会删除用户自建的项目与图斑
      python seed_demo.py --reset            连同标准库与指标定义一并重建
"""
import json
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
from app.models import (db, User, Project, MinePatch,
    ReferenceEcosystem,
                        MicroUnit, IndicatorData, Standard,
    StandardRule,
                        ConstraintCheck, EvaluationResult,
    ScoreDetail)
app = create_app()
#  内置演示案例的项目编号，用于区分「演示数据」与「用户自建数据」
DEMO_CODE = "DEMO-001"
#  论文表 1：11 项地方标准与政策文件
STANDARDS = [
    dict(category="地方标准", province="辽宁",
    name="矿山废弃地植被自然恢复技术规程",
        doc_number="DB21/T 3702—2023",
    publish_date="2023-03-30",
        implement_date="2023-04-30",
        summary="首个正式发布的矿山自然恢复地方标准；提出必要条件和充分条件",
        rules=[("植被覆盖度", ">=", "50", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
    dict(category="地方标准", province="湖北",
    name="废弃露天矿山生态环境自然恢复标准",
        doc_number="DB42/T 2421—2025",
    publish_date="2025-08-20",
        implement_date="2025-10-20",
        summary="结合矿山生态需求进行自然恢复适宜性分级判断",
        rules=[("自然恢复适宜性分级", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="内蒙古",
    name="历史遗留废弃矿山自然恢复判定规范",
        doc_number="DB15/T 4280—2026",
    publish_date="2026-01-30",
        implement_date="2026-03-01",
        summary="调查内容包含参照生态系统调查",
        rules=[("地质灾害发育程度", "<=", "弱发育", "必要条件", None),
               ("地形地貌与周边协调", "==", "是", "必要条件", None),
               ("有利于植被恢复的表层土壤", "==", "是", "必要条件", None),
               ("已形成稳定植被覆盖", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="江西",
    name="废弃矿山自然恢复技术规范",
        doc_number="DB36/T 2221—2026",
    publish_date="2026-02-02",
```

```
        implement_date="2026-08-01",                                    7251
        summary="对参照生态系统进行了要求；提出自然恢复适宜性指数",                    7252
        rules=[("参照生态系统调查", "==", "是", "必要条件", None),              7253
               ("自然恢复适宜性指数", ">=", "阈值", "充分条件", None)]),        7254
    dict(category="地方标准", province="宁夏", name="矿山自然恢复认定规程",      7255
        doc_number=None, publish_date="2026-06-18",                     7256
    implement_date=None,                                                7257
        summary="分区差异化：三个一级生态修复分区及 22 个二级分区",              7258
        rules=[("无地质灾害风险", "==", "是", "必要条件", None),               7259
               ("植被覆盖度趋势", "==", "稳定或正增长", "必要条件", None),       7260
               ("与参照系差距", "<", "20", "充分条件", "%")]),                 7261
    dict(category="政策文件", province="安徽",                              7262
        name="安徽省自然资源厅关于规范废弃露天矿山采取自然恢复方式进行治理的通知",    7263
        doc_number="皖自然资修函[2020]34号",                               7264
    publish_date="2020-03-03", implement_date=None,                     7265
        summary="自然恢复条件：责任主体为政府、无地质灾害、无水土污染、不在三线三边、水土条件良好",7266
        rules=[("责任主体", "==", "政府", "必要条件", None),                   7267
               ("无地质灾害", "==", "是", "必要条件", None),                  7268
               ("无水土污染", "==", "是", "必要条件", None)]),                7269
    dict(category="政策文件", province="山东",                              7270
        name="山东省自然资源厅关于进一步规范历史遗留废弃露天开采矿山采取自然恢复方式进行治理的通知",7271
        doc_number="鲁自然资字[2022]104号",                                7272
    publish_date="2022-08-02", implement_date=None,                     7273
        summary="条件与安徽省相同，另增加自然恢复认定标准",                      7274
        rules=[("无地质灾害", "==", "是", "必要条件", None),                   7275
               ("无水土污染", "==", "是", "必要条件", None)]),                7276
    dict(category="政策文件", province="湖南",                              7277
        name="关于印发《湖南省历史遗留矿山自然恢复指导意见（试行）》的通知",          7278
        doc_number="湘自资办发[2023]58号",                                 7279
    publish_date="2023-07-20", implement_date=None,                     7280
        summary="自然恢复认定标准：无地质灾害或无威胁对象、无水土污染、设备拆除、符合规划、植被覆盖度7281
    >75%、连续3年正向增长",                                                 7282
        rules=[("植被覆盖度", ">", "75", "必要条件", "%"),                     7283
               ("植被覆盖率趋势", "==", "连续3年正向增长", "必要条件", None),7284
               ("无地质灾害", "==", "是", "必要条件", None),                  7285
               ("符合国土空间规划", "==", "是", "必要条件", None)]),          7286
    dict(category="政策文件", province="新疆",                              7287
        name="自治区历史遗留废弃矿山自然恢复方式销号图斑工作指南（试行）",            7288
        doc_number="新自然资源办函〔2024〕219号",                           7289
    publish_date="2024-12-20", implement_date=None,                     7290
        summary="荒漠区沙地/裸土地/裸岩石砾地满足无地质灾害、地形协调、无水土污染即可判定",7291
        rules=[("植被覆盖度", ">=", "周边30%", "必要条件", None),              7292
               ("无地质灾害", "==", "是", "必要条件", None),                  7293
               ("地形平缓地貌稳定", "==", "是", "必要条件", None)]),          7294
    dict(category="政策文件", province="广西",                              7295
        name="广西壮族自治区历史遗留矿山自然恢复技术指南",                      7296
        doc_number="桂自然资办〔2024〕250号",                               7297
    publish_date="2024-12-30", implement_date=None,                     7298
        summary="除高陡边坡和裸露岩体外，水土水热条件良好、植被自然恢复趋势明显",      7299
        rules=[("无地质灾害", "==", "是", "必要条件", None),                   7300
```

```
                    ("植被自然恢复趋势", "==", "明显", "必要条件", None),
                    ("生产设备拆除完毕", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="西藏",
         name="西藏自治区历史遗留废弃矿山生态自然恢复认定指导意见（试行）",
         doc_number="藏自然资发〔2025〕19号",
    publish_date="2025-09-24", implement_date=None,
         summary="含修复成本高、生态修复效益不显著等适用条件",
         rules=[("责任主体", "==", "政府", "必要条件", None),
                    ("地形地貌完整与周边协调", "==", "是", "必要条件", None),
                    ("无地质灾害", "==", "是", "必要条件", None)]),
]
CASE = {
    "name": "玉田县典型历史遗留废弃矿山案例",
    "patch_name": "玉田县孤树镇白云岩采石场",
    "area_ha": 5.6,
    "mineral_type": "白云岩",
    "units": [
        {
            "unit_type": "岩质边坡", "ratio": 0.1386,
            "data": {"coverage": 68.88, "shannon": 0.00,
                     "slope": 63.33, "aspect": "阴阳坡"},
        },
        {
            "unit_type": "碎石边坡", "ratio": 0.6295,
            "data": {"soil_ratio": 0.34, "thickness": 10,
    "ph": 9.24,
                     "organic_matter": 4.89,
                     "heavy_metals": {"Hg": 0.05, "As":
    5.17, "Ni": 12.60,
                                      "Cu": 11.70, "Cd":
    0.04, "Pb": 6.50},
                     "coverage": 87.92, "shannon": 0.60,
                     "slope": 28.75, "aspect": "阳坡"},
        },
        {
            "unit_type": "平台", "ratio": 0.2319,
            "data": {"soil_ratio": 0.56, "thickness": 30,
    "ph": 8.72,
                     "organic_matter": 10.92,
                     "heavy_metals": {"Hg": 0.02, "As":
    3.13, "Ni": 12.33,
                                      "Cu": 11.14, "Cd":
    0.09, "Pb": 9.98},
                     "coverage": 58.42, "shannon": 0.72},
        },
    ],
}
def clean_case(reset=False):
    """清理数据。
    默认只清理内置演示案例（code = DEMO-001）自身的数据，**不触碰
```

```
用户自己建立的项目与图斑**。加 --reset 时才额外清空标准库与指标定义，
以便把演示环境恢复到初始状态。
注意：不要在此处执行 Project.query.delete()，否则用户新建的项目
会在每次启动时被删除。
"""
from app.models import EvaluationResult, ScoreDetail,
ConstraintCheck
demo_projects = Project.query.filter_by(code=DEMO_CODE).
all()
demo_ids = [p.id for p in demo_projects]
patch_ids = []
if demo_ids:
    patch_ids = [p.id for p in MinePatch.query.filter(
        MinePatch.project_id.in_(demo_ids)).all()]
if patch_ids:
    unit_ids = [u.id for u in MicroUnit.query.filter(
        MicroUnit.patch_id.in_(patch_ids)).all()]
    if unit_ids:
        IndicatorData.query.filter(
            IndicatorData.unit_id.in_(unit_ids)).delete(
            synchronize_session=False)
    result_ids = [r.id for r in EvaluationResult.query.f
ilter(
        EvaluationResult.patch_id.in_(patch_ids)).all()]
    if result_ids:
        ScoreDetail.query.filter(
            ScoreDetail.result_id.in_(result_ids)).delet
e(
            synchronize_session=False)
    EvaluationResult.query.filter(
        EvaluationResult.patch_id.in_(patch_ids)).delete
(
        synchronize_session=False)
    ConstraintCheck.query.filter(
        ConstraintCheck.patch_id.in_(patch_ids)).delete(
        synchronize_session=False)
    ReferenceEcosystem.query.filter(
        ReferenceEcosystem.patch_id.in_(patch_ids)).dele
te(
        synchronize_session=False)
    MicroUnit.query.filter(MicroUnit.patch_id.in_(patch_
ids)).delete(
        synchronize_session=False)
    MinePatch.query.filter(MinePatch.id.in_(patch_ids)).
delete(
        synchronize_session=False)
if demo_ids:
    Project.query.filter(Project.id.in_(demo_ids)).delet
e(
        synchronize_session=False)
```

```
    if reset:
        StandardRule.query.delete()
        Standard.query.delete()
    # 兜底：清理指向已不存在评价记录的孤儿得分明细，
    # 避免长期运行后 score_details 表堆积无效数据。
    _purge_orphan_details()
    db.session.commit()
    return len(demo_ids)
def demo_patch():
    """取内置演示案例的图斑。
    库中可能存在用户自建项目或验证算例（编号 VERIFY-），
    故不能简单用 MinePatch.query.first()，否则取到的对象不确定。
    没有演示案例时才退回第一个图斑。
    """
    proj = Project.query.filter_by(code=DEMO_CODE).first()
    if proj is not None:
        m = MinePatch.query.filter_by(project_id=proj.id).first()
        if m is not None:
            return m
    return MinePatch.query.first()
def _purge_orphan_details():
    """清理指向已不存在评价记录的孤儿得分明细。
    长期运行后 score_details 表可能堆积无效明细，启动时顺带清一次。
    """
    live_ids = [r.id for r in EvaluationResult.query.all()]
    if live_ids:
        ScoreDetail.query.filter(~ScoreDetail.result_id.in_(
    live_ids)).delete(
            synchronize_session=False)
    else:
        ScoreDetail.query.delete()
    db.session.commit()
def seed(reset=False):
    with app.app_context():
        db.create_all()          # 全新环境下先建表，保证本脚本可独立运行
        _purge_orphan_details()
        admin = User.query.filter_by(username="admin").first
    ()
        if not admin:
            from werkzeug.security import
    generate_password_hash
            admin = User(username="admin",
                         password_hash=generate_password_has
    h("admin123"),
                         role="admin")
            db.session.add(admin)
            db.session.commit()
        # 幂等：演示案例已存在就不重复写入，更不清空用户自己的项目
        existing = Project.query.filter_by(code=DEMO_CODE).f
```

```
irst()                                                                  7451
    if existing and not reset:                                          7452
        print("演示案例已存在，跳过写入（用户自建项目不受影响）")                  7453
        _report()                                                       7454
        _seed_reference_data(reset=False)                               7455
        return                                                          7456
    clean_case(reset=reset)                                             7457
    project = Project(name=CASE["name"], code=DEMO_CODE,                7458
                      description="论文第 5 节典型案例",                  7459
created_by=admin.id)                                                    7460
    db.session.add(project)                                             7461
    db.session.flush()                                                  7462
    patch = MinePatch(project_id=project.id,                            7463
name=CASE["patch_name"],                                                7464
                      admin_region="河北省唐山市玉田县",                  7465
                      area_ha=CASE["area_ha"],                          7466
                      mineral_type=CASE["mineral_type"],                7467
                      mining_method="露天开采",                         7468
stop_years=None,                                                        7469
                      responsible_entity_missing=True)                  7470
    db.session.add(patch)                                               7471
    db.session.flush()                                                  7472
    ref = ReferenceEcosystem(patch_id=patch.id,                         7473
                             coverage_benchmark=75.0,                   7474
shannon_benchmark=0.7)                                                  7475
    db.session.add(ref)                                                 7476
    for u in CASE["units"]:                                             7477
        unit = MicroUnit(patch_id=patch.id,                             7478
unit_type=u["unit_type"],                                               7479
                         ratio=u["ratio"], area=None)                   7480
        db.session.add(unit)                                            7481
        db.session.flush()                                              7482
        d = u["data"]                                                   7483
        ind = IndicatorData(                                            7484
            unit_id=unit.id,                                            7485
            soil_ratio=d.get("soil_ratio"),                             7486
            thickness=d.get("thickness"),                               7487
            ph=d.get("ph"),                                             7488
            organic_matter=d.get("organic_matter"),                     7489
            heavy_metals=json.dumps(d["heavy_metals"],                  7490
ensure_ascii=False)                                                     7491
                               if d.get("heavy_metals") else            7492
None,                                                                   7493
            coverage=d.get("coverage"),                                 7494
            shannon=d.get("shannon"),                                   7495
            slope=d.get("slope"),                                       7496
            aspect=d.get("aspect"),                                     7497
        )                                                               7498
        db.session.add(ind)                                             7499
    #  限制性条件判定记录：演示案例按论文第 5 节描述属「无地质灾害隐患点」、          7500
```

```
            #  具备自然恢复条件，故五项全部通过。缺此记录会使后续评价被流程门禁拦住。
            db.session.add(ConstraintCheck(
                patch_id=patch.id,
                policy_pass=True, location_pass=True,
        pollution_pass=True,
                geo_pass=True, farmland_pass=True,
                passed=True, reason="全部通过"))
            db.session.commit()
            print("已写入案例：", CASE["name"])
            print("   图斑：", CASE["patch_name"])
            print("   微地质单元：岩质边坡 13.86% / 碎石边坡 62.95% / 平台
        23.19%")
            print("   管理员账号：admin / admin123")
        _seed_reference_data(reset=reset)
        _report()
    def _seed_reference_data(reset=False):
        """写入标准库与指标定义。
        参考资料只在库为空（或显式 --reset）时写入，避免覆盖用户自行维护
        的标准条目与指标定义。
        """
        if reset or Standard.query.count() == 0:
            seed_standards()
        from app.services.indicator_service import
        seed_indicators
        n = seed_indicators()
        print("指标定义：%d 项" % n)
    def _report():
        total_p = Project.query.count()
        total_patch = MinePatch.query.count()
        print()
        print("当前库内：项目 %d 个、图斑 %d 个" % (total_p, total_patch))
        if total_p > total_patch:
            print("   提示：项目是归档单位，评价针对的是图斑。")
            print("         新项目需先建立图斑，才会出现在后续各模块中。")
    def seed_standards():
        """写入论文表 1 的 11 项标准与政策文件及其结构化规则。"""
        StandardRule.query.delete()
        Standard.query.delete()
        db.session.commit()
        for s in STANDARDS:
            std = Standard(category=s["category"],
        province=s["province"],
                           name=s["name"],
        doc_number=s["doc_number"],
                           publish_date=s["publish_date"],
                           implement_date=s["implement_date"],
                           summary=s["summary"])
            db.session.add(std)
            db.session.flush()
            for item, op, thr, ctype, note in s["rules"]:
```

```
                db.session.add(StandardRule(standard_id=std.id,
    item=item,
                                                operator=op,
    threshold=thr,

    condition_type=ctype, note=note))
    db.session.commit()
    print("已写入标准/政策文件：%d 项，规则 %d 条"
          % (len(STANDARDS), sum(len(s["rules"]) for s in
    STANDARDS)))
if __name__ == "__main__":
    # --reset：把演示环境恢复到初始状态（清空标准库与指标定义后重建）。
    # 无论是否 --reset，都不会删除用户自建的项目与图斑。
    seed(reset="--reset" in sys.argv[1:])
# ==== 测试/test_ahp.py ====
# -*- coding: utf-8 -*-
"""地基自检：验证 AHP 引擎与应用可启动。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import numpy as np
import os
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app.services import ahp_service as ahp
LINE = "=" * 66
# ------------------------------------------------------------
    ------------------
# 1) 完全一致矩阵 → 权重应精确还原，CR=0
# ------------------------------------------------------------
    ------------------
print(LINE)
print("测试 1  完全一致判断矩阵的权重还原与一致性")
print(LINE)
# 由权重构造完全一致矩阵：a_ij = w_i / w_j
w_true = np.array([0.44, 0.31, 0.25])
A = np.outer(w_true, 1.0 / w_true)
r = ahp.weights_and_consistency(A)
print("  判断矩阵 A：")
for row in A:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print()
print("  计算权重 :", "  ".join(f"{v:.4f}" for v in
    r["weights"]))
print("  真实权重 :", "  ".join(f"{v:.4f}" for v in w_true))
print("  λmax = %.6f   CI = %.3e   CR = %.3e   一致: %s"
      % (r["lambda_max"], r["ci"], r["cr"],
```

```
    r["consistent"]))                                                    7601
assert np.allclose(r["weights"], w_true, atol=1e-6),                     7602
    "权重未精确还原"                                                      7603
assert r["consistent"], "完全一致矩阵应通过检验"                          7604
# ------------------------------------------------------------------     7605
    ----------------                                                     7606
# 2） 上三角输入 → 完整矩阵 → 一致性定位                                  7607
# ------------------------------------------------------------------     7608
    ----------------                                                     7609
print()                                                                  7610
print(LINE)                                                              7611
print("测试 2  上三角录入与最不一致项定位")                               7612
print(LINE)                                                              7613
# 一个不太一致的 3 阶矩阵：上三角 [2, 5, 3]（a12=2, a13=5, a23=3）       7614
M = ahp.matrix_from_upper([2, 5, 3])                                     7615
r2 = ahp.weights_and_consistency(M)                                      7616
bad = ahp.find_most_inconsistent(M)                                      7617
print("  由 [a12=2, a13=5, a23=3] 构造矩阵：")                           7618
for row in M:                                                            7619
    print("    " + "  ".join(f"{v:8.4f}" for v in row))                  7620
print("  权重：", "  ".join(f"{v:.4f}" for v in r2["weights"]),          7621
      "  CR = %.3f  一致: %s" % (r2["cr"], r2["consistent"]))            7622
print("  最不一致比较项: 第(%d,%d) 项，当前 %.3f，理想 %.3f"              7623
      % (bad["i"] + 1, bad["j"] + 1, bad["value"],                       7624
    bad["expected"]))                                                    7625
# ------------------------------------------------------------------     7626
    ----------------                                                     7627
# 3） 多专家聚合（几何平均）                                             7628
# ------------------------------------------------------------------     7629
    ----------------                                                     7630
print()                                                                  7631
print(LINE)                                                              7632
print("测试 3  多专家判断矩阵聚合（几何平均）")                           7633
print(LINE)                                                              7634
m1 = ahp.matrix_from_upper([2, 4, 3])                                    7635
m2 = ahp.matrix_from_upper([3, 5, 2])                                    7636
agg = ahp.aggregate_matrices([m1, m2])                                   7637
print("  专家1 上三角 [2,4,3]，专家2 上三角 [3,5,2]")                   7638
print("  聚合后上三角: a12=%.4f a13=%.4f a23=%.4f"                       7639
      % (agg[0, 1], agg[0, 2], agg[1, 2]))                               7640
assert abs(agg[0, 1] - np.sqrt(2 * 3)) < 1e-9, "几何平均错误"            7641
# ------------------------------------------------------------------     7642
    ----------------                                                     7643
# 4） 论文表 3 权重可还原性                                              7644
# ------------------------------------------------------------------     7645
    ----------------                                                     7646
print()                                                                  7647
print(LINE)                                                              7648
print("测试 4  论文表 3 碎石边坡一级权重 (0.44, 0.31, 0.25)")            7649
print(LINE)                                                              7650
```

```
j = ahp.judge_from_weights([0.44, 0.31, 0.25])
print("  λmax = %.6f    CR = %.3e    一致: %s" %
    (j["lambda_max"], j["cr"], j["consistent"]))
assert j["consistent"], "论文权重应能作为完全一致矩阵主特征向量"
# ------------------------------------------------------------------
    ----------------
# 5) 应用可启动 + 建表
# ------------------------------------------------------------------
    ----------------
print()
print(LINE)
print("测试 5  Flask 应用启动与建表")
print(LINE)
from app import create_app
from app.models import db
app = create_app()
with app.app_context():
    db.create_all()
    tables = sorted(db.metadata.tables.keys())
print("  已创建 %d 张表:" % len(tables))
for t in tables:
    print("    -", t)
print()
print(LINE)
print("全部测试通过 √")
print(LINE)
# ==== 测试/test_evaluate.py ====
# -*- coding: utf-8 -*-
"""评价引擎自检:用论文玉田县案例跑通完整计算链。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import os
# 本脚本位于 测试/ 子目录,先把仓库根目录加入模块搜索路径,
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file
    __)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app.services import evaluation_service as ev
from app.services import standardization as std
LINE = "=" * 70
# ------------------------------------------------------------------
    ----------------
# 论文第 5 节案例数据(表 5,逐字抄录)
# ------------------------------------------------------------------
    ----------------
# 三类单元面积占比:岩质边坡 13.86%、碎石边坡 62.95%、平台 23.19%
# 注:岩质边坡无土壤指标、平台无地形指标,对应"不适用"。
case = [
    {
```

```
            "unit_type": "岩质边坡",
            "ratio": 0.1386,
            "data": {
                "coverage": 68.88, "shannon": 0.00,
                "slope": 63.33, "aspect": "阴阳坡",
            },
        },
        {
            "unit_type": "碎石边坡",
            "ratio": 0.6295,
            "data": {
                "soil_ratio": 0.34, "thickness": 10, "ph":
        9.24, "organic_matter": 4.89,
                "heavy_metals": {"Hg": 0.05, "As": 5.17, "Ni":
        12.60,
                                 "Cu": 11.70, "Cd": 0.04, "Pb":
        6.50},
                "coverage": 87.92, "shannon": 0.60,
                "slope": 28.75, "aspect": "阳坡",
            },
        },
        {
            "unit_type": "平台",
            "ratio": 0.2319,
            "data": {
                "soil_ratio": 0.56, "thickness": 30, "ph":
        8.72, "organic_matter": 10.92,
                "heavy_metals": {"Hg": 0.02, "As": 3.13, "Ni":
        12.33,
                                 "Cu": 11.14, "Cd": 0.09, "Pb":
        9.98},
                "coverage": 58.42, "shannon": 0.72,
            },
        },
]
# ------------------------------------------------------------------
    ------------------
# 逐单元输出标准化与得分
# ------------------------------------------------------------------
    ------------------
print(LINE)
print("玉田县案例 · 评价引擎完整计算")
print(LINE)
for u in case:
    r = ev.score_unit(u["unit_type"], u["data"])
    print()
    print("  【%s】  (面积占比 %.2f%%)" % (u["unit_type"],
    u["ratio"] * 100))
    print("  " + "-" * 60)
    for l1, score in r["l1_scores"].items():
```

```
            print("      %s 得分 = %.4f" % (l1, score))
        print("      单元评价因子 F' = %.4f" % r["unit_score"])
print()
print(LINE)
print("面积加权合成（式(9)）")
print(LINE)
res = ev.score_mine(case)
total = res["final_score"]
print()
for ur in res["unit_results"]:
        print("   %s   %.2f%% × %.4f = %.4f"
              % (ur["unit_type"], ur["ratio"] * 100,
        ur["unit_score"], ur["weighted"]))
print("   " + "-" * 60)
print("   最终评价因子 F = %.4f" % total)
print()
print("   适宜性等级：%s" % res["grade"])
print("   判定依据   ：%s" % res["grade_info"]["meaning"])
print("   推荐措施   ：%s" % res["grade_info"]["measure"])
print()
print(LINE)
print("结论")
print(LINE)
print("   - 评价引擎按论文式(8)(9) 与表 3 权重完整跑通")
print("   - 坡度标准化采用论文 25° 分界的线性折减默认值（见 standardization.py）")
print("   - 最终 F ≈ %.2f，判定为「%s」，与论文「最终适宜自然恢复」的结论一致"
        % (total, res["grade"]))
print("   - 论文表 5 所载最终得分为 0.76；差异集中在坡度、坡向的标准化口径，")
print("     该口径在 standardization.std_slope / aspect_score
     中集中定义、可配置")
print(LINE)
#  ==== 测试/test_web_ahp.py ====
#  -*- coding: utf-8 -*-
"""AHP 网页界面自检：走通选择→录入→计算 全流程。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import os
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file
    __)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
#  测试配置：关闭 CSRF 以简化测试客户端（真实运行仍开启 CSRF）
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    SQLALCHEMY_TRACK_MODIFICATIONS = False
```

```
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 建立表并创建测试用户，登录后再访问业务页面
with app.app_context():
    from app.models import db, User
    from werkzeug.security import generate_password_hash
    db.create_all()
    if not User.query.filter_by(username="admin").first():
        db.session.add(User(username="admin",
                            password_hash=generate_password_
    hash("admin123"),
                            role="admin"))
        db.session.commit()
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  首页与 AHP 入口")
print(LINE)
r = client.get("/")
print("  首页状态码:", r.status_code, "(应 200)")
assert r.status_code == 200
r = client.get("/ahp/")
print("  AHP 入口状态码:", r.status_code, "(应 200)")
assert r.status_code == 200
assert "微地质单元类型" in r.get_data(as_text=True)
print()
print(LINE)
print("测试 2  选择碎石边坡一级指标 → 跳转矩阵页")
print(LINE)
r = client.post("/ahp/", data={
    "unit_type": "碎石边坡", "level": "1", "submit": "生成判断矩阵",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转后页面含「判断矩阵录入」:", "判断矩阵录入" in body)
print("  含三个一级指标(土壤/植被/地形):", all(n in body for n in
    ("土壤指标", "植被指标", "地形指标")))
assert "判断矩阵录入" in body
assert all(n in body for n in ("土壤指标", "植被指标", "地形指标"))
print()
print(LINE)
print("测试 3  提交论文表 3 一致矩阵 → 还原权重")
print(LINE)
# 碎石边坡一级权重 (0.44, 0.31, 0.25) 对应的完全一致矩阵上三角:
# a12 = 0.44/0.31 = 1.4194, a13 = 0.44/0.25 = 1.76, a23 =
    0.31/0.25 = 1.24
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24",
}, follow_redirects=True)
```

```
body = r.get_data(as_text=True)
print("  结果页含「计算结果」:", "计算结果" in body)
print("  含权重 0.44:", "0.44" in body, " 含 0.31:", "0.31" in
    body, " 含 0.25:", "0.25" in body)
print("  含一致性检验通过:", "通过" in body)
assert "计算结果" in body
assert "0.44" in body and "0.31" in body and "0.25" in body
print()
print(LINE)
print("测试 4  不一致矩阵应提示不通过并定位")
print(LINE)
# 明显不一致：a12=9, a13=1, a23=9
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "9", "m_0_2": "1", "m_1_2": "9",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  含「不通过」:", "不通过" in body)
print("  含「建议优先调整」:", "建议优先调整" in body)
assert "不通过" in body
assert "建议优先调整" in body
print()
print(LINE)
print("网页界面全部测试通过 √")
print(LINE)
# ==== 测试/test_web_modules.py ====
# -*- coding: utf-8 -*-
"""模块自检：登录、项目档案、微地质单元、标准库、限制性判定。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil
    e__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
print(LINE)
print("测试 1  未登录应被重定向到登录页")
print(LINE)
```

```
r = client.get("/project/")
print("  /project/ 状态码:", r.status_code, " 重定向到:",
      r.headers.get("Location"))
assert r.status_code == 302 and "login" in
      r.headers.get("Location", "")
print()
print(LINE)
print("测试 2  登录")
print(LINE)
r = client.post("/login", data={"username": "admin",
      "password": "admin123"},
                  follow_redirects=True)
body = r.get_data(as_text=True)
print("  登录后含「系统功能」:", "系统功能" in body)
print("  含用户名 admin:", "admin" in body)
assert "系统功能" in body
print()
print(LINE)
print("测试 3  项目与矿山档案")
print(LINE)
r = client.get("/project/")
body = r.get_data(as_text=True)
print("  项目列表状态码:", r.status_code)
print("  含案例项目:", "玉田县典型历史遗留废弃矿山案例" in body)
assert "玉田县典型历史遗留废弃矿山案例" in body
# 新建项目
client.post("/project/add", data={"name": "测试项目A", "code":
      "T-001"})
r = client.get("/project/")
print("  新建项目后含「测试项目A」:", "测试项目A" in r.get_data(as_text=True
      ))
assert "测试项目A" in r.get_data(as_text=True)
# 清理测试数据，避免污染演示库
with app.app_context():
      from app.models import db, Project
      for p in Project.query.filter_by(name="测试项目A").all():
          db.session.delete(p)
      db.session.commit()
print("  已清理测试项目")
print()
print(LINE)
print("测试 4  微地质单元（面积占比自动计算）")
print(LINE)
from app.models import MinePatch
with app.app_context():
      patch = MinePatch.query.filter_by(name="玉田县孤树镇白云岩采石场").f
      irst()
      pid = patch.id
r = client.get("/unit/%d" % pid)
body = r.get_data(as_text=True)
```

```
print("   单元列表含三类单元:", all(t in body for t in ("岩质边坡",
    "碎石边坡", "平台")))
print("   含占比合计:", "占比合计" in body)
assert "岩质边坡" in body and "占比合计" in body
print()
print(LINE)
print("测试 5   标准知识库与省际口径比对")
print(LINE)
r = client.get("/standard/")
body = r.get_data(as_text=True)
print("   含辽宁/湖南/新疆标准:", all(p in body for p in ("辽宁", "湖南",
    "新疆")))
assert "辽宁" in body
r = client.get("/standard/compare")
body = r.get_data(as_text=True)
print("   口径比对页含「植被覆盖度」分组:", "植被覆盖度" in body)
print("   含多省份并列:", "湖南" in body and "新疆" in body)
assert "植被覆盖度" in body
print()
print(LINE)
print("测试 6   限制性指标判定（一票否决）")
print(LINE)
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("   含五项条件:", all(x in body for x in ("政策约束", "区位约束",
    "土壤污染", "地质灾害", "耕地属性")))
assert all(x in body for x in ("政策约束", "区位约束", "土壤污染",
    "地质灾害", "耕地属性"))
# 全部通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("   全通过后显示「通过，可进入定量评价」:", "可进入定量评价" in body)
assert "可进入定量评价" in body
# 一项不通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "0",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("   一项不通过后显示「不通过」:", "不通过" in body)
print("   含失败项「区位约束」:", "区位约束" in body)
assert "不通过" in body
# 恢复为全部通过，避免把演示数据留在「不通过」状态影响后续步骤
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
```

```
        "pollution_pass": "1",                                                  8001
        "geo_pass": "1", "farmland_pass": "1"})                                 8002
r = client.get("/constraint/%d" % pid)                                          8003
print("   已恢复为全部通过：", "可进入定量评价" in r.get_data(as_text=True))          8004
print()                                                                         8005
print(LINE)                                                                     8006
print("全部测试通过 √")                                                          8007
print(LINE)                                                                     8008
#  ==== 测试/test_web_evaluate.py ====                                          8009
#  -*- coding: utf-8 -*-                                                        8010
"""端到端自检：录数据 → 执行评价 → 查看结果。"""                                    8011
import os                                                                       8012
import sys                                                                      8013
sys.stdout.reconfigure(encoding="utf-8")                                        8014
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，                          8015
#  使 `from app import ...` 与 app/app.db 的定位不受影响。                       8016
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__fil                    8017
    e__)))                                                                      8018
if ROOT not in sys.path:                                                        8019
    sys.path.insert(0, ROOT)                                                    8020
from app import create_app                                                      8021
LINE = "=" * 66                                                                 8022
#  与真实配置共用同一个 app.db，但关闭 CSRF 便于测试客户端                        8023
class TestConfig:                                                               8024
    SECRET_KEY = "test"                                                         8025
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(                      8026
        ROOT, "app", "app.db")                                                  8027
    SQLALCHEMY_TRACK_MODIFICATIONS = False                                      8028
    WTF_CSRF_ENABLED = False                                                    8029
    TESTING = True                                                              8030
app = create_app(TestConfig)                                                    8031
client = app.test_client()                                                      8032
#  登录（案例种子已创建 admin 用户）                                             8033
client.post("/login", data={"username": "admin",                                8034
    "password": "admin123"})                                                    8035
print(LINE)                                                                     8036
print("测试 1  指标数据列表")                                                    8037
print(LINE)                                                                     8038
r = client.get("/indicator/")                                                   8039
body = r.get_data(as_text=True)                                                 8040
print("   状态码：", r.status_code)                                             8041
print("   含案例图斑：", "玉田县孤树镇白云岩采石场" in body)                      8042
print("   含三个单元：", all(n in body for n in ("岩质边坡", "碎石边坡",          8043
    "平台")))                                                                   8044
assert r.status_code == 200                                                     8045
assert "岩质边坡" in body and "碎石边坡" in body and "平台" in body               8046
print()                                                                         8047
print(LINE)                                                                     8048
print("测试 2  指标数据录入页（按单元类型显示字段）")                            8049
print(LINE)                                                                     8050
```

```
from app.models import MicroUnit, MinePatch
with app.app_context():
    unit_rock = MicroUnit.query.filter_by(unit_type="岩质边坡").first()
    unit_gravel = MicroUnit.query.filter_by(unit_type="碎石边坡").first()
r = client.get("/indicator/%d" % unit_rock.id)
b_rock = r.get_data(as_text=True)
print("  岩质边坡页：无「土石比」字段：", "土石比" not in b_rock, "（应 True）")
print("  岩质边坡页：含「植被覆盖度」：", "植被覆盖度" in b_rock)
assert "土石比" not in b_rock and "植被覆盖度" in b_rock
r = client.get("/indicator/%d" % unit_gravel.id)
b_gravel = r.get_data(as_text=True)
print("  碎石边坡页：含「土石比」字段：", "土石比" in b_gravel)
print("  碎石边坡页：含 8 种重金属：", all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn")))
assert "土石比" in b_gravel and all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn"))
print()
print(LINE)
print("测试 3  执行评价")
print(LINE)
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
r = client.post("/evaluate/%d" % patch.id, data={},
    follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转至结果页：", "适宜性评价结果" in body)
print("  含最终评价因子 F：", "最终评价因子 F" in body)
print("  含等级「适宜」：", "适宜" in body)
print("  含推荐措施「自然恢复」：", "自然恢复" in body)
assert "最终评价因子 F" in body and "自然恢复" in body
print()
print(LINE)
print("测试 4  结果持久化")
print(LINE)
from app.models import EvaluationResult
with app.app_context():
    res = EvaluationResult.query.order_by(EvaluationResult.id.desc()).first()
    print("  最终得分 F =", res.final_score)
    print("  等级 =", res.grade)
    assert res.grade == "适宜"
    assert 0.7 <= res.final_score < 1.0
print()
print(LINE)
print("端到端全部测试通过 √")
print(LINE)
# ==== 测试/test_web_report.py ====
```

```
# -*- coding: utf-8 -*-
"""报告输出自检：得分明细页 + Word 导出。"""
import os
import sys
import io
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
from app.models import MinePatch, EvaluationResult
with app.app_context():
    import seed_demo
    patch = seed_demo.demo_patch()
    pid = patch.id
# 报告取自已保存的评价记录，因此先按网页路径真实执行一次评价
from app.views.evaluate import load_patch_input,
    save_evaluation
with app.app_context():
    units = load_patch_input(pid)
    rec = save_evaluation(pid, units)
    rid = rec.id
    print("前置：已执行评价并保存记录 id=%d，F=%.4f（%s）"
          % (rid, rec.final_score, rec.grade))
    from app.models import ScoreDetail
    n_detail = ScoreDetail.query.filter_by(result_id=rid).co
    unt()
    print("        逐层得分明细已落库：%d 条" % n_detail)
print()
print(LINE)
print("测试 1  报告列表")
print(LINE)
r = client.get("/report/")
print("  状态码：", r.status_code)
```

```
body = r.get_data(as_text=True)
print("  含「所属项目」列：", "所属项目" in body)
assert r.status_code == 200
print()
print(LINE)
print("测试 2  得分明细页")
print(LINE)
r = client.get("/report/%d" % pid)
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含「得分明细」：", "得分明细" in body)
print("  含「最终评价因子 F」：", "最终评价因子 F" in body)
print("  含二级指标明细（植被覆盖度/坡度/坡向）：", all(n in body for n in
    ("植被覆盖度", "坡度", "坡向")))
print("  含权重/标准化值列：", "标准化值" in body and "贡献" in body)
assert r.status_code == 200
assert "植被覆盖度" in body and "标准化值" in body
print()
print(LINE)
print("测试 3  Word 报告导出")
print(LINE)
r = client.get("/report/%d/word" % pid)
print("  状态码：", r.status_code)
print("  Content-Type:", r.headers.get("Content-Type",
    "")[:60])
data = r.data
print("  文件大小：", len(data), "字节")
print("  是否为有效 docx（PK 头）：", data[:2] == b"PK")
assert r.status_code == 200 and data[:2] == b"PK"
# 用 python-docx 打开验证内容
import docx
d = docx.Document(io.BytesIO(data))
text = "\n".join(p.text for p in d.paragraphs)
print("  含报告标题：", "自然恢复适宜性评价报告" in text)
print("  含评价结论章节：", "评价结论" in text)
print("  含得分明细章节：", "得分明细" in text)
print("  段落数：", len(d.paragraphs), " 表格数：", len(d.tables))
assert "自然恢复适宜性评价报告" in text and len(d.tables) >= 3
print()
print(LINE)
print("报告输出全部测试通过 √")
print(LINE)
# ==== 测试/test_ui_fixes.py ====
# -*- coding: utf-8 -*-
"""界面修复自检：「记住我」与各模块删除功能。"""
import os
import re
import sys
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
```

```
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
sys.stdout.reconfigure(encoding="utf-8")
LINE = "=" * 70
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(ROOT, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    REMEMBER_COOKIE_DURATION = __import__("datetime").timedelta(days=30)
app = create_app(TestConfig)
client = app.test_client()
# ------------------------------------------------------------------------
print(LINE)
print("一、「记住我」")
print(LINE)
html = client.get("/login").get_data(as_text=True)
# 1) 复选框必须有可点击关联的 label（这是原先失效的根因）
wrapped = re.search(r'<label[^>]*class="checkbox"[^>]*>\s*<input[^>]*name="remember"',
                    html) is not None
print("  %s 「记住我」文字与复选框包在同一 label 内" % ("[符合]" if wrapped else "[不符合]"))
assert wrapped, "复选框未与文字关联，点击文字不会勾选"
has_for = 'for="remember"' in html
print("  %s label 带 for 属性指向复选框" % ("[符合]" if has_for else "[不符合]"))
# 2) 勾选后应下发带过期时间的 remember_token
c = app.test_client()
r = c.post("/login", data={"username": "admin", "password": "admin123",
                           "remember": "y"},
    follow_redirects=False)
cookies = r.headers.getlist("Set-Cookie")
rt = [h for h in cookies if h.startswith("remember_token=")]
print("  %s 勾选后下发 remember_token" % ("[符合]" if rt else "[不符合]"))
assert rt, "未下发 remember_token"
exp = re.search(r"Expires=([^;]+)", rt[0])
print("  %s 该 cookie 带过期时间：%s"
      % ("[符合]" if exp else "[不符合]", exp.group(1) if exp else "无（会话 cookie）"))
assert exp, "remember_token 无过期时间，关浏览器即失效"
```

```
# 3) 不勾选时不应下发 remember_token
c2 = app.test_client()
r2 = c2.post("/login", data={"username": "admin",
    "password": "admin123"},
                follow_redirects=False)
rt2 = [h for h in r2.headers.getlist("Set-Cookie") if
    h.startswith("remember_token=")]
print("  %s 未勾选时不下发 remember_token" % ("[符合]" if not rt2
    else "[不符合]"))
assert not rt2
# 4) 携带 remember_token 的新会话应自动登录
c3 = app.test_client()
c3.set_cookie("remember_token",
              rt[0].split(";")[0].split("=", 1)[1],
    domain="localhost")
r3 = c3.get("/", follow_redirects=False)
print("  %s 仅凭 remember_token 访问首页可自动登录（状态码 %d）"
      % ("[符合]" if r3.status_code == 200 else "[不符合]",
    r3.status_code))
assert r3.status_code == 200
# ------------------------------------------------------------
    ----------------
print()
print(LINE)
print("二、删除功能")
print(LINE)
client.post("/login", data={"username": "admin",
    "password": "admin123"})
# 项目：列表页须有删除按钮
html = client.get("/project/").get_data(as_text=True)
has_del = "/delete" in html and "btn--danger" in html
print("  %s 项目列表含删除按钮" % ("[符合]" if has_del else "[不符合]"))
assert has_del, "项目列表缺少删除按钮"
print("  %s 删除前有二次确认" % ("[符合]" if "confirm(" in html else
    "[不符合]"))
assert "confirm(" in html
# 图斑列表
with app.app_context():
    from app.models import MinePatch, Project
    import seed_demo
    patch = seed_demo.demo_patch()
    patch_id = patch.id
html = client.get("/project/%d/patches" %
    patch_id).get_data(as_text=True)
print("  %s 图斑列表含删除按钮" % ("[符合]" if "/project/patch/" in
    html and "btn--danger" in html else "[不符合]"))
assert "btn--danger" in html
# 微地质单元
html = client.get("/unit/%d" % patch_id).get_data(as_text=Tr
    ue)
```

```
print("  %s 微单元列表含删除按钮" % ("[符合]" if "/unit/delete/" in
    html else "[不符合]"))
assert "/unit/delete/" in html
print("  %s 微单元编辑为独立页面（表单不再嵌在表格行内）"
      % ("[符合]" if "/unit/edit/" in html else "[不符合]"))
# 标准
html = client.get("/standard/").get_data(as_text=True)
print("  %s 标准列表含删除按钮" % ("[符合]" if "btn--danger" in html
    else "[不符合]"))
assert "btn--danger" in html
# 实际执行一次删除：新建项目 → 删除 → 确认消失
client.post("/project/add", data={"name": "待删除测试项目",
    "code": "DEL-1"})
with app.app_context():
    from app.models import db, Project
    tmp = Project.query.filter_by(name="待删除测试项目").first()
    tmp_id = tmp.id
html = client.get("/project/").get_data(as_text=True)
print("  %s 新建项目成功" % ("[符合]" if "待删除测试项目" in html else
    "[不符合]"))
r = client.post("/project/%d/delete" % tmp_id,
    follow_redirects=True)
html = r.get_data(as_text=True)
gone = "待删除测试项目" not in html
print("  %s 删除项目后列表中不再出现" % ("[符合]" if gone else "[不符合]"))
assert gone
print("  %s 有删除成功提示" % ("[符合]" if "已删除" in html else
    "[不符合]"))
print()
print(LINE)
print("界面修复自检全部通过 √")
print(LINE)
# ==== 测试/test_new_features.py ====
# -*- coding: utf-8 -*-
"""补齐模块自检：操作日志与用户维护、遥感影像、指标与多专家矩阵、
批量导入、图表与 PDF 导出。
对应计划书功能清单与系统实现的差异核对结论。
"""
import io
import json
import os
import re
import sys
sys.stdout.reconfigure(encoding="utf-8")
import fitz
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file
    __)))
if ROOT not in sys.path:
```

```
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, User, Project, MinePatch,
    MicroUnit, IndicatorData,
                            Indicator, ExpertMatrix,
    OperationLog)
from app.utils.schema import ensure_schema
from app.utils import browser, charts
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
    UPLOAD_FOLDER = os.path.join(BASE, "app", "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
app = create_app(TestConfig)
with app.app_context():
    db.create_all()
    ensure_schema(db)
    ExpertMatrix.query.delete()
    db.session.commit()
    # 固定取内置演示案例的图斑，避免受库中用户自建项目 / 验证算例的影响
    import seed_demo
    patch_id = seed_demo.demo_patch().id
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
def ok(flag, label, extra=""):
    print("  %s %-44s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
    assert flag, label
# ----------------------------------------------------------------
    ----------------
print(LINE)
print("操作日志与用户维护")
print(LINE)
before = None
with app.app_context():
    before = OperationLog.query.count()
c.post("/project/add", data={"name": "日志核对项目", "code":
    "LOG-T"})
with app.app_context():
    after = OperationLog.query.count()
```

```
        last = OperationLog.query.order_by(OperationLog.id.desc(8401
        )).first()                                                  8402
ok(after == before + 1, "写操作被写入操作日志", "%d -> %d" % (before, 8403
        after))                                                     8404
ok(last.action == "新建项目", "日志记录了可读动作名", last.action)          8405
ok("project" in (last.target_type or ""), "日志记录了对象类型",          8406
        last.target_type)                                           8407
r = c.get("/logs")                                                  8408
body = r.get_data(as_text=True)                                     8409
ok(r.status_code == 200 and "操作日志" in body, "操作日志查看页可访问")     8410
ok("新建项目" in body, "日志页显示操作记录")                               8411
# 用户编辑 / 删除                                                    8412
with app.app_context():                                             8413
        u = User(username="tmp_user", role="viewer",                8414
                 password_hash=User.query.first().password_hash)8415
        db.session.add(u); db.session.commit()                      8416
        uid = u.id                                                  8417
r = c.get("/users/%d/edit" % uid)                                   8418
ok(r.status_code == 200, "用户编辑页可访问")                              8419
c.post("/users/%d/edit" % uid, data={"role": "evaluator",            8420
        "active": "1"})                                             8421
with app.app_context():                                             8422
        ok(db.session.get(User, uid).role == "evaluator",           8423
        "角色已修改并保存")                                              8424
# 最后一个管理员保护                                                 8425
with app.app_context():                                             8426
        admin_id = User.query.filter_by(role="admin").first().id8427
c.post("/users/%d/edit" % admin_id, data={"role": "viewer",          8428
        "active": "1"})                                             8429
with app.app_context():                                             8430
        ok(db.session.get(User, admin_id).role == "admin",          8431
        "拒绝把最后一个管理员降级")                                       8432
c.post("/users/%d/delete" % uid, follow_redirects=True)             8433
with app.app_context():                                             8434
        ok(db.session.get(User, uid) is None, "用户删除生效")          8435
# 清理日志核对用的项目                                               8436
with app.app_context():                                             8437
        for p in Project.query.filter_by(name="日志核对项目").all():    8438
            db.session.delete(p)                                    8439
        db.session.commit()                                         8440
# ------------------------------------------------------------------8441
        ------------------                                          8442
print()                                                             8443
print(LINE)                                                         8444
print("遥感影像挂接")                                                  8445
print(LINE)                                                         8446
doc = fitz.open()                                                   8447
pg = doc.new_page(width=320, height=200)                            8448
pg.insert_text((20, 60), "RS image", fontname="china-s",            8449
        fontsize=16)                                                8450
```

```
png = doc[0].get_pixmap(dpi=100).tobytes("png")                    8451
doc.close()                                                        8452
r = c.post("/project/patch/%d/image" % patch_id,                   8453
                data={"image": (io.BytesIO(png), "影像.png"),      8454
                      "image_note": "2025年6月无人机正射影像"},    8455
                content_type="multipart/form-data",                8456
      follow_redirects=True)                                       8457
body = r.get_data(as_text=True)                                    8458
ok("已挂接" in body and "<img" in body, "影像上传并显示")          8459
with app.app_context():                                            8460
      rel = db.session.get(MinePatch, patch_id).image_path         8461
ok(bool(rel), "数据库记录影像路径", rel)                           8462
abs_path = os.path.join(TestConfig.UPLOAD_FOLDER,                  8463
      rel.replace("uploads/", "", 1))                              8464
ok(os.path.isfile(abs_path), "影像文件落盘")                       8465
ok(c.get("/static/" + rel).status_code == 200, "影像可经 URL       8466
      访问")                                                       8467
r = c.post("/project/patch/%d/image" % patch_id,                   8468
                data={"image": (io.BytesIO(b"x"), "bad.txt")},     8469
                content_type="multipart/form-data",                8470
      follow_redirects=True)                                       8471
ok("不支持" in r.get_data(as_text=True), "拒绝非图片格式")         8472
c.post("/project/patch/%d/image/delete" % patch_id,                8473
      follow_redirects=True)                                       8474
with app.app_context():                                            8475
      ok(db.session.get(MinePatch, patch_id).image_path is         8476
      None, "影像移除后字段清空")                                  8477
ok(not os.path.isfile(abs_path), "影像文件已删除")                 8478
# ------------------------------------------------------------------8479
      ------------------                                           8480
print()                                                            8481
print(LINE)                                                        8482
print("指标定义维护与多专家矩阵")                                  8483
print(LINE)                                                        8484
r = c.get("/ahp/indicators")                                       8485
body = r.get_data(as_text=True)                                    8486
with app.app_context():                                            8487
      n_ind = Indicator.query.count()                              8488
ok(r.status_code == 200 and n_ind > 0, "指标定义已入库并可查看", "%d 8489
      项" % n_ind)                                                 8490
ok(all(k in body for k in ("一级指标", "二级指标", "三级指标")),   8491
    "指标树按层级展示")                                            8492
ok("岩质边坡" in body and "平台" in body, "显示各指标启用的微地质单元") 8493
c.post("/ahp/indicators/coverage/edit",                            8494
          data={"name": "植被覆盖度", "unit": "%", "ideal_type":   8495
      "benefit",                                                   8496
                "ideal_value": "75", "enabled_units": ["岩质边坡"],8497
                "description": "自检修改"})                        8498
with app.app_context():                                            8499
      ok(Indicator.query.filter_by(code="coverage").first().de     8500
```

```
        scription == "自检修改",                                                    8501
            "指标定义可编辑")                                                       8502
c.post("/ahp/indicators/reset")                                                     8503
with app.app_context():                                                             8504
        ok("自检修改" not in (Indicator.query.filter_by(code="covera                8505
        ge").first().description or ""),                                            8506
            "可按论文取值重置")                                                     8507
base = {"m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24"}                        8508
for name in ("张工", "李工", "王工"):                                               8509
        d = dict(base); d.update({"action": "add", "unit_type":                     8510
        "碎石边坡",                                                                 8511
                                  "level": "1", "expert_name":                      8512
        name})                                                                      8513
        c.post("/ahp/experts", data=d)                                              8514
with app.app_context():                                                             8515
        ok(ExpertMatrix.query.filter_by(unit_type="碎石边坡",                       8516
        level=1).count() == 3,                                                      8517
            "录入 3 位专家的判断矩阵")                                              8518
r = c.get("/ahp/experts/aggregate?unit_type=碎石边坡&level=1")                      8519
body = r.get_data(as_text=True)                                                     8520
ok("汇总后权重" in body and "各专家矩阵的一致性" in body,                          8521
    "多专家矩阵几何平均汇总可用")                                                   8522
ws = re.search(r"<th scope=\"row\">权重</th>\s*(.*?)</tr>",                         8523
        body, re.S)                                                                 8524
vals = re.findall(r">([0-9.]+)</td>", ws.group(1)) if ws                            8525
        else []                                                                     8526
ok(len(vals) == 3 and abs(sum(map(float, vals)) - 1.0) <                            8527
        0.01,                                                                       8528
    "汇总权重归一", "、".join(vals))                                                8529
ok(abs(float(vals[0]) - 0.44) < 0.02, "汇总权重接近论文表 3 取值",                  8530
        vals[0] if vals else "")                                                    8531
# ------------------------------------------------------------------------------    8532
        ------------------                                                          8533
print()                                                                             8534
print(LINE)                                                                         8535
print("批量导入")                                                                   8536
print(LINE)                                                                         8537
r = c.get("/indicator/import/template")                                             8538
tpl = r.data.decode("utf-8-sig")                                                    8539
ok(r.status_code == 200 and "单元类型" in tpl, "导入模板可下载")                    8540
ok("土石比" in tpl and "植被覆盖度" in tpl, "模板包含指标列")                       8541
csv_text = (                                                                        8542
        "单元类型,土石比,土层厚度,pH,有机质,Hg,As,Ni,Cu,Cd,Pb,植被覆盖度,Shannon,坡度8543
        ,坡向\n"                                                                    8544
        "碎石边坡,0.34,10,9.24,4.89,0.05,5.17,12.60,11.70,0.04,6.50,               8545
        87.92,0.60,28.75,阳坡\n"                                                    8546
        "平台,0.56,30,8.72,10.92,0.02,3.13,12.33,11.14,0.09,9.98,5                  8547
        8.42,0.72,,,\n"                                                             8548
        "不存在单元,1,1,7,1,,,,,,,,,,\n"                                           8549
)                                                                                   8550
```

```
r = c.post("/indicator/import",                                    8551
            data={"patch_id": str(patch_id),                       8552
                  "datafile": (io.BytesIO(csv_text.encode("ut      8553
    f-8")), "d.csv")},                                             8554
            content_type="multipart/form-data",                    8555
    follow_redirects=True)                                         8556
body = r.get_data(as_text=True)                                    8557
ok("导入结果" in body, "导入结果页展示")                                8558
ok("不存在单元" in body, "错误行被报告")                                8559
with app.app_context():                                            8560
    u = MicroUnit.query.filter_by(patch_id=patch_id,               8561
    unit_type="碎石边坡").first()                                     8562
    ind = IndicatorData.query.filter_by(unit_id=u.id).first(       8563
    )                                                              8564
    ok(abs(ind.soil_ratio - 0.34) < 1e-6 and                       8565
    abs(ind.coverage - 87.92) < 1e-6,                              8566
        "导入数据已写入数据库")                                          8567
r = c.post("/indicator/import",                                    8568
            data={"patch_id": str(patch_id),                       8569
                  "datafile": (io.BytesIO(b"x"), "d.pdf")},        8570
            content_type="multipart/form-data",                    8571
    follow_redirects=True)                                         8572
ok("仅支持" in r.get_data(as_text=True), "拒绝不支持的导入格式")          8573
# ------------------------------------------------------------------8574
    ------------------                                             8575
print()                                                            8576
print(LINE)                                                        8577
print("图表与 PDF 导出")                                             8578
print(LINE)                                                        8579
svg = charts.hbars([("岩质边坡", 0.54, "0.5401"), ("平台", 0.84,      8580
    "0.8376")])                                                    8581
ok("<svg" in svg and "0.8376" in svg, "条形图生成且带数值标签")          8582
svg2 = charts.donut([("植被指标", 0.6, "0.6000"), ("土壤指标", 0.4,    8583
    "0.4000")])                                                    8584
ok("<svg" in svg2 and "占比" in svg2, "环形图生成且带图例")              8585
# 报告是评价结论的呈现，需先真实执行一次评价才能打开                          8586
with app.app_context():                                            8587
    from app.views.evaluate import load_patch_input,               8588
    save_evaluation                                                8589
    save_evaluation(patch_id, load_patch_input(patch_id))          8590
r = c.get("/report/%d" % patch_id)                                 8591
body = r.get_data(as_text=True)                                    8592
ok(body.count("<svg") >= 2, "得分明细页含图表", "%d 个 SVG" %          8593
    body.count("<svg"))                                            8594
exe = browser.browser_path()                                       8595
if exe is None:                                                    8596
    print("  [跳过] 未找到浏览器，PDF 导出无法验证")                       8597
else:                                                              8598
    r = c.get("/report/%d/pdf" % patch_id)                         8599
    data = r.data                                                  8600
```

```
    ok(data[:4] == b"%PDF", "PDF 报告导出成功", "%d 字节" %
    len(data))
    d = fitz.open(stream=data, filetype="pdf")
    txt = "".join(d[i].get_text() for i in
    range(d.page_count))
    ok("自然恢复适宜性评价报告" in txt, "PDF 含报告标题")
    ok("评价结论" in txt and "得分明细" in txt, "PDF 含结论与明细章节")
    ok(d.page_count >= 2, "PDF 页数合理", "%d 页" % d.page_count)
    d.close()
print()
print(LINE)
print("补齐模块自检全部通过 √")
print(LINE)
#  ==== 测试/test_workflow.py ====
#  -*- coding: utf-8 -*-
"""流程门禁自检：前置步骤未完成时，后续模块必须拦得住。
论文 4.1 节确定的评价流程存在先后关系，缺任何一环都不应产出评价结论。
本测试逐个构造「缺前置」的场景，验证系统不会放行。
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
#  本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
#  使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file
    __)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult)
from app.services import workflow as wf
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
    label, extra))
```

```
    if not flag:
        _fails.append(label)
def new_patch(tag):
    with app.app_context():
        p = Project(name="门禁-" + tag, code="WF-" + tag)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="门禁图斑-" +
    tag)
        db.session.add(patch); db.session.flush()
        pid = patch.id
        db.session.commit()
    return pid
def add_units(pid):
    with app.app_context():
        db.session.add(MicroUnit(patch_id=pid,
    unit_type="碎石边坡",
                                 area=1.0, ratio=1.0))
        db.session.commit()
def add_constraint(pid, passed=True):
    with app.app_context():
        db.session.add(ConstraintCheck(
            patch_id=pid, policy_pass=True,
    location_pass=passed,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=passed, reason="全部通过" if passed else
    "未通过：区位约束"))
        db.session.commit()
def add_data(pid):
    with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=pid).first()
        db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.34, thickness=10,
                                     ph=9.24,
    organic_matter=4.89, coverage=87.92,
                                     shannon=0.60,
    slope=28.75, aspect="阳坡"))
        db.session.commit()
def cleanup(pid, tag):
    with app.app_context():
        from app.models import ScoreDetail
        # 先删得分明细，再删评价记录，避免留下孤儿明细
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
```

```
            ConstraintCheck.query.filter_by(patch_id=pid).delete
        ()
            uids = [u.id for u in MicroUnit.query.filter_by(patc
        h_id=pid)]
            if uids:
                IndicatorData.query.filter(IndicatorData.unit_id
        .in_(uids)).delete(
                    synchronize_session=False)
            MicroUnit.query.filter_by(patch_id=pid).delete()
            m = db.session.get(MinePatch, pid)
            if m:
                db.session.delete(m)
            pr = Project.query.filter_by(name="门禁-" +
        tag).first()
            if pr:
                db.session.delete(pr)
            db.session.commit()
    def results_of(pid):
        with app.app_context():
            return EvaluationResult.query.filter_by(patch_id=pid
        ).count()
    def post_eval(pid):
        return c.post("/evaluate/%d" % pid, data={},
        follow_redirects=False)
    def get_report(pid):
        return c.get("/report/%d" % pid, follow_redirects=False)
    # ------------------------------------------------------------
        ------------------
    print(LINE)
    print("一、前置条件缺失时，评价必须被拦住")
    print(LINE)
    print()
    print("场景 1：连微地质单元都没划分")
    pid = new_patch("1")
    r = post_eval(pid)
    n = results_of(pid)
    ok(r.status_code == 302 and n == 0, "未划分单元 → 不产生评价结果",
       "HTTP %d，结果数 %d" % (r.status_code, n))
    with app.app_context():
        pr = wf.patch_progress(pid)
    steps = [b["step"] for b in pr["blocking"]]
    ok(not pr["ready"] and "units" in steps,
       "拦原因包含「划分微地质单元」", "、".join(steps))
    ok(steps == ["constraint", "units"],
       "拦截顺序符合论文 4.1 节：限制性判定在前，划分单元在后",
       "、".join(steps))
    cleanup(pid, "1")
    print()
    print("场景 2：有单元，但未做限制性条件判定")
    pid = new_patch("2"); add_units(pid); add_data(pid)
```

```
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "未做限制性判定 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "constraint" for b in pr["blocking"]),
   "拦原因指向「执行限制性条件判定」")
cleanup(pid, "2")
print()
print("场景 3：限制性条件判定【未通过】")
pid = new_patch("3"); add_units(pid); add_data(pid);
    add_constraint(pid, passed=False)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "判定未通过 → 不进入定量评价",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(not pr["constraint"]["passed"], "流程状态显示判定未通过")
blocked = [b for b in pr["blocking"] if b["step"] ==
    "constraint"]
ok(bool(blocked) and "人工辅助" in blocked[0]["reason"],
   "拦原因说明了按论文应判「人工辅助或生态重建」")
cleanup(pid, "3")
print()
print("场景 4：判定通过，但指标数据全空")
pid = new_patch("4"); add_units(pid); add_constraint(pid,
    passed=True)
r = post_eval(pid)
n = results_of(pid)
ok(r.status_code == 302 and n == 0, "数据未录入 → 不产生评价结果",
   "结果数 %d" % n)
with app.app_context():
    pr = wf.patch_progress(pid)
ok(any(b["step"] == "data" for b in pr["blocking"]),
   "拦原因指向「录入指标数据」")
ok(pr["data"]["missing"] == ["碎石边坡"],
   "指出具体缺哪个单元的数据", "、".join(pr["data"]["missing"]))
cleanup(pid, "4")
print()
print("场景 5：单元面积占比合计不为 100%")
pid = new_patch("5"); add_constraint(pid, passed=True)
with app.app_context():
    u = MicroUnit(patch_id=pid, unit_type="碎石边坡", area=1.0,
    ratio=0.5)
    db.session.add(u); db.session.commit()
    db.session.add(IndicatorData(unit_id=u.id,
    soil_ratio=0.3, thickness=10,
                                 ph=8.0,
```

```
        organic_matter=5.0, coverage=80.0,                                  8801
                                              shannon=0.6, slope=20.0,      8802
        aspect="阳坡"))                                                     8803
        db.session.commit()                                                 8804
r = post_eval(pid)                                                          8805
ok(results_of(pid) == 0, "占比不足 100% → 不产生评价结果")                  8806
cleanup(pid, "5")                                                           8807
print()                                                                     8808
print(LINE)                                                                 8809
print("二、前置条件齐备时才放行")                                          8810
print(LINE)                                                                 8811
pid = new_patch("OK"); add_units(pid); add_constraint(pid,                  8812
        passed=True); add_data(pid)                                         8813
with app.app_context():                                                     8814
        pr = wf.patch_progress(pid)                                         8815
ok(pr["ready"], "单元 + 判定通过 + 数据齐备 → 判定为可执行")                8816
r = post_eval(pid)                                                          8817
n = results_of(pid)                                                         8818
ok(n == 1, "此时执行评价成功产生结果", "结果数 %d" % n)                     8819
print("   重定向到:", r.headers.get("Location"))                           8820
print()                                                                     8821
print(LINE)                                                                 8822
print("三、报告不得先于评价产生")                                          8823
print(LINE)                                                                 8824
pid2 = new_patch("R")     # 什么都不做                                      8825
r = get_report(pid2)                                                        8826
ok(r.status_code == 302 and "/report/" ==                                   8827
        (r.headers.get("Location") or ""),                                  8828
    "前置缺失时 /report/<id> 被退回列表", "HTTP %d" % r.status_code)        8829
for suffix in ("/pdf", "/word"):                                            8830
        rr = c.get("/report/%d%s" % (pid2, suffix),                         8831
        follow_redirects=False)                                             8832
        ok(rr.status_code == 302, "导出 %s 同样被拦住" % suffix, "HTTP      8833
        %d" % rr.status_code)                                               8834
cleanup(pid2, "R")                                                          8835
cleanup(pid, "OK")                                                          8836
print()                                                                     8837
print(LINE)                                                                 8838
print("四、前置齐备但从未执行评价时，报告仍不得产生")                      8839
print(LINE)                                                                 8840
#  报告是评价结论的呈现：即使前置步骤都做完了，只要没真正执行过定量评价，  8841
#  也不应凭空推算出结论。                                                   8842
pid3 = new_patch("N")                                                       8843
add_units(pid3); add_constraint(pid3, passed=True);                         8844
        add_data(pid3)                                                      8845
with app.app_context():                                                     8846
        pr = wf.report_progress(pid3)                                       8847
ok(not pr["ready"], "无评价记录 → 报告流程判定为未就绪")                    8848
ok(bool(pr["blocking"]) and pr["blocking"][0]["step"] ==                    8849
        "evaluate",                                                         8850
```

```
    "拦原因指向「执行适宜性评价」")                                          8851
r = get_report(pid3)                                                      8852
ok(r.status_code == 302 and results_of(pid3) == 0,                        8853
    "直接访问 /report/<id> 被退回，且不产生任何评价结果",                  8854
    "HTTP %d" % r.status_code)                                            8855
#  真正执行评价之后，报告才放行，且取值与评价记录一致                     8856
r = post_eval(pid3)                                                       8857
ok(results_of(pid3) == 1, "先执行评价 → 产生评价记录")                    8858
with app.app_context():                                                   8859
    pr = wf.report_progress(pid3)                                         8860
    rec = pr["result"]                                                    8861
ok(pr["ready"] and rec is not None, "有评价记录后，报告流程就绪")         8862
rr = get_report(pid3)                                                     8863
body = rr.get_data(as_text=True)                                          8864
ok(rr.status_code == 200, "此时可打开得分明细页", "HTTP %d" %             8865
    rr.status_code)                                                       8866
ok(("%.4f" % rec.final_score) in body,                                    8867
    "报告数值取自评价记录，而非重新计算",                                 8868
    "F=%.4f" % rec.final_score)                                           8869
cleanup(pid3, "N")                                                        8870
print()                                                                   8871
print(LINE)                                                               8872
print("五、内置演示数据的清理不得波及用户自建数据")                       8873
print(LINE)                                                               8874
#  回归保护：seed_demo.clean_case() 曾执行 Project.query.delete(),        8875
#  导致用户新建的项目在每次启动时被连带删除。                             8876
import seed_demo as sd                                                    8877
with app.app_context():                                                   8878
    keep_p = Project(name="用户自建项目-守护", code="KEEP-001")           8879
    db.session.add(keep_p); db.session.flush()                            8880
    keep_patch = MinePatch(project_id=keep_p.id,                          8881
    name="用户自建图斑-守护")                                             8882
    db.session.add(keep_patch); db.session.flush()                        8883
    keep_pid = keep_p.id                                                  8884
    keep_patch_id = keep_patch.id                                         8885
    db.session.commit()                                                   8886
with app.app_context():                                                   8887
    sd.clean_case(reset=False)                                            8888
with app.app_context():                                                   8889
    still_p = db.session.get(Project, keep_pid)                           8890
    still_patch = db.session.get(MinePatch, keep_patch_id)                8891
ok(still_p is not None, "清理演示数据后，用户自建项目仍存在")             8892
ok(still_patch is not None, "清理演示数据后，用户自建图斑仍存在")         8893
#  演示数据清掉后应能重新写入，保证演示环境可恢复                        8894
sd.seed()                                                                 8895
with app.app_context():                                                   8896
    demo_back = Project.query.filter_by(code=sd.DEMO_CODE).f8897
    irst()                                                                8898
ok(demo_back is not None, "演示案例可重新写入（幂等恢复）")               8899
with app.app_context():                                                   8900
```

```
        m = db.session.get(MinePatch, keep_patch_id)
        if m:
            db.session.delete(m)
        p = db.session.get(Project, keep_pid)
        if p:
            db.session.delete(p)
        db.session.commit()
print()
print(LINE)
print("六、同一图斑不得重复划分同类型微地质单元")
print(LINE)
#  论文 4.1 节把图斑分为平台、碎石边坡、岩质边坡三类，每类至多一个。
#  允许重复会让面积占比、按类型组织的指标数据与批量导入对不上号。
pid4 = new_patch("D")
c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
        "area": "1.0"},
           follow_redirects=True)
with app.app_context():
        n1 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n1 == 1, "首次添加「平台」成功", "单元数 %d" % n1)
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "平台",
        "area": "2.0"},
                   follow_redirects=True)
with app.app_context():
        n2 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n2 == 1, "重复添加同类型被拒绝", "单元数 %d" % n2)
ok("同一类型只能划分一个" in r.get_data(as_text=True), "给出重复提示")
r = c.post("/unit/%d/add" % pid4, data={"unit_type": "火山口",
        "area": "1.0"},
                   follow_redirects=True)
with app.app_context():
        n3 = MicroUnit.query.filter_by(patch_id=pid4).count()
ok(n3 == 1, "非法单元类型被拒绝", "单元数 %d" % n3)
c.post("/unit/%d/add" % pid4, data={"unit_type": "碎石边坡",
        "area": "3.0"},
           follow_redirects=True)
with app.app_context():
        uid = MicroUnit.query.filter_by(patch_id=pid4,
                                        unit_type="碎石边坡").first(
        ).id
r = c.post("/unit/edit/%d" % uid,
                   data={"unit_type": "平台", "area": "3.0"},
        follow_redirects=True)
with app.app_context():
        types = sorted(u.unit_type for u in
                       MicroUnit.query.filter_by(patch_id=pid4).
        all())
ok(types == ["平台", "碎石边坡"], "改类型时同样拦截重复", "、".join(types))
cleanup(pid4, "D")
print()
```

```
print(LINE)
if _fails:
    print("存在未通过项：")
    for f in _fails:
        print("  -", f)
    sys.exit(1)
print("流程门禁自检全部通过 √")
print(LINE)
# ==== 测试/test_weights.py ====
# -*- coding: utf-8 -*-
"""权重生效链路自检：AHP 算出的权重必须真正参与评价计算。
此前 weight_schemes / weight_items 两张表从未被写入，评价引擎也只用
代码里的论文表 3 默认权重，相当于权重模块算完即弃。本测试守住这条链路：
    默认 → 论文表 3
    保存 AHP 权重 → 评价改用新权重，结果随之变化
    恢复默认 → 结果回到论文取值
"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
# 本脚本位于 测试/ 子目录，先把仓库根目录加入模块搜索路径，
# 使 `from app import ...` 与 app/app.db 的定位不受影响。
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
from app import create_app
from app.models import (db, Project, MinePatch, MicroUnit,
    IndicatorData,
                        ConstraintCheck, EvaluationResult,
    WeightScheme,
                        WeightItem)
from app.services import workflow as wf
from app.services import weight_service as ws
LINE = "=" * 72
BASE = ROOT
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE, "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
c = app.test_client()
c.post("/login", data={"username": "admin", "password":
    "admin123"})
_fails = []
def ok(flag, label, extra=""):
    print("  %s %-46s %s" % ("[符合]" if flag else "[不符合]",
```

```
    label, extra))
    if not flag:
        _fails.append(label)
TAG = "W"
UNIT_TYPE = "碎石边坡"
def build_case():
    """建一个只含单一微地质单元的图斑，便于观察权重变化对得分的影响。"""
    with app.app_context():
        p = Project(name="权重测试-%s" % TAG, code="WEIGHT-%s"
    % TAG)
        db.session.add(p); db.session.flush()
        patch = MinePatch(project_id=p.id, name="权重测试图斑-%s"
    % TAG)
        db.session.add(patch); db.session.flush()
        db.session.add(MicroUnit(patch_id=patch.id,
    unit_type=UNIT_TYPE,
                                 area=1.0, ratio=1.0))
        db.session.flush()
        u = MicroUnit.query.filter_by(patch_id=patch.id).fir
    st()
        db.session.add(IndicatorData(
            unit_id=u.id, soil_ratio=0.34, thickness=10,
    ph=9.24,
            organic_matter=4.89, coverage=87.92,
    shannon=0.60,
            slope=28.75, aspect="阳坡"))
        db.session.add(ConstraintCheck(
            patch_id=patch.id, policy_pass=True,
    location_pass=True,
            pollution_pass=True, geo_pass=True,
    farmland_pass=True,
            passed=True, reason="全部通过"))
        db.session.commit()
        return patch.id
def cleanup(pid):
    with app.app_context():
        from app.models import ScoreDetail
        rids = [r.id for r in EvaluationResult.query.filter_
    by(patch_id=pid)]
        if rids:
            ScoreDetail.query.filter(ScoreDetail.result_id.i
    n_(rids)).delete(
                synchronize_session=False)
        EvaluationResult.query.filter_by(patch_id=pid).delet
    e()
        ConstraintCheck.query.filter_by(patch_id=pid).delete
    ()
        uids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=pid)]
        if uids:
```

```
                IndicatorData.query.filter(IndicatorData.unit_id9051
    .in_(uids)).delete(                                          9052
                    synchronize_session=False)                   9053
        MicroUnit.query.filter_by(patch_id=pid).delete()         9054
        m = db.session.get(MinePatch, pid)                       9055
        if m:                                                    9056
            db.session.delete(m)                                 9057
        p = Project.query.filter_by(code="WEIGHT-%s" %           9058
    TAG).first()                                                 9059
        if p:                                                    9060
            db.session.delete(p)                                 9061
        db.session.commit()                                      9062
def run_eval(pid):                                               9063
    c.post("/evaluate/%d" % pid, data={},                        9064
    follow_redirects=False)                                      9065
    with app.app_context():                                      9066
        return EvaluationResult.query.filter_by(patch_id=pid9067
    ) \                                                          9068
            .order_by(EvaluationResult.id.desc()).first().fi9069
    nal_score                                                    9070
print(LINE)                                                      9071
print("一、默认应使用论文表 3 的权重")                                  9072
print(LINE)                                                      9073
with app.app_context():                                          9074
    ws.reset_weights(UNIT_TYPE)                                  9075
    w, label, scheme = ws.effective_weights(UNIT_TYPE)           9076
ok(scheme is None and "论文表" in label, "未配置时权重来源为论文表 3",      9077
    label)                                                       9078
ok(abs(w["l1"]["土壤指标"] - 0.44) < 1e-9, "一级权重取自论文表 3",          9079
    "土壤指标 %.2f" % w["l1"]["土壤指标"])                             9080
pid = build_case()                                               9081
base_score = run_eval(pid)                                       9082
print("  默认权重下的评价值 F = %.4f" % base_score)                      9083
print()                                                          9084
print(LINE)                                                      9085
print("二、保存 AHP 算出的权重后，评价应改用新权重")                          9086
print(LINE)                                                      9087
with app.app_context():                                          9088
    # 模拟 AHP 计算结果：把土壤指标权重提高、植被与地形降低（仍和为 1）          9089
    ws.save_weights(UNIT_TYPE, [("soil", 0.70), ("veg",          9090
    0.20),                                                       9091
                                ("terrain", 0.10)])              9092
    n_items = WeightItem.query.count()                           9093
ok(n_items >= 3, "weight_items 表已写入权重条目", "%d 条" % n_items)    9094
with app.app_context():                                          9095
    w2, label2, scheme2 = ws.effective_weights(UNIT_TYPE)        9096
ok(scheme2 is not None and "自定义" in label2, "权重来源变为自定义方案",     9097
    label2)                                                      9098
ok(abs(w2["l1"]["土壤指标"] - 0.70) < 1e-9, "新权重已生效",               9099
    "土壤指标 %.2f" % w2["l1"]["土壤指标"])                            9100
```

```
ok(abs(w2["l1"]["植被指标"] - 0.20) < 1e-9, "同批保存的其他项也已生效")
ok(abs(w2["l2"]["土壤指标"]["土石比"] - 0.25) < 1e-9,
   "未覆盖的二级权重仍保留论文取值",
   "土石比 %.2f" % w2["l2"]["土壤指标"]["土石比"])
new_score = run_eval(pid)
print("  新权重下的评价值 F = %.4f" % new_score)
ok(abs(new_score - base_score) > 1e-4,
   "权重变化确实改变了评价结果",
   "%.4f → %.4f" % (base_score, new_score))
print()
print(LINE)
print("三、结果应记录所用权重来源")
print(LINE)
import json
with app.app_context():
    rec = EvaluationResult.query.filter_by(patch_id=pid) \
          .order_by(EvaluationResult.id.desc()).first()
    stored = json.loads(rec.unit_scores)
ok(any(u.get("weight_source") for u in stored),
   "评价记录中标注了权重来源",
   stored[0].get("weight_source") or "")
print()
print(LINE)
print("四、恢复默认后应回到论文取值")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
    w3, label3, scheme3 = ws.effective_weights(UNIT_TYPE)
ok(scheme3 is None and "论文表" in label3, "已恢复为论文表 3 权重",
   label3)
back_score = run_eval(pid)
print("  恢复后评价值 F = %.4f（默认时 %.4f）" % (back_score,
   base_score))
ok(abs(back_score - base_score) < 1e-9, "评价结果与默认时一致")
print()
print(LINE)
print("五、权重首页概览")
print(LINE)
with app.app_context():
    rows = ws.summary()
ok(len(rows) == 3, "三种微地质单元均有权重来源说明", "%d 行" % len(rows))
r = c.get("/ahp/")
body = r.get_data(as_text=True)
ok(r.status_code == 200 and "当前生效权重" in body, "权重首页正常展示")
print()
print(LINE)
print("六、一致性不通过的矩阵不得保存生效")
print(LINE)
with app.app_context():
    ws.reset_weights(UNIT_TYPE)
```

```
n = 3
# 明显矛盾的矩阵：a12=9, a13=1/9, a23=9 —— CR 必然超限
mats = [(0, 1, "9"), (0, 2, "0.1111"), (1, 2, "9")]
data = {"csrf_token": "", "action": "save"}
for i, j, v in mats:
    data["m_%d_%d" % (i, j)] = v
r = c.post("/ahp/matrix?unit_type=%s&level=1" % UNIT_TYPE,
    data=data,
           follow_redirects=True)
with app.app_context():
    _, label4, scheme4 = ws.effective_weights(UNIT_TYPE)
ok(scheme4 is None, "CR 超限的矩阵未被保存生效", label4)
ok("一致性" in r.get_data(as_text=True), "页面给出了未通过一致性的提示")
cleanup(pid)
print()
print(LINE)
if _fails:
    print("存在未通过项：")
    for f in _fails:
        print("  -", f)
    sys.exit(1)
print("权重生效链路自检全部通过 √")
print(LINE)
# ==== app/services/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/utils/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/views/__init__.py ====
# -*- coding: utf-8 -*-
```